

Таким образом, модули-сценарии позволяют нам написать собственные функции, которые будут работать аналогично встроенным командам, автоматически загружаясь в оболочку при каждом запуске PowerShell.

Для примера запишем две функции, `Out-Hello` и `Out-GoodBye`, в файл `MyModule.psm1`. Эти функции будут выводить сообщения с именем текущего пользователя:

```
PS C:\Users\andrv> 'function Out-Hello() { "Привет, $env:UserName!" }' >
MyModule.psm1
PS C:\Users\andrv> 'function Out-GoodBye() { "Пока, $env:UserName!" }' >>
MyModule.psm1
```

Проверим содержимое файла `MyModule.psm1`:

```
PS C:\Users\andrv> Get-Content .\MyModule.psm1
function Out-Hello() { "Привет, $env:UserName!" }
function Out-GoodBye() { "Пока, $env:UserName!" }
```

В переменную `$moduleDir` сохраним каталог для хранения пользовательских модулей (первый каталог из переменной окружения `PSModulePath`):

```
PS C:\Users\andrv> $moduleDir = ($ENV:PSModulePath -split ';')[0]
```

Создадим в этом каталоге подкаталог `MyModule` (имя подкаталога должно совпадать с именем файла с модулем):

```
PS C:\Users\andrv> New-Item -Path $moduleDir/MyModule -ItemType Directory
```

Каталог: `C:\Users\andrv\OneDrive\Documents\WindowsPowerShell\Modules`

Mode	LastWriteTime	Length	Name
----	-----	-----	----
d-----	11.06.2021	6:06	MyModule

Скопируем в созданный подкаталог файл `MyModule.psm1` нашего модуля:

```
PS C:\Users\andrv> Copy-Item -Path .\MyModule.psm1 -Destination
$moduleDir\MyModule
```

Теперь наш модуль стал доступным для загрузки. Проверить это можно с помощью командлета `Get-Module` с параметром `-ListAvailable`:

```
PS C:\Users\andrv> Get-Module -ListAvailable MyModule | Format-List
```

Name	: MyModule
Path	: C:\Users\andrv\OneDrive\Documents\WindowsPowerShell\Modules\MyModule\MyModule.psm1
Description	:
ModuleType	: Script
Version	: 0.0
NestedModules	: {}
ExportedFunctions	: {Out-Hello, Out-GoodBye}
ExportedCmdlets	:

```
ExportedVariables :  
ExportedAliases  :
```

Как видим, оболочка нашла наш модуль и определила, что из него экспортируются функции `Out-Hello` и `Out-GoodBye`.

ЗАМЕЧАНИЕ

По умолчанию из модуля экспортируются все его члены (функции, переменные, псевдонимы и т. д.). Это поведение можно изменить, явно перечислив экспортируемые члены с помощью командлета `Export-ModuleMember`.

Загрузим наш модуль с помощью командлета `Import-Module`. Параметр `-Verbose` позволяет увидеть, какие функции импортируются из модуля:

```
PS C:\Users\andrv> Import-Module MyModule -Verbose  
ПОДРОБНО: Импорт функции "Out-GoodBye".  
ПОДРОБНО: Импорт функции "Out-Hello".
```

После импорта модуля `MyModule` в оболочке станут доступны функции `Out-Hello` и `Out-GoodBye`:

```
PS C:\Users\andrv> Out-Hello  
Привет, andrv!  
PS C:\Users\andrv> Out-GoodBye  
Пока, andrv!
```

При следующем запуске PowerShell модуль `MyModule` будет загружен автоматически, объявленные в нем функции останутся доступными.

Репозиторий сценариев PowerShell Gallery

По умолчанию централизованным репозиторием для модулей и сценариев PowerShell является ресурс PowerShell Gallery (<https://www.powershellgallery.com/>), которым управляет Microsoft. Информацию об этом репозитории можно получить с помощью командлета `Get-PSRepository`:

```
PS C:\Users\andrv> Get-PSRepository | Format-List
```

Name	: PSGallery
SourceLocation	: https://www.powershellgallery.com/api/v2
Trusted	: False
Registered	: True
InstallationPolicy	: Untrusted
PackageManagementProvider	: NuGet
PublishLocation	: https://www.powershellgallery.com/api/v2/package/
ScriptSourceLocation	: https://www.powershellgallery.com/api/v2/items/psscript
ScriptPublishLocation	: https://www.powershellgallery.com/api/v2/package/
ProviderOptions	: {}

Как видим, централизованный репозиторий имеет имя `PSGallery`.

Работать с репозиторием можно с помощью команд из модуля PowerShellGet:

```
PS C:\Users\andrv> Get-Command -Module PowerShellGet | Format-Wide -Column 2

Find-Command          Find-DscResource
Find-Module            Find-RoleCapability
Find-Script            Get-InstalledModule
Get-InstalledScript    Get-PSRepository
Install-Module         Install-Script
New-ScriptFileInfo    Publish-Module
Publish-Script        Register-PSRepository
Save-Module           Save-Script
Set-PSRepository       Test-ScriptFileInfo
Uninstall-Module      Uninstall-Script
Unregister-PSRepository Update-Module
Update-ModuleManifest Update-Script
Update-ScriptFileInfo
```

Поиск модулей выполняется с помощью командлета Find-Module. Если выполнить эту команду без параметров, то будет выведен полный список всех имеющихся в PowerShell Gallery модулей:

```
PS C:\Users\andrv> Find-Module -Repository PSGallery | Format-Wide -Column 2

SpeculationControl      NetworkingDsc
PSWindowsUpdate        AzureRM.profile
PackageManagement      DellBIOSProvider
PowerShellGet           Az.Accounts
Carbon                 ComputerManagementDsc
Azure.Storage           Az.Storage
Az.Resources           Az.KeyVault
Az.Automation          Az.AnalysisServices
Az.ApiManagement       xPowerShellExecutionPolicy
Az.Compute             PSLogging
Az.ApplicationInsights xCertificate
Az.Aks                 Az.Network
Az.Sql                 Az.Batch
Az.DataFactory         Az.Cdn
Az.ContainerRegistry   Az.DataLakeStore
Az.EventHub            Az.Websites
Az.ServiceBus          Az.HDInsight
Az.Monitor             Az.ContainerInstance
Az.OperationalInsights Az.DataLakeAnalytics
. . .
```

Если имя нужного модуля известно, то следует указать его в качестве значения параметра -Name. Например:

```
PS C:\Users\andrv> Find-Module -Repository PSGallery -Name PSLogging |
Format-List
```

```

Name                : PSLogging
Version             : 2.5.2
Type                : Module
Description          : Creates and manages log files for your scripts.
Author              : LucaSturlese
CompanyName         : 9to5IT
Copyright           : (c) 2015 Luca Sturlese. All rights reserved.
PublishedDate       : 11/22/2015 10:26:55 AM
InstalledDate       :
UpdatedDate         :
LicenseUri          : http://9to5it.com/powershell-logging-v2-easily-
                    create-log-files
ProjectUri          : http://9to5it.com/powershell-logging-v2-easily-
                    create-log-files
IconUri             :
Tags                : {Logging, LogFiles, PSModule}
Includes            : {Function, RoleCapability, Command, DscResource...}
PowerShellGetFormatVersion :
ReleaseNotes        : Removed HelpInfoURI from Module Manifest file as
                    was causing an issue with PowerShell 2.0
Dependencies        : {}
RepositorySourceLocation : https://www.powershellgallery.com/api/v2
Repository          : PSGallery
PackageManagementProvider : NuGet

```

Устанавливается модуль из репозитория с помощью командлета `Install-Module`. По умолчанию модули устанавливаются в каталог `C:\Program Files\WindowsPowerShell\Modules`, что требует выполнения команды `Install-Module` в оболочке, запущенной от имени администратора. Для установки модуля в профиль текущего пользователя нужно указать параметр `-Scope` со значением `CurrentUser` и утвердительно ответить на вопрос об установке:

```
PS C:\Users\andrv> Install-Module -Name PSLogging -Scope CurrentUser
```

Untrusted repository

You are installing the modules from an untrusted repository. If you trust this repository, change its `InstallationPolicy` value by running the `Set-PSRepository` cmdlet. Are you sure you want to install the modules from 'PSGallery'?

```
[Y] Yes [A] Yes to All [N] No [L] No to All [S] Suspend [?] Help (default is "N"): y
```

Для проверки загруженного модуля можно посмотреть, какие команды в нем содержатся:

```
PS C:\Users\andrv> Get-Command -Module PSLogging
```

CommandType	Name	Version
-----	----	-----
Function	Send-Log	2.5.2
Function	Start-Log	2.5.2

Function	Stop-Log	2.5.2
Function	Write-LogError	2.5.2
Function	Write-LogInfo	2.5.2
Function	Write-LogWarning	2.5.2

Обновить модуль (т. е. установить его новую версию) можно с помощью командлета `Update-Module`, удалить модуль с локальной машины — с помощью `Uninstall-Module`.

Итоги

- ❑ Функции в PowerShell создаются с помощью ключевого слова `function`.
- ❑ В отличие от других языков программирования, функция в PowerShell является командой, поэтому вызывать функции нужно как команды — аргументы указываются через пробел после имени функции.
- ❑ Самый простой способ получения переданных в функцию аргументов — перебор элементов массива `$args`.
- ❑ Для функции в PowerShell можно задать формальные параметры, указав их в круглых скобках после имени функции (как это делается в других языках программирования) или внутри самой функции с помощью специального оператора `Param()`. При этом можно явно указать типы таких параметров, задать для них значения по умолчанию и другие атрибуты для валидации аргументов, передаваемых в функцию при вызове.
- ❑ При вызове функции ее формальные параметры заменяются на фактические аргументы, которые определяются по позиции в командной строке или по имени параметра.
- ❑ Тело функции может содержать разделы `BEGIN`, `PROCESS` и `END`. В этом случае функции можно корректно использовать внутри конвейеров.
- ❑ Если для функции задать атрибут `CmdletBinding()`, то она будет вести себя как командлет. Такие функции называются расширенными, в них становятся доступными общие параметры командлетов и дополнительные выходные потоки `Verbose`, `Debug`, `Error`.
- ❑ Каждое вычисляемое в функции выражение помещает свой результат в выходной поток.
- ❑ Для управления функциями, доступными в текущем сеансе оболочки PowerShell, используется виртуальный диск `Function::`.
- ❑ Сценарий PowerShell — это функция, которая находится не в оперативной памяти, а хранится в текстовом файле с расширением `ps1`. Определение, разбор и обработка аргументов, передаваемых в сценарии, производятся так же, как и в функциях.
- ❑ Режим запуска сценариев в PowerShell определяется политикой выполнения, которую можно изменить с помощью команды `Set-ExecutionPolicy`. Рекоменду-

ется установить политику RemoteSigned, позволяющую выполнять неподписанные локальные сценарии.

- ❑ Функции в PowerShell могут иметь разные области видимости. Для того чтобы функции, объявленные в сценарии, оставались доступными в оболочке после выполнения этого сценария, можно создать эти функции в глобальной области видимости или запустить сценарий в режиме dot-sourcing.
- ❑ По умолчанию в PowerShell переменная является локальной относительно сценария, функции или блока кода, в которых она была инициализирована. Области видимости переменных можно задавать явно, указывая спецификаторы перед их именами.
- ❑ Для упрощения повторного использования кода используются модули PowerShell, содержащие командлеты, функции, переменные и псевдонимы, которые при необходимости можно загружать в оболочку и пользоваться ими из командной строки или сценариев.
- ❑ Компания Microsoft поддерживает централизованный репозиторий модулей и сценариев на языке PowerShell — PowerShell Gallery.

ГЛАВА 10



Обработка ошибок при выполнении команд

При применении сценариев для решения реальных администраторских задач (управление учетными записями пользователей, резервное копирование информации на серверах, анализ журналов событий для выявления попыток несанкционированного доступа в систему и т. п.) актуальной становится проблема обработки возможных ошибок или исключительных ситуаций. Нужно быть уверенным, что определенная задача выполнена полностью, а в случае возникновения ошибки проанализировать и устранить ее. Язык PowerShell предлагает несколько механизмов обработки ошибок, которые мы рассмотрим подробнее в данной главе.

Возникновение ошибки при выполнении команды PowerShell приводит не просто к выводу текстовых сообщений на экран, при этом автоматически генерируется настоящий объект, в свойствах которого содержится полная информация о данной ошибке.

Кроме этого, один аспект обработки ошибок в PowerShell отличает эту систему от других языков программирования: ошибки здесь могут быть *критическими* (прерывающими выполнение команды) и *некритическими* (при их возникновении выполнение команды продолжается). Это связано с тем, что для обработки объектов в PowerShell используется модель конвейера команд. При возникновении некритической ошибки информация об ошибке записывается в соответствующий объект, а текущая команда продолжает обрабатывать поступающие к ней по конвейеру объекты. Подобная ошибка может возникнуть, например, при копировании с помощью командлета множества файлов, один из которых занят другим приложением. Если же выполняется какая-то важная операция, то может иметь смысл останавливать ее после возникновения первой ошибки (тогда подобная ошибка будет критической). При этом одна и та же ошибка в одной ситуации должна считаться критической, а в другой — некритической (в PowerShell имеется механизм, позволяющий явно задавать эти типы для ошибок).

При возникновении некритических ошибок информация о них помещается в объекты типа `ErrorRecord`, которые записываются в специальный поток ошибок. Рассмотрим, каким образом можно обращаться к данным объектам и какую информацию можно из них извлечь.

Объект *ErrorRecord* и поток ошибок

В PowerShell информация о возникающих ошибках записывается в поток ошибок, который по умолчанию отображается на экране, например:

```
PS C:\Users\andr> dir "Несуществующий каталог"
dir : Не удастся найти путь "C:\Users\andr\Несуществующий каталог", т. к.
он не существует.
строка:1 знак:1
+ dir "Несуществующий каталог"
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (C:\Users\andr\Несуществующий
                           каталог:String) [Get-ChildItem],
                           ItemNotFoundException
+ FullyQualifiedErrorId : PathNotFound,Microsoft.PowerShell.Commands.
                           GetChildItemCommand
```

Поток ошибок в PowerShell имеет внутренний номер 2, этот поток можно перенаправить в текстовый файл с помощью специального оператора 2>, например:

```
PS C:\Users\andr> dir "Несуществующий каталог" 2> err.txt
```

Проверим теперь содержимое файла err.txt:

```
PS C:\Users\andr> Get-Content .\err.txt
dir : Не удастся найти путь "C:\Users\andr\Несуществующий каталог", т. к.
он не существует.
строка:1 знак:1
+ dir "Несуществующий каталог" 2> err.txt
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (C:\Users\andr\Несуществующий
                           каталог:String) [Get-ChildItem],
                           ItemNotFoundException
+ FullyQualifiedErrorId : PathNotFound,Microsoft.PowerShell.Commands.
                           GetChildItemCommand
```

Как видим, в файл err.txt полностью записалось сообщение об ошибке.

Объект *ErrorRecord*, создающийся при возникновении ошибки, можно сохранить в переменной. Для этого используется оператор 2>&1, который перенаправляет поток ошибок в стандартный выходной поток (имеющий внутренний номер 1). Например:

```
PS C:\Users\andr> $err = dir "Несуществующий каталог" 2>&1
PS C:\Users\andr> $err.GetType().FullName
System.Management.Automation.ErrorRecord
```

Проверим с помощью командлета *Get-Member*, какие свойства имеет объект типа *ErrorRecord*:

```
PS C:\Users\andr> $err | Get-Member -Type Property
```

```
TypeName: System.Management.Automation.ErrorRecord
```

Name	MemberType	Definition
----	-----	-----
CategoryInfo	Property	System.Management.Automation.Er...
ErrorDetails	Property	System.Management.Automation.Er...
Exception	Property	System.Exception Exception {get...
FullyQualifiedErrorId	Property	System.String FullyQualifiedErr...
InvocationInfo	Property	System.Management.Automation.In...
PipelineIterationInfo	Property	System.Collections.ObjectModel....
ScriptStackTrace	Property	string ScriptStackTrace {get;}
TargetObject	Property	System.Object TargetObject {get...

Эти свойства имеют отношение к возникающим ошибкам, описание наиболее важных из них приведено в табл. 10.1.

Таблица 10.1. Свойства объекта *ErrorRecord*

Свойство	Тип	Описание
CategoryInfo	System.Management.Automation.ErrorCategoryInfo	Категория ошибки
ErrorDetails	System.Management.Automation.ErrorDetails	Содержит дополнительное описание ошибки (свойство ErrorDetails.Message) Данное свойство может быть пустым
Exception	System.Exception	Исключение NET, соответствующее произошедшей ошибке
FullyQualifiedErrorId	System.String	Точное определение категории ошибки
InvocationInfo	System.Management.Automation.InvocationInfo	Объект, содержащий информацию о том, где именно произошла ошибка (обычно это имя сценария и номер строки в нем)
TargetObject	System.Object	Объект, над которым производились манипуляции во время возникновения ошибки. Данное свойство может быть пустым

Проверим содержимое переменной \$err:

```
PS C:\Users\andr> $err | Format-List * -Force
```

```
writeErrorStream      : True
PSMessageDetails      :
Exception             : System.Management.Automation.ItemNotFoundException:
                        Не удастся найти путь "C:\Users\andr\Несуществующий
                        каталог", т. к. он не существует.
                        в System.Management.Automation.SessionStateInternal.GetChildItems(String
                        path, Boolean recurse, UInt32 depth, CmdletProviderContext context)
                        в Microsoft.PowerShell.Commands.GetChildItemCommand.ProcessRecord()
```

```

TargetObject      : C:\Users\andrv\Несуществующий каталог
CategoryInfo      : ObjectNotFound: (C:\Users\andrv\Несуществующий
                    каталог:String) [Get-ChildItem], ItemNotFoundException
FullyQualifiedErrorId : PathNotFound,
                    Microsoft.PowerShell.Commands.GetChildItemCommand
ErrorDetails      :
InvocationInfo     : System.Management.Automation.InvocationInfo
ScriptStackTrace   : в <ScriptBlock>, <Нет файла>: строка 1
PipelineIterationInfo : {0, 1}

```

Как видим, в нашем примере ошибка относится к категории `PathNotFound`, свойство `TargetObject` содержит полный путь, который командлет `dir` использовал для поиска каталога или файла. Вызвана ошибка была исключением `ItemNotFoundException`.

Выясним, что содержит свойство `InvocationInfo`:

```
PS C:\Users\andrv> $err.InvocationInfo
```

```

MyCommand        : Get-ChildItem
BoundParameters  : {}
UnboundArguments : {}
ScriptLineNumber  : 1
OffsetInLine     : 8
HistoryId        : 1
ScriptName       :
Line            : $err = dir "Несуществующий каталог" 2>&1
PositionMessage  : строка:1 знак:8
                  + $err = dir "Несуществующий каталог" 2>&1
                  + ~~~~~
PSScriptRoot     :
PSCommandPath    :
InvocationName    : dir
PipelineLength    : 0
PipelinePosition  : 0
ExpectingInput    : False
CommandOrigin     : Internal
DisplayScriptPosition :

```

Свойство `ScriptName` здесь не заполнено, т. к. команда была введена непосредственно в интерактивном режиме оболочки, а не из сценария.

Итак, мы изучили структуру объекта `ErrorRecord` и узнали, какую информацию можно извлечь из его свойств. Напомним, что получили мы этот объект путем перенаправления потока ошибок в стандартный выходной поток в одной конкретной команде. Понятно, что для каждой выполняемой команды повторять эти манипуляции мы не сможем (стандартный выходной поток может понадобиться для других целей) — необходим более удобный механизм, позволяющий определять, возникла ли ошибка, и обращаться к объекту `ErrorRecord`, который соответствует той или иной ошибке. Подобные механизмы реализованы в PowerShell с помощью несколь-

ких специальных переменных и параметров, к рассмотрению которых мы сейчас перейдем.

Сохранение объектов, соответствующих ошибкам

В PowerShell имеется специальная переменная `$error`, которая содержит коллекцию (массив) объектов `ErrorRecord`, соответствующих ошибкам, возникавшим в текущем сеансе работы. Максимальное количество элементов в данной коллекции задается значением переменной `$MaximumErrorCount` (по умолчанию 256):

```
PS C:\Users\andr> $MaximumErrorCount
256
```

После заполнения массива `$error` объекты для вновь возникающих ошибок будут заменять объекты, соответствующие старым ошибкам. Возникновение каждой новой ошибки приводит к смещению элементов в массиве `$error`: объект для последней ошибки хранится в первом элементе (`$error[0]`), объект для предыдущей ошибки — во втором элементе (`$error[1]`) и т. д.

Повторим еще раз ошибку из предыдущего раздела:

```
PS C:\Users\andr> dir "Несуществующий каталог"
dir : Не удается найти путь "C:\Users\andr\Несуществующий каталог", т. к.
он не существует.
строка:1 знак:1
+ dir "Несуществующий каталог"
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (C:\Users\andr\Несуществующий
                           каталог:String) [Get-ChildItem],
                           ItemNotFoundException
+ FullyQualifiedErrorId : PathNotFound,Microsoft.PowerShell.Commands.
                           GetChildItemCommand
```

Теперь убедимся, что элемент `$error[0]` имеет тип `ErrorRecord`, и выведем содержимое этого элемента:

```
PS C:\Users\andr> $error[0].GetType().FullName
System.Management.Automation.ErrorRecord
PS C:\Users\andr> $error[0]
dir : Не удается найти путь "C:\Users\andr\Несуществующий каталог", т. к.
он не существует.
строка:1 знак:1
+ dir "Несуществующий каталог"
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (C:\Users\andr\Несуществующий
                           каталог:String) [Get-ChildItem],
                           ItemNotFoundException
+ FullyQualifiedErrorId : PathNotFound,Microsoft.PowerShell.Commands.
                           GetChildItemCommand
```

Естественно, мы можем обращаться ко всем свойствам объекта `ErrorRecord`:

```
PS C:\Users\andrv> $error[0].exception
```

Не удастся найти путь "C:\Users\andrv\Несуществующий каталог", т. к. он не существует.

```
PS C:\Users\andrv> $error[0].invocationInfo
```

```
MyCommand           : Get-ChildItem
BoundParameters     : {}
UnboundArguments    : {}
ScriptLineNumber    : 1
OffsetInLine        : 1
HistoryId           : 10
ScriptName          :
Line                : dir "Несуществующий каталог"
PositionMessage     : строка:1 знак:1
                    + dir "Несуществующий каталог"
                    + ~~~~~
PSScriptRoot        :
PSCommandPath       :
InvocationName      : dir
PipelineLength      : 0
PipelinePosition    : 0
ExpectingInput      : False
CommandOrigin       : Internal
DisplayScriptPosition :
```

```
PS C:\Users\andrv> $error[0].targetobject
```

```
C:\Users\andrv\Несуществующий каталог
```

Для перехвата ошибок определенной команды без перенаправления ошибок в стандартный выходной поток можно воспользоваться стандартным параметром `-ErrorVariable`, который определен во всех командах PowerShell. Например, в следующей команде информация об ошибках будет записываться в переменную `$errs` (обратите внимание, что при задании значения параметра `-ErrorVariable` знак `$` перед именем переменной указывать не нужно):

```
PS C:\Users\andrv> dir "Каталог 1", "Каталог 2" -ErrorVariable errs
```

```
dir : Не удастся найти путь "C:\Users\andrv\Каталог 1", т. к. он не существует.
строка:1 знак:1
```

```
+ dir "Каталог 1", "Каталог 2" -ErrorVariable errs
```

```
+ ~~~~~
```

```
+ CategoryInfo           : ObjectNotFound: (C:\Users\andrv\Каталог 1:String)
                           [Get-ChildItem], Item NotFoundException
```

```
+ FullyQualifiedErrorId : PathNotFound,Microsoft.PowerShell.Commands.
```

```
GetChildItemCommand
```

```
dir : Не удастся найти путь "C:\Users\andrv\Каталог 2", т. к. он не существует.
строка:1 знак:1
```



```
+ dir "Каталог 1", "Каталог 2" -ErrorVariable errs
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (C:\Users\andrv\Каталог 2:String)
                           [Get-ChildItem], Item NotFoundException
+ FullyQualifiedErrorId : PathNotFound,Microsoft.PowerShell.Commands.
                           GetChildItemCommand
```

При выполнении данной команды возникают две не критические ошибки, т. е. переменная `$errs` должна быть массивом, содержащим два объекта `ErrorRecord`. Проверим это:

```
C:\Users\andrv> $errs.count
2
C:\Users\andrv> $errs[0].GetType().FullName
System.Management.Automation.ErrorRecord
C:\Users\andrv> $errs[1].GetType().FullName
System.Management.Automation.ErrorRecord
C:\Users\andrv> $errs[0].targetObject
C:\Users\andrv\Каталог 1
C:\Users\andrv> $errs[1].targetObject
C:\Users\andrv\Каталог 2
```

Как видим, при указании параметра `ErrorVariable` сообщения о возникающих ошибках по-прежнему выводятся на экран. Для подавления этих сообщений нужно перенаправить поток ошибок на пустое устройство `$null`, например:

```
PS C:\Users\andrv> dir "Каталог 1", "Каталог 2" -ErrorVariable errs 2> $null
C:\Users\andrv>
```

На экран ничего не выводится, а в переменную `$errs` по-прежнему записываются два объекта:

```
PS C:\Users\andrv> $errs.count
2
```

При этом объекты, соответствующие ошибкам, записываются и в переменную `$error`:

```
PS C:\Users\andrv> $error[0]
dir : Не удастся найти путь "C:\Users\andrv\Каталог 2", т. к. он не существует.
строка:1 знак:1
+ dir "Каталог 1", "Каталог 2" -ErrorVariable errs 2> $null
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (C:\Users\andrv\Каталог 2:String)
                           [Get-ChildItem], Item NotFoundException
+ FullyQualifiedErrorId : PathNotFound,Microsoft.PowerShell.Commands.
                           GetChildItemCommand
```

```
PS C:\Users\andrv> $error[1]
dir : Не удастся найти путь "C:\Users\andrv\Каталог 1", т. к. он не существует.
строка:1 знак:1
```

```
+ dir "Каталог 1", "Каталог 2" -ErrorVariable errs 2> $null
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (C:\Users\andrv\Каталог 1:String)
                           [Get-ChildItem], Item NotFoundException
+ FullyQualifiedErrorId : PathNotFound,Microsoft.PowerShell.Commands.
                           GetChildItemCommand
```

Мониторинг возникновения ошибок

При интерактивной работе в оболочке PowerShell мы узнаем о возникновении ошибки по сообщению на экране. После этого мы можем проанализировать объект `$error[0]` и узнать всю информацию об ошибке. Но как узнать о возникновении ошибки внутри сценария?

Оказывается, в PowerShell имеется логическая переменная `$?`, которая равна `True`, если последняя выполняемая операция завершена успешно, и равна `False`, если во время выполнения последней операции возникли какие-либо ошибки.

Например, если выполнить командлет `Get-Item` для заведомо существующего каталога, то значение переменной `$?` будет равно `True`:

```
PS C:\Users\andrv> Get-Item C:\
```

Каталог:

Mode	LastWriteTime	Length	Name
----	-----	-----	----
d--hs-	01.08.2021 12:57		C:\

```
PS C:\Users\andrv> $?
```

True

Если же выполнить командлет `Get-Item` для несуществующего каталога, то значение переменной `$?` будет равно `False`:

```
PS C:\Users\andrv> Get-Item C:\АБВГГ
```

```
Get-Item : Не удастся найти путь "C:\АБВГГ", т. к. он не существует.
```

```
строка:1 знак:1
```

```
+ Get-Item C:\АБВГГ
```

```
+ ~~~~~
```

```
+ CategoryInfo          : ObjectNotFound: (C:\АБВГГ:String) [Get-Item],
                           ItemNotFoundException
+ FullyQualifiedErrorId : PathNotFound,Microsoft.PowerShell.Commands.
                           GetItemCommand
```

```
PS C:\Users\andrv> $?
```

False

Для внешних команд Windows и сценариев PowerShell определено понятие кода возврата (напомним, что для сценариев PowerShell этот код можно установить

с помощью инструкции `Exit`). В операционной системе код возврата последней команды доступен через переменную среды `%ERRORLEVEL%`; в оболочке PowerShell данный код возврата хранится в специальной переменной `$LASTEXITCODE`. При этом если код возврата равен нулю, то переменной `$?` присваивается значение `True`. Если же код возврата не равен нулю, то считается, что при выполнении данной команды произошла ошибка, и переменной `$?` присваивается значение `False`.

В качестве примера выполним команду интерпретатора `cmd.exe`, которая устанавливает нулевой код возврата. Для этого можно запустить `cmd.exe` с ключом `/c` (выполнить команду и завершить работу интерпретатора) и указать для исполнения команду `exit 0`:

```
PS C:\Users\andr> cmd /c exit 0
```

Проверим значения переменных `$LASTEXITCODE` и `$?`:

```
PS C:\Users\andr> $LASTEXITCODE
```

```
0
```

```
PS C:\Users\andr> $?
```

```
True
```

Теперь выполним команду интерпретатора `cmd.exe` с ненулевым кодом возврата (пусть, например, код возврата равен 10):

```
PS C:\Users\andr> cmd /c exit 10
```

Вновь проверим переменные `$LASTEXITCODE` и `$?`:

```
PS C:\Users\andr> $?
```

```
False
```

```
PS C:\Users\andr> $LASTEXITCODE
```

```
10
```

Итак, мы убедились, что переменными `$LASTEXITCODE` и `$?` можно пользоваться для обнаружения ошибок при выполнении сценариев PowerShell.

Режимы обработок ошибок

В начале данной главы мы говорили о том, что ошибки в PowerShell делятся на критические (прерывающие выполнение команды) и некритические (при их возникновении выполнение команды продолжается). При этом в зависимости от ситуации иногда может потребоваться рассматривать некритические ошибки как критические и наоборот. Подобное изменение типа ошибок реализовано в PowerShell с помощью различных режимов обработки ошибок (табл. 10.2).

Для установки нужного режима обработки ошибок следует присвоить соответствующее значение переменной `$ErrorActionPreference`, например:

```
$ErrorActionPreference = "SilentlyContinue"
```

В этом случае режим продолжения выполнения без выдачи сообщений об ошибке будет действовать на все команды.

Таблица 10.2. Режимы обработки ошибок в PowerShell

Режим	Символьная константа	Описание
Продолжение выполнения команды	Continue	Данный режим используется системой по умолчанию. Объект, соответствующий ошибке, записывается в выходной поток, добавляется в массив \$error, переменная \$? устанавливается в False На экран выводится сообщение об ошибке Выполнение сценария продолжается со следующей строки
Продолжение выполнения команды без выдачи сообщений об ошибке	SilentlyContinue	Если установлен данный режим, то при возникновении ошибки на экран не выводится никаких сообщений Объект, соответствующий ошибке, записывается в выходной поток, добавляется в массив \$error, переменная \$? устанавливается в False Выполнение сценария продолжается со следующей строки
Прекращение выполнения команд	Stop	Установка данного режима делает ошибки критическими Объект, соответствующий ошибке, не записывается в выходной поток, а генерирует исключение При этом переменные \$? и \$error по-прежнему обновляются Выполнение сценария прекращается
Выполнение команды с запросом	Inquire	Перед выполнением операции пользователю выдается запрос на подтверждение Здесь операцию можно продолжить, отменить или приостановить
Игнорирование ошибки	Ignore	В этом режиме возникновение ошибки игнорируется, выполнение команд продолжается Режим можно установить только в общем параметре -ErrorAction

Если нужно переключить режим обработки ошибок только для одного командлета, то при его вызове следует указать соответствующую символьную константу в качестве значения общего параметра -ErrorAction (или сокращенно — ea), например:

```
PS C:\Users\andr> Get-Item "несуществующий файл" -ErrorAction SilentlyContinue
```

Обработка критических ошибок (исключений)

Напомним, что критическими ошибками в PowerShell называются ошибки, приводящие к прекращению выполнения команды. В традиционных языках программирования такие ошибки обычно называют *исключениями* (exceptions). В PowerShell перехватывать критические ошибки, т. е. выполнять определенные действия при их возникновении, можно с помощью двух инструкций: `Trap` и `Try/Catch/Finally`.

Инструкция *Trap*

Инструкция *Trap* была реализована в первой версии PowerShell и была тогда единственным инструментом для обработки исключений.

Возможны два варианта синтаксиса *Trap*:

```
trap [тип_исключения] {блок_кода}
```

или

```
trap {блок_кода}
```

В первом случае блок кода будет выполнен только при возникновении исключения определенного типа (например, для перехвата ошибок, возникающих при попытках деления на нуль, нужно в качестве типа исключения указать *DivideByZeroException*), а во втором — при возникновении любого исключения. Приведем пример.

```
PS C:\Users\andr> trap {"Исключение перехвачено!"} $n=0; 1/$n; "n=$n"
```

Исключение перехвачено!

Попытка деления на нуль.

строка:1 знак:40

```
+ trap {"Исключение перехвачено!"} $n=0; 1/$n; "n=$n"
```

```
+
```

```
~~~~
```

```
+ CategoryInfo          : NotSpecified: (:) [], RuntimeException
```

```
+ FullyQualifiedErrorId : RuntimeException
```

```
n=0
```

В данном случае при вычислении выражения *1/\$n* возникает исключение (попытка деления на нуль) и управление передается в тело инструкции *Trap*, где выводится строка *Исключение перехвачено!*. Убедимся, что объект, соответствующий ошибке, записывается в массив *\$error* (поэтому на экран также выводится стандартное сообщение об ошибке):

```
PS C:\Users\andr> $error[0]
```

Попытка деления на нуль.

строка:1 знак:40

```
+ trap {"Исключение перехвачено!"} $n=0; 1/$n; "n=$n"
```

```
+
```

```
~~~~
```

```
+ CategoryInfo          : NotSpecified: (:) [], RuntimeException
```

```
+ FullyQualifiedErrorId : RuntimeException
```

Обратите внимание, что после выполнения тела инструкции *Trap* управление в нашем примере передалось на команду, идущую после инструкции, вызвавшей ошибку. На самом деле так происходит не всегда. Если выход из обработчика исключения производится с помощью инструкции *Break*, то оставшиеся команды выполнены не будут:

```
PS C:\Users\andr> trap {"Исключение перехвачено!"; break} $n=0; 1/$n; "n=$n"
```

Исключение перехвачено!

Попытка деления на нуль.

```

строка:1 знак:47
+ trap {"Исключение перехвачено!"; break} $n=0; 1/$n; "n=$n"
+
+ CategoryInfo          : NotSpecified: (:) [],
                                   ParentContainsErrorRecordException
+ FullyQualifiedErrorId : RuntimeException

```

Как видим, здесь отсутствует вывод на экран строки "n=0". Если выход из обработчика исключения производится с помощью инструкции `Continue`, то оставшиеся команды выполняются:

```

PS C:\Users\andr> trap {"Исключение перехвачено!"; $_.Exception ; continue}
$n=0; 1/$n; "n=$n"
Исключение перехвачено!
Попытка деления на нуль.
n=0

```

Данный пример показывает также, что при обработке исключения доступ к соответствующему ему объекту осуществляется, как обычно, с помощью переменной `$_`.

Если выход из обработчика исключения выполнен обычным образом (без применения инструкций `Break` или `Continue`), то дальнейшее поведение определяется значением переменной `$ErrorActionPreference`, рассмотренной в предыдущем разделе. Так, в следующем примере выполнение команд после обработки исключения прерывается:

```

PS C:\Users\andr> $ErrorActionPreference = "Stop"
PS C:\Users\andr> trap {"Исключение перехвачено!"} $n=0; 1/$n; "n=$n"
Исключение перехвачено!
Попытка деления на нуль.
строка:1 знак:40
+ trap {"Исключение перехвачено!"} $n=0; 1/$n; "n=$n"
+
+ CategoryInfo          : NotSpecified: (:) [],
                                   ParentContainsErrorRecordException
+ FullyQualifiedErrorId : RuntimeException

```

Изменим теперь режим обработки ошибок и еще раз выполним те же команды после инструкции `Trap`:

```

PS C:\Users\andr> $ErrorActionPreference = "SilentlyContinue"
PS C:\Users\andr> trap {"Исключение перехвачено!"} $n=0; 1/$n; "n=$n"
Исключение перехвачено!
n=0

```

В данном случае стандартное сообщение об ошибке на экран не выводится и после обработки исключения выполняется следующая команда.

Инструкция *Try/Catch/Finally*

Инструкция *Try/Catch/Finally*, аналог которой есть во многих языках программирования, поддерживается со второй версии PowerShell. Это более удобный и привычный вариант обработки исключений, чем инструкция *Trap*.

В блоке *Try* записываются команды и выражения, для которых мы хотим перехватывать возникновение критических ошибок. Этому блоку должен соответствовать хотя бы один блок *Catch* или *Finally*. Блоков *Catch* может быть несколько, в них могут проверяться типы исключений, которые нужно обработать. Если блок *Finally* указан, то код в нем будет выполнен в любом случае, независимо от возникновения исключения. Обычно в *Finally* помещают код для освобождения использованных ресурсов (закрытие подключения к базе данных, очистка сеанса удаленного доступа и т. п.).

Приведем пример обработки исключения без проверки его типа:

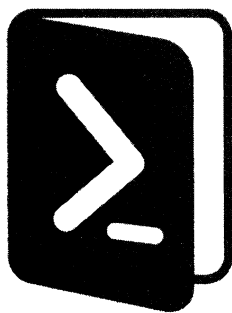
```
PS C:\Users\andrv> try {
>> 1
>> 2
>> 3/0
>> 4
>> } catch {
>> "Ошибка: $_"
>> } finally {
>> 'Завершающий блок'
>> }
1
2
Ошибка: Попытка деления на нуль.
Завершающий блок
```

В блоке *Try* здесь возникает ошибка при попытке разделить число на нуль и управление передается в блок *Catch*, где объект ошибки доступен под именем *\$_*. После отработки блока *Catch* выполняются команды из блока *Finally*.

Итоги

- ☐ В PowerShell ошибки делятся на два типа: критические, прерывающие выполнение команды, и некритические, при возникновении которых выполнение команды продолжается.
- ☐ При возникновении ошибки система генерирует объект типа *ErrorRecord*, содержащий информацию об этой ошибке.
- ☐ Объекты с информацией об ошибках направляются в стандартный поток ошибок, содержимое которого по умолчанию отображается на экране.

- ❑ Объекты `ErrorRecord`, соответствующие ошибкам, возникавшим в текущем сеансе работы, автоматически сохраняются в специальный массив `$error`. Объект для последней ошибки содержится в `$error[0]`.
- ❑ Объекты, соответствующие ошибкам, можно сохранять в другие переменные, указывая их имена в стандартном параметре `-ErrorVariable`.
- ❑ В переменной `$?` система сохраняет статус завершения последней команды.
- ❑ Код возврата внешней консольной утилиты или сценария записывается в переменную `$LASTEXITCODE`.
- ❑ В PowerShell поддерживается несколько режимов обработки ошибок команд, которые можно переключать с помощью переменной `$ErrorActionPreference` и общего параметра `-ErrorAction`.
- ❑ Критические ошибки (исключения) можно обрабатывать с помощью инструкций `Trap` и `Try/Catch/Finally`.



ЧАСТЬ III

Автоматизируем рутину

- Глава 11.** Работа с файловой системой и оболочкой Windows
- Глава 12.** Обработка структурированных данных
- Глава 13.** Управление процессами, службами и серверами автоматизации
- Глава 14.** HTTP-запросы к веб-ресурсам

ГЛАВА 11



Работа с файловой системой и оболочкой Windows

Все люди, работающие с компьютером, ежедневно выполняют множество обращений к файловой системе, читая и сохраняя файлы, создавая каталоги, копируя файлы из одного места в другое и т. п. В данной главе мы научимся манипулировать объектами файловой системы с помощью PowerShell.

ЗАМЕЧАНИЕ

Не забывайте при работе в командной строке пользоваться автодополнением путей к файлам и каталогам. Не нужно полностью набирать длинные пути — достаточно ввести первые символы, нажать клавишу <Tab>, и оболочка сама подставит подходящий путь в файловой системе.

Рассмотрим наиболее часто выполняемые операции при работе с файлами и каталогами.

Навигация в файловой системе

В оболочках cmd.exe и bash переходить по каталогам можно с помощью команды `cd`. В PowerShell команда `cd` имеет тот же смысл, при этом она является стандартным псевдонимом командлета `Set-Location`. Например, следующая команда делает текущим каталог `C:\Windows`:

```
PS C:\> cd C:\WINDOWS
PS C:\WINDOWS>
```

Как и в оболочке cmd.exe, в качестве пути можно указывать символы `..` (для перехода в родительский каталог) и `\` (для перехода в корневой каталог текущего диска). Например:

```
PS C:\WINDOWS> cd \
PS C:\>
```

Получение списка файлов и каталогов

Напомним, что в оболочке cmd.exe список файлов и каталогов формируется с помощью внутренней команды `dir`, которая имеет множество ключей, позволяющих,

например, выводить только файлы с определенными атрибутами, обрабатывать вложенные подкаталоги, сортировать список по различным полям и т. д.

В PowerShell также можно использовать команду `dir`, которая является здесь псевдонимом командлета `Get-ChildItem`. Если выполнить эту команду без параметров, то будет выведено содержимое текущего каталога:

```
PS C:\> dir
```

Каталог: C:\

Mode	LastWriteTime		Length	Name
----	-----		-----	----
d-----	03.03.2021	19:02		Far3x64
d-----	29.08.2020	22:01		Intel
d-----	13.05.2021	21:38		MRSU
d-----	07.12.2019	12:14		PerfLogs
. . .				

В пути, который указывается для команды `dir`, можно применять подстановочные знаки. Например, следующая команда выведет все файлы с расширением `log` из каталога `C:\Windows`:

```
PS C:\> dir C:\WINDOWS\*.log
```

Каталог: C:\WINDOWS

Mode	LastWriteTime		Length	Name
----	-----		-----	----
-a----	28.09.2020	16:16	762	comsetup.log
-a----	07.12.2019	12:17	776	DtcInstall.log
-a----	26.05.2021	3:20	63456	PFR0.log
-a----	11.05.2021	22:03	623	setupact.log
-a----	28.09.2020	16:13	0	setuperr.log
-a----	04.06.2021	16:14	276	WindowsUpdate.log

Параметр `-Exclude` позволяет задать маску файлов, которые не будут обрабатываться командой `dir`. Например, следующая команда выведет все файлы с расширением `log` из каталога `C:\Windows`, кроме тех, чье имя начинается на букву `d`:

```
PS C:\> dir C:\WINDOWS\*.log -exclude d*.log
```

Каталог: C:\WINDOWS

Mode	LastWriteTime		Length	Name
----	-----		-----	----
-a----	28.09.2020	16:16	762	comsetup.log
-a----	26.05.2021	3:20	63456	PFR0.log
-a----	11.05.2021	22:03	623	setupact.log
-a----	28.09.2020	16:13	0	setuperr.log
-a----	04.06.2021	16:14	276	WindowsUpdate.log

Параметр `-Name` позволяет выводить на экран только имена файлов, например:

```
PS C:\> dir C:\WINDOWS\*.log -Name
comsetup.log
DtcInstall.log
PFR0.log
setupact.log
setuperr.log
WindowsUpdate.log
```

Параметр `-Recurse` включает режим рекурсии, при котором команда `dir` действует не только на содержимое указанного каталога, но и на все его подкаталоги:

```
PS C:\> dir .\Users\ -Recurse
```

Каталог: C:\Users

Mode	LastWriteTime	Length	Name
d----	31.05.2021 7:04		andr
d-r--	28.09.2020 16:06		Public

Каталог: C:\Users\andr

Mode	LastWriteTime	Length	Name
d----	15.02.2021 20:30		.config
d----	10.10.2020 22:02		.dbus-keyrings
d----	11.02.2021 16:12		.local
. . .			

По умолчанию командлет `dir` не "видит" скрытые файлы. Если необходимо такие файлы также включать в список, то нужно указать параметр `-Force`:

```
PS C:\> dir -Force
```

Каталог: C:\

Mode	LastWriteTime	Length	Name
d--hs-	01.10.2020 14:17		\$Recycle.Bin
d--h--	28.09.2020 4:27		\$WinREAgent
d--hsl	29.08.2020 21:52		Documents and Settings
d----	03.03.2021 19:02		Far3x64
d----	29.08.2020 22:01		Intel
. . .			

У скрытых файлов в списке атрибутов (свойство `Mode`) присутствует буква `h` (атрибут `Hidden`).

Иногда бывает нужно сформировать список, состоящий только из подкаталогов или только из файлов. Для этого служат ключи `-Directory` и `-File` соответственно. Например:

```
PS C:\> dir 'C:\program files\Windows Defender\' -Directory
```

Каталог: C:\program files\Windows Defender

Mode	LastWriteTime		Length	Name
----	-----		-----	----
d-----	20.12.2020	22:19		en-US
d-----	20.12.2020	22:19		Offline
d-----	28.09.2020	16:18		platform
d-----	12.05.2021	21:23		ru-RU

```
PS C:\> dir 'C:\program files\Windows Defender\' -File
```

Каталог: C:\program files\Windows Defender

Mode	LastWriteTime		Length	Name
----	-----		-----	----
-a----	07.12.2019	12:08	9398	AmMonitoringInstall.mof
-a----	23.01.2021	8:51	204624	AMMonitoringProvider.dll
-a----	07.12.2019	12:08	21004	AmStatusInstall.mof
-a----	07.12.2019	12:08	2460	ClientWMIInstall.mof
-a----	07.12.2019	12:08	318776	ConfigSecurityPolicy.exe
-a----	07.12.2019	12:08	210872	DefenderCSP.dll
-a----	07.12.2019	12:08	733200	EppManifest.dll
-a----	07.12.2019	12:08	361	FepUnregister.mof
. . .				

Сформированный с помощью `dir` список объектов файловой системы можно сортировать по любому полю с помощью конвейера с командлетом `Sort-Object`. Например, следующая команда выведет список всех файлов и каталогов диска `C:\`, отсортированный в обратном порядке по времени обращения к ним:

```
PS C:\> dir | Sort-Object -descending LastWriteTime
```

Каталог: C:\

Mode	LastWriteTime		Length	Name
----	-----		-----	----
d-----	13.05.2021	21:38		MRSU
d-----	12.05.2021	21:23		Windows
d-r---	04.04.2021	11:19		Program Files (x86)
d-----	03.04.2021	21:39		sqlite
d-----	01.04.2021	19:17		projects
d-----	03.03.2021	19:02		Far3x64
d-r---	15.02.2021	22:15		Program Files

d-----	15.02.2021	16:09	Python39
d-----	11.02.2021	16:12	vifm
d-----	11.02.2021	16:10	Soft
d-r---	01.10.2020	14:19	Users
d-----	29.08.2020	22:01	Intel
d-----	07.12.2019	12:14	PerfLogs

Определение размера каталогов

Любому каталогу файловой системы в PowerShell соответствует объект типа `System.IO.DirectoryInfo`, который не имеет свойства, в котором хранился бы размер этого каталога. Поэтому для определения размера каталога нужно подсчитать общий размер файлов, записанных в данном каталоге и всех его подкаталогах. Сделать это можно путем суммирования с помощью командлета `Measure-Object` значений свойства `Length` объектов-файлов, поставляемых по конвейеру командлетом `dir (Get-ChildItem)`. Например, следующая команда показывает размер в байтах каталога `C:\Program Files` (свойство `Sum`) и количество файлов и подкаталогов в этом каталоге (свойство `Count`):

```
PS C:\> dir "C:\Program Files" -Recurse | Measure-Object -Property length -Sum
```

```
Count      : 21539
Average    :
Sum        : 2492086334
Maximum    :
Minimum    :
Property   : length
```

Если нужно определить размер в мегабайтах или гигабайтах, то значение свойства `Sum` следует разделить на соответствующую константу-суффикс (`1Mb` или `1Gb`):

```
PS C:\> (dir "C:\Program Files" -Recurse | Measure-Object -Property length
-Sum) .Sum/1Mb
2376,63873100281
```

Как видим, результат содержит 11 знаков после запятой. Можно округлить полученное число до двух знаков:

```
PS C:\> $FolderSize = (dir "C:\Program Files" -Recurse | Measure-Object
-Property length -Sum) .Sum/1Mb
PS C:\> [math]::round($FolderSize,2)
2376,64
```

Создание файлов и каталогов

Создать новый файл или каталог в PowerShell позволяет командлет `New-Item` (короткий псевдоним `ni`). Путь к создаваемому элементу указывается в виде значения параметра `-Path`, а в качестве значения параметра `-ItemType` указывается "directory", если нужно создать каталог, и "file", если нужно создать файл.

Например, следующая команда создает на диске C:\ каталог с именем Test:

```
PS C:\> New-Item -Path C:\Test -Type "directory"
```

Каталог: C:\

Mode	LastWriteTime	Length	Name
d----	04.06.2021 20:03		Test

Иногда бывает нужно создавать каталоги, в имени которых присутствовала бы текущая дата в определенном формате. Требуемое текстовое представление даты может быть получено с помощью параметра `-uformat` командлета `Get-Date`. Например, следующая команда создаст на диске C:\ каталог с именем, совпадающим с текущей датой в формате ГГММДД:

```
PS C:\> New-Item -Path C:\ -Name "$(Get-Date -uformat '%Y%m%d')"
-Type "directory"
```

Каталог: C:\

Mode	LastWriteTime	Length	Name
d----	04.06.2021 22:30	210604	

Если в номере года нужно указывать четыре цифры, то следует изменить спецификатор формата на `'%Y%m%d'`:

```
PS C:\> New-Item -Path C:\ -Name "$(Get-Date -uformat '%Y%m%d')"
-Type "directory"
```

Каталог: C:\

Mode	LastWriteTime	Length	Name
d----	04.06.2021 22:31	20210604	

При создании файла в него сразу можно записать строку, указав ее в качестве значения параметра `-Value`, например:

```
PS C:\> New-Item -Path C:\Test\test_file.txt -Type "file" -Value "Test"
```

Каталог: C:\Test

Mode	LastWriteTime	Length	Name
-a----	04.06.2021 22:34	4	test_file.txt

Создание нескольких файлов

В качестве значения параметра `-Path` можно указать несколько файлов через запятую, они все будут созданы:


```
PS C:\Users\andr> New-Item -Path C:\Test\test1.txt, .\test2.txt, .\test3.txt
```

Каталог: C:\Test

Mode	LastWriteTime		Length	Name
----	-----		-----	----
-a----	12.06.2021	6:28	0	test1.txt

Каталог: C:\Users\andr

Mode	LastWriteTime		Length	Name
----	-----		-----	----
-a----	12.06.2021	6:28	0	test2.txt
-a----	12.06.2021	6:28	0	test3.txt

Пересоздание файла

Если попытаться создать файл с именем уже существующего файла, то возникнет ошибка:

```
PS C:\> New-Item -Path C:\Test\test_file.txt -Type "file" -Value "Test2"
new-item : Файл "C:\Test\test_file.txt" уже существует.
строка:1 знак:1
+ new-item -path C:\Test\test_file.txt -type "file" -value "Test2 ...
+ ~~~~~
+ CategoryInfo          : WriteError: (C:\Test\test_file.txt:String)
                        [New-Item], IOException
+ FullyQualifiedErrorId : NewItemIOError,Microsoft.PowerShell.Commands.
                        NewItemCommand
```

Для пересоздания существующего файла нужно указать параметр `-Force`:

```
PS C:\> New-Item -Path C:\Test\test_file.txt -Type "file" -Value "Test2" -Force
```

Каталог: C:\Test

Mode	LastWriteTime		Length	Name
----	-----		-----	----
-a----	04.06.2021	22:37	5	test_file.txt

Создание файла в несуществующем каталоге

Параметр `-Force` также помогает в том случае, когда нам нужно создать файл в каталоге, который еще не существует. По умолчанию без этого параметра в данном случае возникнет ошибка. Например, попробуем создать файл в несуществующем подкаталоге `inner`:

```
PS C:\Users\andr> New-Item -Path C:\Test\inner\test_file.txt
New-Item : Не удалось найти часть пути "C:\Test\inner\test_file.txt".
```

```

строка:1 знак:1
+ New-Item -Path C:\Test\inner\test_file.txt
+ ~~~~~
+ CategoryInfo          : WriteError: (C:\Test\inner\test_file.txt:String)
                           [New-Item], DirectoryNotFoundException
+ FullyQualifiedErrorId : NewItemIOError,Microsoft.PowerShell.Commands.
                           NewItemCommand

```

Если указать параметр `-Force`, то подкаталог `inner` будет создан автоматически:

```
PS C:\Users\andr> New-Item -Path C:\Test\inner\test_file.txt -Force
```

```
Каталог: C:\Test\inner
```

Mode	LastWriteTime	Length	Name
----	-----	-----	----
-a----	12.06.2021	6:34	0 test_file.txt

Чтение содержимого файлов

В оболочке `cmd.exe` имеется команда `type`, которая выводит содержимое текстового файла на экран. В PowerShell команда `type` является псевдонимом командлета `Get-Content` (другие псевдонимы — `cat` и `gc`), предназначенного для построчного считывания содержимого текстового файла с возвращением объекта для каждой строки (при этом строки отображаются на экране). Например:

```
PS C:\> Get-Content C:\WINDOWS\win.ini
```

```

; for 16-bit app support
[fonts]
[extensions]
[mci extensions]
[files]
[Mail]
MAPI=1
CMCDLLNAME32=mapi32.dll
CMC=1
MAPIX=1
MAPIXVER=1.0.0.1
OLEMessaging=1

```

Параметр `-Encoding` командлета `Get-Content` позволяет явно указывать кодировку файла для корректной обработки его содержимого. Допустимые значения данного параметра: `Unicode`, `Byte`, `BigEndianUnicode`, `UTF8`, `UTF7`, `Ascii`.

По умолчанию командлет `Get-Content` считывает все строки из файла; их количество можно ограничить с помощью параметра `-TotalCount`. Например, следующая команда считывает первые пять строк из файла `C:\WINDOWS\win.ini`:

```

PS C:\> Get-Content C:\WINDOWS\win.ini -TotalCount 5
; for 16-bit app support

```

```
[fonts]
[extensions]
[mci extensions]
[files]
```

Результат выполнения командлета `Get-Content` можно сохранять в переменной, обрабатываясь в дальнейшем к ней как к массиву строк, например:

```
PS C:\> $f = Get-Content C:\WINDOWS\win.ini -TotalCount 5
PS C:\> $f.Length
5
PS C:\> $f[0]
; for 16-bit app support
PS C:\> $f[1]
[fonts]
```

Разумеется, возвращаемый массив строк можно сразу передавать по конвейеру для обработки другими командлетами, например для какого-либо преобразования и записи в другой файл (соответствующий пример приведен в следующем разделе).

Параметр `-Raw` позволяет считать все содержимое файла в одну строку, например:

```
PS C:\Users\andrv> $s = Get-Content C:\WINDOWS\win.ini -Raw
PS C:\Users\andrv> $s
; for 16-bit app support
[fonts]
[extensions]
[mci extensions]
[files]
[Mail]
MAPI=1
CMCDLLNAME32=mapi32.dll
CMC=1
MAPIX=1
MAPIXVER=1.0.0.1
OLEMessaging=1

PS C:\Users\andrv> $s.Length
167
```

Запись файлов

В главе 6 мы уже рассматривали варианты записи данных во внешние файлы с помощью операторов перенаправления (`>` и `>>`) и командлета `Out-File`. При этом командлет `Out-File` пытается форматировать записываемые в файл объекты. Если нужно просто записать в файл текстовую информацию (без дополнительного форматирования), то лучше воспользоваться командлетами `Set-Content` и `Add-Content`.

ЗАМЕЧАНИЕ

Во многих языках программирования файл для чтения или записи нужно предварительно открывать с помощью специальных операторов или функций. В PowerShell об этом беспокоиться не нужно, оболочка все делает автоматически.

Данные для записи в файл могут задаваться в качестве значения параметра `-Value`. Например, следующая команда записывает в файл `C:\Test\test.txt` строку "Строка из PowerShell":

```
PS C:\> Set-Content C:\Test\test.txt -Value "Строка из PowerShell"
```

Проверим содержимое файла `C:\Test\test.txt`:

```
PS C:\> type C:\Test\test.txt
```

```
Строка из PowerShell
```

Массив записываемых строк может приниматься по конвейеру от других командлетов. В качестве примера мы считаем пять первых строк из файла `C:\WINDOWS\win.ini` и запишем их в файл `C:\Test\num.txt` с добавлением перед каждой строкой ее порядкового номера. Для этого сначала обнулим переменную `$n` (счетчик строк):

```
PS C:\> $n = 0
```

Теперь выполним следующий конвейер команд:

```
PS C:\> Get-Content C:\WINDOWS\win.ini -TotalCount 5 |  
ForEach-Object {$n++;"$n $_"} | Set-Content C:\Test\num.txt
```

Командлет `Get-Content` в этом конвейере считывает строки из файла `C:\WINDOWS\win.ini`, которые по очереди обрабатываются командлетом `ForEach-Object`. Для каждой обрабатываемой строки значение переменной `$n` увеличивается на единицу и в выходной поток помещается значение, вычисляемое при расширении строки `"$n $_"` (значение переменной `$n` и текущая строка, поступившая по конвейеру). Затем полученная строка передается по конвейеру командлету `Set-Content`, который добавляет ее в файл `C:\Test\num.txt`.

Убедимся, что в файле `C:\Test\num.txt` записаны нужные строки:

```
PS C:\> type C:\Test\num.txt  
1 ; for 16-bit app support  
2 [fonts]  
3 [extensions]  
4 [mci extensions]  
5 [files]
```

Если файл, в который производится запись, уже существует, то командлет `Set-Content` заменит его содержимое. В случае, когда требуется добавить строки в конец существующего файла, следует воспользоваться командлетом `Add-Content`.

Копирование файлов и каталогов

В PowerShell копирование файлов и каталогов осуществляется командлетом `Copy-Item`, имеющим псевдоним `copy`. Путь к копируемым файлам при этом указывается в качестве значения параметра `-Path` (данный параметр используется по умолчанию).

нию), а путь к целевому каталогу, в который нужно скопировать файлы, задается значением параметра `-Destination`.

Например, следующая команда копирует файл `win.ini` из каталога `C:\Windows` в каталог `C:\Test`:

```
PS C:\> copy C:\Windows\win.ini -Destination C:\Test
```

По умолчанию командлет `Copy-Item` не выводит информацию на экран. Для отображения результата копирования нужно указать параметр `-PassThru`:

```
PS C:\> copy C:\Windows\win.ini -Destination C:\Test -PassThru
```

Каталог: C:\Test

Mode	LastWriteTime	Length	Name
----	-----	-----	----
-a----	21.09.2020	16:29	167 win.ini

Копирование каталога с файлами

Если путь к копируемым объектам указывает на каталог, то по умолчанию будет скопирован только этот каталог без своего содержимого (этим PowerShell отличается от большинства других оболочек, в том числе от `cmd.exe`).

Например, создадим каталог `C:\Script` с тремя файлами `1.ps1`, `2.ps1` и `3.txt`:

```
PS C:\Users\andr> New-Item C:\Script\1.ps1, C:\Script\2.ps1, C:\Script\3.txt -Force
```

Каталог: C:\Script

Mode	LastWriteTime	Length	Name
----	-----	-----	----
-a----	11.06.2021	23:23	0 1.ps1
-a----	11.06.2021	23:23	0 2.ps1
-a----	11.06.2021	23:23	0 3.txt

Теперь скопируем каталог `C:\Script` в `C:\Test`:

```
PS C:\Users\andr> copy C:\Script -Destination C:\Test -PassThru
```

Каталог: C:\Test

Mode	LastWriteTime	Length	Name
----	-----	-----	----
d-----	11.06.2021	23:25	Script

Как видим, скопировался только сам каталог без находящихся в нем файлов.

Копирование вложенных каталогов

Копировать содержимое вложенных каталогов позволяет параметр `-Recurse`, например:

```
PS C:\Users\andrv> copy C:\Script -Destination C:\Test -Recurse -PassThru
```

Каталог: C:\Test\Script

Mode	LastWriteTime		Length	Name
----	-----		-----	----
-a----	11.06.2021	23:25	0	1.ps1
-a----	11.06.2021	23:25	0	2.ps1
-a----	11.06.2021	23:25	0	3.txt

Копирование файлов по маске

Можно копировать не все файлы из каталога, а только соответствующие определенной маске. При этом маску можно указать внутри пути для копирования или в качестве значения параметра `-Include`. Например, следующая команда копирует все файлы с расширением `ps1` из каталога `C:\Script` в папку `C:\Test`:

```
PS C:\Users\andrv> copy C:\Script\*.ps1 -Destination C:\Test -PassThru
```

Каталог: C:\Test

Mode	LastWriteTime		Length	Name
----	-----		-----	----
-a----	11.06.2021	23:25	0	1.ps1
-a----	11.06.2021	23:25	0	2.ps1

Однако если необходимо скопировать файлы по маске из всех вложенных подкаталогов, то одним командлетом `Copy-Item` обойтись не удастся. Предварительно требуемые файлы нужно выделить командлетом `Get-ChildItem (dir)`, а затем передать их командлету `Copy-Item` по конвейеру.

Например, создадим два файла в подкаталогах `Posh` и `Misc`:

```
PS C:\Users\andrv> New-Item C:\Script\Posh\4.ps1, C:\Script\Misc\5.ps1 -Force
```

Каталог: C:\Script\Posh

Mode	LastWriteTime		Length	Name
----	-----		-----	----
-a----	12.06.2021	6:39	0	4.ps1

Каталог: C:\Script\Misc

Mode	LastWriteTime		Length	Name
----	-----		-----	----
-a----	12.06.2021	6:39	0	5.ps1

Следующая команда копирует все файлы с расширением `ps1` из каталога `C:\Script` и всех его подкаталогов в папку `C:\Test`:

```
PS C:\Users\andrv> dir -Recurse -Include *.ps1 C:\script\* |
copy -Destination C:\Test -PassThru
```

Каталог: C:\Test

Mode		LastWriteTime		Length	Name
----		-----		-----	----
-a----		12.06.2021	6:39	0	5.ps1
-a----		12.06.2021	6:39	0	4.ps1
-a----		11.06.2021	23:20	0	1.ps1
-a----		11.06.2021	23:20	0	2.ps1

Конкатенация файлов

Команда `copy` оболочки `cmd.exe` позволяла объединять несколько файлов вместе (конкатенация файлов). В PowerShell объединить файлы можно с помощью командлета `Get-Content` (псевдоним `type`) и перенаправления вывода в результирующий файл. Рассмотрим пример. Создадим файлы `C:\Test\1.txt` и `C:\Test\2.txt`:

```
PS C:\Users\andrv> New-Item -Path C:\Test\1.txt -Type "file" -Value "File 1"
```

Каталог: C:\Test

Mode		LastWriteTime		Length	Name
----		-----		-----	----
-a----		12.06.2021	6:56	6	1.txt

```
PS C:\Users\andrv> New-Item -Path C:\Test\2.txt -Type "file" -Value "File 2"
```

Каталог: C:\Test

Mode		LastWriteTime		Length	Name
----		-----		-----	----
-a----		12.06.2021	6:56	6	2.txt

Следующая команда объединяет файлы `1.txt` и `2.txt` в файл `3.txt`:

```
PS C:\Users\andrv> type C:\Test\1.txt, C:\Test\2.txt > C:\Test\3.txt
```

Проверим содержимое файла `3.txt`:

```
PS C:\Users\andrv> type C:\Test\3.txt
```

File 1

File 2

Как видим, конкатенация файлов `1.txt` и `2.txt` выполнена успешно.

Переименование и перемещение файлов и каталогов

Переименовать файл или каталог можно с помощью командлета `Rename-Item` (псевдоним `ren`). Значение параметра `-Path` этого командлета задает путь к элементам для переименования, а значение параметра `-NewName` — новое имя. Имена этих параметров можно опускать (в этом случае первым должно указываться значение параметра `-Path`). Например, создадим файл `C:\1.tmp` и переименуем его в файл `2.tmp`:

```
PS C:\Users\andr> New-Item -Path C:\Test\1.tmp
```

```
Каталог: C:\Test
```

Mode	LastWriteTime	Length	Name
----	-----	-----	----
-a----	12.06.2021 7:01	0	1.tmp

```
PS C:\Users\andr> ren C:\Test\1.tmp 2.tmp
```

Для того чтобы увидеть результат действия командлета `Rename-Item`, нужно указать параметр `-PassThru`:

```
PS C:\Users\andr> ren C:\Test\2.tmp 3.tmp -PassThru
```

```
Каталог: C:\Test
```

Mode	LastWriteTime	Length	Name
----	-----	-----	----
-a----	12.06.2021 7:01	0	3.tmp

Переименование группы файлов

В отличие от команды `ren` оболочки `cmd.exe`, командлет `Rename-Item` не поддерживает подстановочные знаки в значении параметров `-Path` или `-NewName`. Поэтому для переименования группы файлов их нужно сначала получить с помощью командлета `Get-ChildItem` (`dir`), а потом передать по конвейеру командлету `Rename-Item`.

В качестве примера мы переименуем все файлы с расширением `txt` в каталоге `C:\Test`, назначив им новое расширение — `md`:

```
PS C:\Users\andr> dir C:\Test\*.txt | Rename-Item -NewName { $_.Name -replace '.txt',' .md' }
```

Перемещение файлов

Командлет `Rename-Item` позволяет переименовывать файлы или каталоги только внутри того каталога, в котором находился исходный объект. Если нужно переместить файл или каталог в другую папку, то следует воспользоваться командлетом

`Move-Item` (псевдоним `move`). Значение параметра `-Path` данного командлета задает путь к файлам или каталогам для перемещения (в этом пути допускается использование подстановочных знаков), а значение параметра `-Destination` — путь к каталогу, куда будут перемещены эти файлы или каталоги. Результат перемещения можно увидеть на экране, указав параметр `-PassThru`. Например, следующая команда переносит в корневой каталог диска `C:\` каталог `C:\Test\inner` со всем его содержимым:

```
PS C:\Users\andr> Move-Item -Path C:\Test\inner C:\ -PassThru
```

Каталог: C:\

Mode	LastWriteTime	Length	Name
d----	12.06.2021 6:34		inner

Вернем каталог `inner` на место:

```
PS C:\Users\andr> Move-Item -Path C:\inner C:\Test -PassThru
```

Каталог: C:\Test

Mode	LastWriteTime	Length	Name
d----	12.06.2021 6:34		inner

Удаление файлов и каталогов

Удалять объекты файловой системы можно с помощью командлета `Remove-Item` (псевдоним `del`). Значение параметра `-Path` этого командлета задает путь к удаляемым файлам или каталогам (имя параметра в команде можно не указывать). В пути допускаются подстановочные символы, также командлет `Remove-Item` имеет параметр `-Include`, значение которого задает файлы, на которые будет действовать команда, и параметр `-Exclude`, задающий файлы-исключения, которые удаляться не будут.

Например, следующая команда удалит все файлы с расширением `ps1` в каталоге `C:\Test`:

```
PS C:\> del C:\Test\*.ps1
```

Если попытаться удалить все файлы в каталоге, имеющем подкаталоги, то система выдаст предупреждение:

```
PS C:\Users\andr> del C:\Test\*
```

Подтверждение

Элемент в `C:\Test\inner` имеет дочерние объекты, и параметр `Recurse` не указан. При продолжении все дочерние объекты будут удалены вместе с элементом. Вы действительно хотите продолжить?

[Y] Да - Y [A] Да для всех - A [N] Нет - N [L] Нет для всех - L [S]
 Приостановить - S [?] Справка
 (значением по умолчанию является "Y"):

Для удаления без предупреждения всех элементов в каталоге, включая подкаталоги, следует указать параметр `-Recurse`:

```
PS C:\> del C:\Test\* -Recurse
```

Поиск текста в файлах

В принципе, для поиска текста в файле можно считать его содержимое в массив строк с помощью командлета `Get-Content` и применить к ним один из операторов сравнения, описанных в главе 8. Однако в PowerShell имеется командлет `Select-String`, специально предназначенный для поиска строк текста в одном или нескольких файлах (это аналог консольной утилиты `findstr` для Windows и `grep` для UNIX-систем).

Например, следующая команда ищет слово `Error` во всех файлах с расширением `log` в системном каталоге Windows (имена параметров `-Pattern` и `-Path` можно опустить, не забывая при этом, что шаблон поиска должен стоять после имени командлета первым):

```
PS C:\Users\andrv> Select-String -Pattern Error -Path $env:windir\*.log
```

```
C:\WINDOWS\DtcInstall.log:1:12-07-2019 09:17 : DTC Install error = 0,
Enter MsDtcAdvancedInstaller::Configure, base\wcp
\plugins\msdtc\msdtcadvancedinstaller\msdtcadvancedinstaller.cpp (367)
C:\WINDOWS\DtcInstall.log:2:12-07-2019 09:17 : DTC Install error = 0,
Action: None, base\wcp\plugins\msdtc\msdtcadvance
dinstaller\msdtcadvancedinstaller.cpp (396)
. . .
```

Как видим, на экран по умолчанию выводятся: путь к проверяемому файлу, номер строки, в которой найдено совпадение, и сама эта строка. На самом деле результат поиска является объектом типа `Microsoft.PowerShell.Commands.MatchInfo`, содержащим свойства `Path` (путь к файлу), `Pattern` (шаблон, по которому производился поиск), `LineNumber` (номер найденной строки в файле), `Line` (содержимое найденной строки) и другие:

```
PS C:\Users\andrv> Select-String Error $env:windir\*.log | Format-List *
```

```
IgnoreCase : True
LineNumber : 1
Line       : 12-07-2019 09:17 : DTC Install error = 0, Enter
              MsDtcAdvancedInstaller::Configure, base\wcp\plugins\msdtc\
              msdtcadvancedinstaller\msdtcadvancedinstaller.cpp (367)
Filename   : DtcInstall.log
Path       : C:\WINDOWS\DtcInstall.log
Pattern    : Error
```

```
Context      :
Matches      : {0}

IgnoreCase   : True
LineNumber   : 2
Line         : 12-07-2019 09:17 : DTC Install error = 0, Action: None,
              base\wcp\plugins\msdtc\msdtcadvancedinstaller\msdt
              cadvancedinstaller.cpp (396)
Filename     : DtcInstall.log
Path         : C:\WINDOWS\DtcInstall.log
Pattern      : Error
Context      :
Matches      : {0}
. . .
```

Если при поиске нужно учитывать регистр символов, то следует указать параметр `-CaseSensitive`.

По умолчанию командлет `Select-String` находит в файлах все соответствия для строк. В случае необходимости можно ограничиться только первым найденным соответствием в каждом файле, указав параметр `-List`, например:

```
PS C:\Users\andr> Select-String Error $env:windir\*.log -List
```

```
C:\WINDOWS\DtcInstall.log:1:12-07-2019 09:17 : DTC Install error = 0,
Enter MsDtcAdvancedInstaller::Configure, base\wcp
\plugins\msdtc\msdtcadvancedinstaller\msdtcadvancedinstaller.cpp (367)
```

Если нужно определить сам факт наличия искомой строки в файлах и неважно, в каком именно файле и на какой позиции находится эта строка, то можно использовать параметр `-Quiet`. В этом случае командлет `Select-String` возвращает `True`, если поиск завершен успешно, и `False` — в противном случае:

```
PS C:\Users\andr> Select-String Error $env:windir\*.log -Quiet
True
```

В качестве шаблона для поиска можно использовать регулярные выражения. Например, следующая команда будет искать в файлах с расширением `log` в системном каталоге Windows строки, содержащие слова `Error` или `Setup`:

```
PS C:\Users\andr> Select-String -pattern '(Error|Setup)' $env:windir\*.log
```

```
C:\WINDOWS\comsetup.log:2:COM+[4:16:25]: Setup started - [DATE:09,28,2020 TIME:
04:16 pm]
C:\WINDOWS\comsetup.log:8:COM+[4:16:28]: Setup (COMMIG) finished
- [DATE:09,28,2020 TIME: 04:16 pm]
C:\WINDOWS\DtcInstall.log:1:12-07-2019 09:17 : DTC Install error = 0,
Enter MsDtcAdvancedInstaller::Configure, base\wcp
\plugins\msdtc\msdtcadvancedinstaller\msdtcadvancedinstaller.cpp (367)
C:\WINDOWS\DtcInstall.log:2:12-07-2019 09:17 : DTC Install error = 0,
Action: None, base\wcp\plugins\msdtc\msdtcadvan
cedinstaller\msdtcadvancedinstaller.cpp (396)
. . .
```

ЗАМЕЧАНИЕ

Подробное рассмотрение языка регулярных выражений выходит за рамки данной книги. Дополнительную информацию о некоторых аспектах регулярных выражений можно найти во встроенной справке PowerShell (команда `Get-Help about_regular_expression`); более подробные справочные материалы имеются на сайте Microsoft (<https://docs.microsoft.com/ru-ru/dotnet/standard/base-types/regular-expressions>).

Файлы, в которых производится поиск, могут не только указываться в качестве значения параметра `-Path`, но и поступать командлету `Select-String` по конвейеру. Например, следующая команда ищет слово `Microsoft` во всех файлах с расширением `txt`, хранящихся в каталоге `Windows\System32` и всех его подкаталогах:

```
PS C:\Users\andrv> dir $env:windir\System32\* -Include *.txt -Recurse |
Select-String -Pattern Microsoft -CaseSensitive

C:\WINDOWS\system32\catroot2\dberr.txt:1:CatalogDB: 20:43:37 08.06.2021:
DONE Adding Catalog File (78ms): Microsoft-Win
dows-Client-Desktop-Required-
Package04113~31bf3856ad364e35~amd64~~10.0.19041.1023.cat
C:\WINDOWS\system32\catroot2\dberr.txt:2:CatalogDB: 20:43:38 08.06.2021:
DONE Adding Catalog File (125ms): Microsoft-Wi
ndows-Client-Desktop-Required-WOW64-
Package~31bf3856ad364e35~amd64~~10.0.19041.1052.cat
C:\WINDOWS\system32\catroot2\dberr.txt:3:CatalogDB: 20:43:38 08.06.2021:
DONE Adding Catalog File (78ms): Microsoft-Win
dows-Client-Desktop-Required-WOW64-
Package00~31bf3856ad364e35~amd64~~10.0.19041.1052.cat
. . .
```

Также по конвейеру можно передать массив строк, в котором будет производиться поиск, например:

```
PS C:\Users\andrv> "Windows","PowerShell" | Select-String -Pattern [PR]owerShell

PowerShell
```

Замена текста в файлах

Напомним, что замена текста в строках осуществляется оператором `-replace`, поддерживающим регулярные выражения (см. гл. 8). Поэтому для замены текста в файле можно считать его содержимое в массив строк с помощью командлета `Get-Content`, обработать нужные строки оператором `-replace` и сохранить результат в том же или другом файле, используя командлет `Set-Content`.

Рассмотрим простой пример. Создадим файл `C:\Test\test.txt` из трех строк:

```
PS C:\> "11111", "2222", "33333" > C:\Test\test.txt
PS C:\> type C:\Test\test.txt
11111
2222
33333
```

Теперь заменим в этом файле все символы "l" на "a":

```
PS C:\> (Get-Content C:\Test\test.txt) | ForEach-Object {$_ -replace "l", "a"} | Set-Content C:\Test\test.txt
```

Первая команда в конвейере здесь взята в скобки для того, чтобы командлет Set-Content начал работать лишь после того, как командлет Get-Content полностью прочитает файл (в противном случае может возникнуть ошибка при одновременном обращении к файлу C:\Test\test.txt при чтении и записи). Убедимся, что замена прошла успешно:

```
PS C:\> type C:\Test\test.txt
aaaaa
2222
33333
```

Работа с файлами-ярлыками

Ярлыки в Windows — это особый тип файлов, которые имеют расширение lnk или url и служат указателями на другие объекты (файлы, каталоги, веб-ресурсы, приложения, команды). В PowerShell нет отдельного встроенного командлета для работы с ярлыками, но можно создать ярлык с помощью методов внешних COM-объектов, имеющихся в операционной системе.

Доступ к COM-объектам из PowerShell

Технология Component Object Model (COM), разработанная компанией Microsoft еще в 1993 г., предоставляет двоичный (не зависящий от языка программирования) стандарт для программных компонентов. Объекты, созданные по этому стандарту и зарегистрированные определенным образом в операционной системе, могут использоваться в других приложениях без применения языков, средств или систем программирования, а только с помощью двоичных компонентов (исполняемых файлов или динамических библиотек). COM-объекты до настоящего времени продолжают использоваться в Windows и в приложениях Microsoft.

Идентифицируются COM-объекты по их *программным идентификаторам* (ProgID) — символьным псевдонимам, назначаемым при регистрации объектов в системе. ProgID имеют следующий вид:

Библиотека_типов.Класс

Перед точкой в ProgID стоит имя *библиотеки типов* (type library) для объекта, которая может существовать как в виде отдельного файла с расширением tlb, так и в виде части файла с исполняемым кодом объекта (библиотека типов, содержащая сведения о COM-объекте, регистрируется в системном реестре при установке приложения, публикующего этот объект). Довольно часто имя библиотеки типов совпадает с именем приложения, являющегося сервером COM-объектов.

После точки в ProgID указывается имя класса, содержащего свойства и методы COM-объекта, доступные для использования другими приложениями. Примеры

ProgID: InternetExplorer.Application (приложение Internet Explorer) Word.Application (приложение Microsoft Word), WScript.Shell (класс Shell из объектной модели сервера сценариев Windows Script Host).

В PowerShell имеется командлет New-Object, позволяющий, в частности, создавать экземпляры внешних COM-объектов, указывая соответствующий ProgID в качестве значения параметра ComObject. Например, экземпляр COM-объекта с программным идентификатором WScript.Shell создается следующим образом:

```
PS C:\> $shell = New-Object -ComObject WScript.Shell
```

Выполняя командлет New-Object, интерпретатор PowerShell через ProgID получает из системного реестра путь к файлам нужной библиотеки типов. Затем с помощью этой библиотеки в память загружается экземпляр запрашиваемого объекта, и его интерфейсы становятся доступными для использования в PowerShell. Ссылка на созданный объект сохраняется в переменной; в дальнейшем, используя эту переменную, мы получаем доступ к свойствам и методам объекта, а также к его вложенным объектам (если они имеются).

ЗАМЕЧАНИЕ

"Общение" с COM-объектами в PowerShell происходит с помощью соответствующих механизмов .NET Framework (создаются экземпляры .NET-класса System.__ComObject), поэтому на командлет New-Object действуют те же ограничения, какие действуют на платформу .NET во время вызова COM-объектов.

Объект WScript.Shell

Посмотрим, какие свойства и методы имеются у COM-объекта WScript.Shell. Для этого воспользуемся, как обычно, командлетом Get-Member, передав ему по конвейеру переменную \$Shell, в которой сохранена ссылка на данный COM-объект:

```
PS C:\> $shell | Get-Member
```

TypeName: System.__ComObject#{41904400-be18-11d3-a28b-00104bd350...

Name	MemberType	Definition
----	-----	-----
AppActivate	Method	bool AppActivate (...
CreateShortcut	Method	IDispatch CreateSh...
Exec	Method	IWshExec Exec (str...
ExpandEnvironmentStrings	Method	string ExpandEnvir...
LogEvent	Method	bool LogEvent (Var...
Popup	Method	int Popup (string,...
RegDelete	Method	void RegDelete (st...
RegRead	Method	Variant RegRead (s...
RegWrite	Method	void RegWrite (str...
Run	Method	int Run (string, V...
SendKeys	Method	void SendKeys (str...
Environment	ParameterizedProperty	IWshEnvironment En...

CurrentDirectory	Property	string CurrentDire...
SpecialFolders	Property	IWshCollection Spe...

Как видим, здесь есть методы для запуска и управления приложениями, работы с системным реестром и журналами событий. Нам понадобится метод `CreateShortcut`, позволяющий создавать ярлыки для папок и файлов, и коллекция `SpecialFolders`, содержащая пути к специальным папкам Windows для текущего пользователя.

Создание ярлыка на рабочем столе

Для примера мы создадим на рабочем столе активного пользователя ярлык `PSHome.lnk` для папки, в которой установлена оболочка PowerShell.

Первая задача — определить путь к рабочему столу текущего пользователя. Для этого мы воспользуемся коллекцией `SpecialFolders`, сохранив ссылку на нее в переменной `$folders`:

```
PS C:\> $folders = $shell.SpecialFolders
PS C:\> $folders
C:\Users\Public\Desktop
C:\ProgramData\Microsoft\Windows\Start Menu
C:\ProgramData\Microsoft\Windows\Start Menu\Programs
C:\ProgramData\Microsoft\Windows\Start Menu\Programs\Startup
C:\Users\andrv\OneDrive\Рабочий стол
C:\Users\andrv\AppData\Roaming
C:\Users\andrv\AppData\Roaming\Microsoft\Windows\Printer Shortcuts
C:\Users\andrv\AppData\Roaming\Microsoft\Windows\Templates
C:\Windows\Fonts
C:\Users\andrv\AppData\Roaming\Microsoft\Windows\Network Shortcuts
C:\Users\andrv\OneDrive\Рабочий стол
C:\Users\andrv\AppData\Roaming\Microsoft\Windows\Start Menu
C:\Users\andrv\AppData\Roaming\Microsoft\Windows\SendTo
C:\Users\andrv\AppData\Roaming\Microsoft\Windows\Recent
C:\Users\andrv\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup
C:\Users\andrv\Favorites
C:\Users\andrv\OneDrive\Documents
C:\Users\andrv\AppData\Roaming\Microsoft\Windows\Start Menu\Programs
```

Путь к рабочему столу пользователя сохраним в переменной `$desktop`:

```
PS C:\> $desktop = $folders.Item('desktop')
PS C:\> $desktop
C:\Users\andrv\OneDrive\Рабочий стол
```

Для создания ярлыка на рабочем столе вызовем метод `CreateShortcut`, передав в него путь к ярлыку (напомним, что внутри строки в двойных кавычках имя переменной заменяется на ее значение):

```
PS C:\> $lnk = $shell.CreateShortcut("$desktop\PSHome.lnk")
PS C:\> $lnk = $Shell.CreateShortcut("$Home\Рабочий стол\PSHome.lnk")
```

Посмотрим, какие свойства и методы имеет объект, сохраненный в переменной \$lnk:

```
PS C:\> $lnk | Get-Member

TypeName: System.__ComObject#{f935dc23-1cf0-11d0-adb9-00c04fd58a0b}
```

Name	MemberType	Definition
-----	-----	-----
Load	Method	void Load (string)
Save	Method	void Save ()
Arguments	Property	string Arguments () {get} {set}
Description	Property	string Description () {get} {set}
FullName	Property	string FullName () {get}
Hotkey	Property	string Hotkey () {get} {set}
IconLocation	Property	string IconLocation () {get} {set}
RelativePath	Property	string RelativePath () {set}
TargetPath	Property	string TargetPath () {get} {set}
WindowStyle	Property	int WindowStyle () {get} {set}
WorkingDirectory	Property	string WorkingDirectory () {get} {set}

Для создания ярлыка на папку достаточно сохранить путь к ней в свойстве TargetPath и вызвать метод Save для сохранения ярлыка. Путь к домашней папке PowerShell хранится в специальной переменной \$PSHome, поэтому выполняем следующие команды:

```
PS C:\> $lnk.TargetPath = $PSHome
PS C:\> $lnk.Save()
```

Задача решена, ярлык на рабочем столе активного пользователя создан.

Удаление некорректных ярлыков

Ярлыки (файлы с расширением lnk) являются важной частью графического интерфейса Windows. Однако если объект, на который ссылается ярлык, удалить или переместить, то соответствующий ярлык станет бесполезным.

Давайте разберем команды, с помощью которых можно найти и удалить такие некорректные ярлыки на рабочем столе. Как и в предыдущем разделе, создадим экземпляр класса WScript.Shell (переменная \$shell):

```
PS C:\> $shell = New-Object -ComObject WScript.Shell
```

В переменную \$desktop поместим путь к рабочему столу активного пользователя:

```
PS C:\> $folders = $shell.SpecialFolders
PS C:\> $desktop = $folders.Item("desktop")
```

Теперь с помощью конвейера командлетов dir (Get-ChildItem) и ForEach-Object переберем все файлы на рабочем столе с расширением lnk. Для каждого такого файла будем создавать с помощью метода CreateShortcut соответствующий объ-

ект-ярлык и проверять с помощью командлета `Test-Path` наличие объекта, на который ссылается этот ярлык (путь к этому объекту хранится в свойстве `TargetPath` объекта-ярлыка). Если объект не существует (командлет `Test-Path` возвращает `False`), то проверяемый файл с расширением `lnk` удаляется:

```
PS C:\> dir "$desktop\*.lnk" | ForEach-Object{
>> $shortcut = $shell.CreateShortcut($_.FullName);
>> if (-not (Test-Path $shortcut.TargetPath)) {del $_.FullName};
>> }
>>
```

Итоги

- ☐ В PowerShell имеются командлеты для выполнения стандартных операций над файлами и каталогами.
- ☐ Для создания файлов-ярлыков и управления ими можно использовать COM-объект `WScript.Shell`. Работать с COM-объектами позволяет командлет `New-Object` с параметром `ComObject`.

ГЛАВА 12



Обработка структурированных данных

В информационных системах повсюду используются данные, имеющие определенную структуру (конфигурационные файлы, журналы операций, данные от API внешних систем и т. д.). В PowerShell имеются встроенные средства для работы с наиболее распространенными форматами таких данных.

Работа с данными в формате CSV

В самом простом случае структурированный файл содержит табличные данные и представляет собой набор строк, состоящих из отдельных полей, разделенных друг от друга определенным символом. Часто в качестве разделителя выбирается запятая, поэтому подобные файлы называют *CSV-файлами* (Comma-Separated Values — значения, разделенные запятыми).

В первой строке CSV-файла могут быть перечислены названия полей, а в остальных содержатся сами данные. Например:

```
Name, Lastname, Age
Иван, Петров, 22
Сергей, Андреев, 34
Ольга, Белова, 25
Мария, Никитина, 41
```

Рассмотрим, каким образом с помощью PowerShell можно читать и изменять данные в CSV-файлах.

Чтение из CSV-файла

Для рассмотрения примеров создадим в каталоге C:\Script файл test.csv со следующими анкетными данными (имя, фамилия и возраст):

```
Name, Lastname, Age
Иван, Петров, 22
Сергей, Андреев, 34
Ольга, Белова, 25
Мария, Никитина, 41
```

В принципе, содержимое CSV-файла, как и любого другого текстового файла, можно считать в массив строк с помощью командлета `Get-Content`.

```
PS C:\Script> $names = Get-Content .\test.csv
PS C:\Script> Get-Member -InputObject $names
```

```
TypeName: System.Object[]
```

```
...
```

```
PS C:\Script> $names | Get-Member
```

```
TypeName: System.String
```

```
...
```

```
PS C:\Script> $names
```

```
Name, Lastname, Age
```

```
Иван, Петров, 22
```

```
Сергей, Андреев, 34
```

```
Ольга, Белова, 25
```

```
Мария, Никитина, 41
```

Однако командлет `Get-Content` ничего не знает о внутренней структуре строк, поэтому их содержимое придется разбирать (парсить), выделяя из строк составные части.

Поэтому для парсинга структурированных данных из CSV-файла лучше воспользоваться специальным командлетом `Import-Csv`, который сразу преобразовывает данные из файла в массив объектов с нужными полями.

Считаем с помощью `Import-Csv` данные из файла `test.csv` в переменную `$names` и выведем ее содержимое:

```
PS C:\Script> $names = Import-Csv -Path .\test.csv
PS C:\Script> $names
```

```
Name      Lastname Age
```

```
-----
```

```
Иван      Петров  22
```

```
Сергей    Андреев 34
```

```
Ольга     Белова  25
```

```
Мария     Никитина 41
```

Как видим, командлет `Import-Csv` понял, что в первой строке файла записан заголовок, задающий структуру каждой строки. Как и в случае с `Get-Content`, переменная `$names` является массивом объектов:

```
PS C:\Script> Get-Member -InputObject $names
```

```
TypeName: System.Object[]
```

Name	MemberType	Definition
----	-----	-----
Count	AliasProperty	Count = Length

```
Add          Method          int IList.Add(System...
Address      Method          System.Object&, msco...
. . .
```

Однако теперь каждый элемент массива \$names — это не простая строка, а объект типа PSCustomObject, имеющий поля Name, LastName и Age:

```
PS C:\Script> $names | Get-Member

TypeName: System.Management.Automation.PSCustomObject
```

Name	MemberType	Definition
-----	-----	-----
Equals	Method	bool Equals(System.Object obj)
GetHashCode	Method	int GetHashCode()
GetType	Method	type GetType()
ToString	Method	string ToString()
Age	NoteProperty	string Age=22
LastName	NoteProperty	string Lastname=Петров
Name	NoteProperty	string Name=Иван

Имя такой массив, мы легко можем выполнять операции поиска, сортировки или группировки данных с помощью командлетов, рассмотренных в главе 5. Например, отсортируем записи из нашей таблицы по возрасту:

```
PS C:\Script> $names | Sort-Object Age
```

Name	LastName	Age
----	-----	---
Иван	Петров	22
Ольга	Белова	25
Сергей	Андреев	34
Мария	Никитина	41

Названия полей при импорте можно изменить с помощью параметра -Header:

```
PS C:\Script> Import-Csv -Path .\test.csv -Header 'Имя', 'Фамилия', 'Возраст'
```

Имя	Фамилия	Возраст
---	-----	-----
Name	LastName	Age
Иван	Петров	22
Сергей	Андреев	34
Ольга	Белова	25
Мария	Никитина	41

По умолчанию Import-Csv считает разделителем запятую. Если в файле в качестве разделителя используется другой символ, то его нужно указать в параметре -Delimiter. Например, заменим в файле test.csv запятые на символ точки с запятой:

```
PS C:\Script> (Get-Content .\test.csv -Raw).Replace(',', ';') | Set-Content .\test.csv
```

```
PS C:\Script> Get-Content .\test.csv
Name;Lastname;Age
Иван;Петров;22
Сергей;Андреев;34
Ольга;Белова;25
Мария;Никитина;41
```

Импортировать такой файл можно следующей командой:

```
PS C:\Script> Import-Csv -Path .\test.csv -Delimiter ';' 
```

```
Name      Lastname Age
----      -
Иван      Петров  22
Сергей    Андреев 34
Ольга     Белова  25
Мария     Никитина 41
```

ЗАМЕЧАНИЕ

Если значения в файле разделены символами табуляции, то в качестве значения параметра `-Delimiter` нужно указать ``t`.

Запись в CSV-файл

С помощью командлета `Export-Csv` можно сформировать CSV-файл из массива объектов PowerShell.

Например, получим список всех запущенных на локальном компьютере служб (статус, имя и отображаемое имя):

```
PS C:\Script> Get-Service | Where-Object Status -eq Running | Select-Object
Status, Name, DisplayName
```

Status	Name	DisplayName
-----	----	-----
Running	AdobeARMService	Adobe Acrobat Update Service
Running	AMD External Events Utility	AMD External Events Utility
Running	AMD FUEL Service	AMD FUEL Service
Running	Appinfo	Application Information
Running	AppXSvc	AppX Deployment Service (AppXSVC)
Running	AudioEndpointBuilder	Windows Audio Endpoint Builder
Running	Audiosrv	Windows Audio
Running	BFE	Base Filtering Engine
...		

Сохраним этот список в переменную `$services`:

```
PS C:\Script> $services = Get-Service | Where-Object Status -eq Running |
Select-Object Status, Name, DisplayName
```

Экспортируем массив объектов `$services` в CSV-файл `services.csv`:

```
PS C:\Script> $services | Export-Csv -Path .\services.csv
```

Проверим содержимое файла `services.csv`:

```
PS C:\Script> Get-Content .\services.csv
#TYPE Selected.System.ServiceProcess.ServiceController
"Status","Name","DisplayName"
"Running","AdobeARMservice","Adobe Acrobat Update Service"
"Running","AMD External Events Utility","AMD External Events Utility"
"Running","AMD FUEL Service","AMD FUEL Service"
. . .
```

Как видим, файл создан корректно, но в самой первой его строке указан тип объектов, из которых он получен (в нашем случае это объекты типа `Selected.System.ServiceProcess.ServiceController`). Если эта информация в файле не нужна, то при вызове `Export-Csv` следует указать параметр `-NoTypeInfoInformation`:

```
PS C:\Script> $services | Export-Csv -Path .\services.csv -NoTypeInfoInformation
```

В этом случае в первую строку в файле будут записаны названия полей:

```
PS C:\Script> Get-Content .\services.csv
"Status","Name","DisplayName"
"Running","AdobeARMservice","Adobe Acrobat Update Service"
"Running","AMD External Events Utility","AMD External Events Utility"
"Running","AMD FUEL Service","AMD FUEL Service"
"Running","Appinfo","Application Information"
. . .
```

Обработка данных без обращения к файлу

В PowerShell есть командлеты `ConvertFrom-Csv` и `ConvertTo-Csv`, которые действуют аналогично `ImportFrom-Csv` и `ExportTo-Csv`, но при этом не обращаются к внешним файлам.

Например, повторим наш пример со списком запущенных служб:

```
PS C:\Script> $services = Get-Service | Where-Object Status -eq Running |
Select-Object Status, Name, DisplayName
```

Преобразуем массив объектов `$services` в CSV:

```
PS C:\Script> $csv = $services | ConvertTo-Csv
```

В переменной `$csv` будет содержаться строка в CSV-формате:

```
PS C:\Script> $csv | Get-Member
```

```
TypeName: System.String
```

```
. . .
```

```
PS C:\Script> $csv
#TYPE Selected.System.ServiceProcess.ServiceController
"Status","Name","DisplayName"
"Running","AdobeARMservice","Adobe Acrobat Update Service"
```

```
"Running","AMD External Events Utility","AMD External Events Utility"
"Running","AMD FUEL Service","AMD FUEL Service"
. . .
```

Теперь конвертируем строку `$csv` в переменную `$objs`:

```
PS C:\Script> $objs = $csv | ConvertFrom-Csv
```

Массив `$objs` будет содержать объекты типа `Selected.System.ServiceProcess.ServiceController`, который был указан в комментарии `#TYPE`:

```
PS C:\Script> Get-Member -InputObject $objs
```

```
TypeName: System.Object[]
```

```
. . .
```

```
PS C:\Script> $objs | Get-Member
```

```
TypeName: CSV:Selected.System.ServiceProcess.ServiceController
```

```
. . .
```

```
PS C:\Script> $objs
```

Status	Name	DisplayName
-----	-----	-----
Running	AdobeARMService	Adobe Acrobat Update Service
Running	AMD External Events Utility	AMD External Events Utility
Running	AMD FUEL Service	AMD FUEL Service
. . .		

Обработка данных в JSON-формате

Данные в формате JSON (JavaScript Object Notation) широко используются в веб-разработке, а также для хранения настроек приложений. В PowerShell работать с JSON можно с помощью командлетов `ConvertFrom-Json` и `ConvertTo-Json`.

ЗАМЕЧАНИЕ

В отличие от CSV, специальных командлетов для импорта/экспорта JSON-файлов в PowerShell нет.

Давайте преобразуем в JSON содержимое нашего CSV-файла `C:\Script\test.csv`. Для этого мы сначала импортируем этот файл в массив объектов PowerShell, а затем конвертируем этот массив в JSON, передав его по конвейеру командлету `ConvertTo-Json`:

```
PS C:\Script> Import-Csv -Path .\test.csv
```

Name	Lastname	Age
----	-----	----
Иван	Петров	22
Сергей	Андреев	34
Ольга	Белова	25
Мария	Никитина	41

```
PS C:\Script> Import-Csv -Path .\test.csv | ConvertTo-Json
[
  {
    "Name": "Иван",
    "Lastname": "Петров",
    "Age": "22"
  },
  {
    "Name": "Сергей",
    "Lastname": "Андреев",
    "Age": "34"
  },
  {
    "Name": "Ольга",
    "Lastname": "Белова",
    "Age": "25"
  },
  {
    "Name": "Мария",
    "Lastname": "Никитина",
    "Age": "41"
  }
]
```

Убедимся, что в результате получается строка:

```
PS C:\Script> $json = Import-Csv -Path .\test.csv | ConvertTo-Json
PS C:\Script> Get-Member -InputObject $json

TypeName: System.String
. . .
```

По умолчанию JSON-строка формируется в структурированном виде, удобном для восприятия человеком: вложенность элементов оформляется дополнительными отступами, поля разделяются переводами строк. Если JSON обрабатывается автоматически, то от лишних пробелов и переводов строк для форматирования можно отказаться с помощью параметра `-Compress`. В этом случае поля и значения записываются подряд и JSON-строка будет иметь минимальную длину:

```
PS C:\Script> Import-Csv -Path .\test.csv | ConvertTo-Json -Compress
[{"Name":"Иван","Lastname":"Петров","Age":"22"}, {"Name":"Сергей","Lastname":
"Андреев","Age":"34"}, {"Name":"Ольга","Lastname":"Белова","Age":"25"}, {"Name":"
Мария","Lastname":"Никитина","Age":"41"}]
```

Для создания файла в формате JSON полученную в результате конвертации строку нужно сохранить с помощью командлетов `Set-Content` или `Out-File`:

```
PS C:\Script> Import-Csv -Path .\test.csv | ConvertTo-Json | Set-Content
.\test.json
```

Отметим, что, в отличие от экспорта в CSV-файл, при конвертации массива объектов в JSON информация о первоначальном типе объектов теряется. Для иллюстра-

ции этого вновь создадим массив `$services`, в котором будет храниться список запущенных на локальном компьютере служб:

```
PS C:\Script> $services = Get-Service | Where-Object Status -eq Running |
Select-Object Status, Name, DisplayName
```

Элементами этого массива будут объекты типа `Selected.System.ServiceProcess.ServiceController`:

```
PS C:\Script> $services | Get-Member

TypeName: Selected.System.ServiceProcess.ServiceController
. . .
```

Преобразуем массив `$services` в JSON и сохраним результат в переменной `$json_services`:

```
PS C:\Script> $json_services = $services | ConvertTo-Json
```

Теперь выполним обратное преобразование — конвертируем строку `$json_services` в массив `$objs` с помощью командлета `ConvertFrom-Json`:

```
PS C:\Script> $objs = $json_services | ConvertFrom-Json
```

Проверим, какой тип имеют элементы массива `$objs`:

```
PS C:\Users\andr> $objs | Get-Member

TypeName: System.Management.Automation.PSCustomObject
```

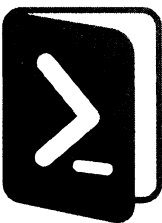
Name	MemberType	Definition
----	-----	-----
Equals	Method	bool Equals(System.Object obj)
GetHashCode	Method	int GetHashCode()
GetType	Method	type GetType()
ToString	Method	string ToString()
DisplayName	NoteProperty	string DisplayName=Adobe Acrob...
Name	NoteProperty	string Name=AdobeARMservice
Status	NoteProperty	int Status=4

Как видим, в результате преобразования из JSON-строки `$json_services` создаются объекты типа `System.Management.Automation.PSCustomObject`. Эти объекты имеют те же поля, что были у первоначальных объектов типа `Selected.System.ServiceProcess.ServiceController`, однако специфические методы для управления службами в преобразованных из JSON объектах будут недоступны.

Итоги

- ❑ Для работы с наиболее распространенными форматами структурированных данных в PowerShell используются командлеты `ConvertFrom-*`, `ConvertTo-*`.
- ❑ Читать и изменять CSV-файлы можно с помощью командлетов `Import-Csv` и `Export-Csv`.

ГЛАВА 13



Управление процессами, службами и серверами автоматизации

В данной главе мы рассмотрим вопросы, связанные с работой из оболочки PowerShell с двумя важными компонентами операционной системы Windows: запущенными процессами и службами, работающими в фоновом режиме.

Управление процессами

Основным графическим инструментом для управления процессами на локальной машине является диспетчер задач Windows (для запуска диспетчера задач можно нажать комбинацию клавиш <Ctrl>+<Shift>+<Esc>), который отображает информацию о выполняющихся процессах, позволяет останавливать их и задавать приоритет выполнения (рис. 13.1).

Диспетчер задач							
Файл Параметры Вид							
Процессы Производительность Журнал приложений Автозагрузка Пользователи Подробности Службы							
Имя	Состояние	7% ЦП	46% Память	0% Диск	0% Сеть	0% GPU	Ядро GPU
Приложения (6)							
> Google Chrome (10)		0,1%	203,4 МБ	0,1 МБ/с	0 Мбит/с	0%	
> Microsoft Word (32 бита)		0%	22,6 МБ	0 МБ/с	0 Мбит/с	0%	
> Windows PowerShell		0%	4,3 МБ	0 МБ/с	0 Мбит/с	0%	
> Windows Terminal (4)		0%	67,6 МБ	0 МБ/с	0 Мбит/с	0%	
> Диспетчер задач		0,6%	23,3 МБ	0 МБ/с	0 Мбит/с	0%	
> Проводник		0,1%	35,9 МБ	0 МБ/с	0 Мбит/с	0%	
Фоновые процессы (69)							
> Adobe Acrobat Update Service (32 бита)		0%	0,1 МБ	0 МБ/с	0 Мбит/с	0%	
> Antimalware Service Executable		0%	120,6 МБ	0,1 МБ/с	0 Мбит/с	0%	
> Application Frame Host		0%	0,8 МБ	0 МБ/с	0 Мбит/с	0%	

Рис. 13.1. Диспетчер задач Windows

Также в Windows имеются консольные утилиты `tasklist` (просмотр списка процессов, запущенных на локальном или удаленном компьютере) и `taskkill` (остановка процесса):

```
PS C:\Users\andr> tasklist /?
```

```
TASKLIST [/S <система> [/U <имя пользователя> [/P [<пароль>]]]]  
        [/M [<модуль>] | /SVC | /V] [/FI <фильтр>] [/FO <формат>] [/NH]
```

Описание:

Отображает список процессов, которые
сейчас выполняются на локальном или удаленном компьютере.

. . .

```
PS C:\Users\andr> taskkill /?
```

```
TASKKILL [/S <система> [/U <пользователь> [/P [<пароль>]]]]  
        { [/FI <фильтр>] [/PID <процесс> | /IM <образ>] } [/T] [/F]
```

Описание:

Завершает процесс по его ID (PID) или имени образа.

. . .

Рассмотрим способы решения с помощью PowerShell некоторых типичных задач, связанных с процессами.

Просмотр списка процессов

В PowerShell получить список запущенных процессов позволяет командлет `Get-Process`. Если запустить этот командлет без параметров, то на экран будет выведена информация обо всех запущенных процессах:

```
PS C:\> Get-Process
```

Handles	NPM(K)	PM(K)	WS(K)	VM(M)	CPU(s)	Id	ProcessName
-----	-----	-----	-----	-----	-----	--	-----
109	5	1128	792	32	0.06	1360	alg
506	6	2308	3604	29	20.34	628	csrss
67	3	832	436	29	1.52	1736	ctfmon
553	17	17220	11992	94	69.16	1164	explorer
33	2	3280	400	35	2.41	3192	Far

. . .

Каждому процессу соответствует объект типа `System.Diagnostics.Process`; командлет `Get-Process` по умолчанию отображает несколько свойств этих объектов (табл. 13.1).

Воспользовавшись параметром `-Name` (данный параметр используется по умолчанию), можно вывести информацию об одном или нескольких процессах с определенными именами (при этом в именах можно применять подстановочные симво-

Таблица 13.1. Свойства, отображаемые командлетом Get-Process по умолчанию

Свойство	Описание
Handles	Счетчик дескрипторов (свойство HandleCount объекта System.Diagnostics.Process)
NPM	Объем невыгружаемого пула (свойство NonpagedSystemMemorySize объекта System.Diagnostics.Process)
PM	Объем используемой виртуальной памяти (свойство PagedSystemMemorySize объекта System.Diagnostics.Process)
WS	Объем используемой памяти (свойство WorkingSet объекта System.Diagnostics.Process)
VM	Объем выгружаемого пула (свойство VirtualMemorySize объекта System.Diagnostics.Process)
CPU	Процессорное время в секундах, затраченное процессом (свойство TotalProcessorTime.TotalSeconds объекта System.Diagnostics.Process)
ID	Идентификатор процесса (свойство Id объекта System.Diagnostics.Process)
ProcessName	Имя процесса (свойство ProcessName объекта System.Diagnostics.Process)

лы). Например, следующая команда выведет все процессы, имена которых начинаются на букву s:

```
PS C:\> Get-Process s*
```

Handles	NPM(K)	PM(K)	WS (K)	VM(M)	CPU(s)	Id	ProcessName
-----	-----	-----	-----	-----	-----	--	-----
106	2	380	32	11	0.03	1880	scardsvr
134	4	1876	200	88	0.11	3640	sdlaunch
267	6	1504	1236	46	3.67	696	services
19	1	164	52	4	0.06	560	smss
71	2	1820	588	30	0.14	1352	SOUNDMAN
297	6	6924	1768	54	5.05	1304	spoolsv
134	3	1424	824	33	0.16	876	svchost
314	13	1768	1280	36	0.75	944	svchost
1185	45	12124	6532	83	14.59	1040	svchost
59	3	1124	740	27	1.23	1076	svchost
224	7	2356	788	38	0.27	1204	svchost
130	4	2392	1324	35	0.81	1952	svchost
517	0	0	52	1	26.75	4	System

Для сортировки или фильтрации списка процессов применяются, как обычно, командлеты Sort-Object, Where-Object и Select-Object. Например, следующий конвейер команд выводит на экран пять процессов, которые тратят наибольшее количество процессорного времени:

```
PS C:\> Get-Process | Sort-Object CPU -Descending | Select-Object -First 5
```

Handles	NPM(K)	PM(K)	WS(K)	VM(M)	CPU(s)	Id	ProcessName
619	19	21044	23156	110	171.19	1164	explorer
790	50	59228	11788	191	93.17	1520	kavsvc
476	14	10320	11120	129	88.25	2860	OUTLOOK
448	19	61204	23412	165	83.59	3884	wjview
411	12	9780	8920	250	61.95	3484	WINWORD

Для просмотра полной информации об одном или нескольких процессах можно воспользоваться командлетом `Format-List`, отобразив все свойства объекта `System.Diagnostics.Process`. Например:

```
PS C:\> Get-Process outlook | Format-List *
```

```

__NounName           : Process
Name                 : OUTLOOK
Handles              : 472
VM                   : 135835648
WS                   : 10010624
PM                   : 10584064
NPM                  : 14552
Path                 : C:\Program Files\Microsoft Office\Offic
                      e\OUTLOOK.EXE
Company              : Microsoft Corporation
CPU                  : 88.875
FileVersion          : 9.0.2416
ProductVersion       : 9.0.2416
Description          : Microsoft Outlook
Product              : Microsoft Outlook
Id                   : 2860
PriorityClass         : Normal
HandleCount          : 472
WorkingSet           : 10010624
PagedMemorySize      : 10584064
PrivateMemorySize    : 10584064
VirtualMemorySize    : 135835648
TotalProcessorTime   : 00:01:28.8750000
BasePriority          : 8
ExitCode              :
HasExited             : False
ExitTime              :
Handle               : 2276
MachineName          : .
MainWindowHandle      : 263442
MainWindowTitle       : Входящие - Microsoft Outlook
MainModule            : System.Diagnostics.ProcessModule (OUTLO OK.EXE)
MaxWorkingSet         : 1413120

```

```

MinWorkingSet           : 204800
Modules                  : {OUTLOOK.EXE, ntdll.dll, kernel32.dll,
                           OUTLLIB.dll...}
NonpagedSystemMemorySize : 14552
NonpagedSystemMemorySize64 : 14552
PagedMemorySize64       : 10584064
PagedSystemMemorySize   : 109464
PagedSystemMemorySize64 : 109464
PeakPagedMemorySize      : 11055104
PeakPagedMemorySize64    : 11055104
PeakWorkingSet           : 22802432
PeakWorkingSet64         : 22802432
PeakVirtualMemorySize    : 152793088
PeakVirtualMemorySize64  : 152793088
PriorityBoostEnabled      : True
PrivateMemorySize64      : 10584064
PrivilegedProcessorTime   : 00:00:51.0156250
ProcessName              : OUTLOOK
ProcessorAffinity         : 1
Responding               : True
SessionId                : 0
StartInfo                : System.Diagnostics.ProcessStartInfo
StartTime                : 24.04.2008 8:05:59
SynchronizingObject      :
Threads                  : {1756, 316, 3108, 808...}
UserProcessorTime         : 00:00:37.8593750
VirtualMemorySize64      : 135835648
EnableRaisingEvents      : False
StandardInput            :
StandardOutput           :
StandardError            :
WorkingSet64             : 10010624
Site                    :
Container                :

```

Для получения списка процессов на удаленных машинах нужно указать их имена или IP-адреса в качестве значения параметра `-ComputerName`.

Определение библиотек, используемых процессом

У объектов `System.Diagnostics.Process`, возвращаемых командлетом `Get-Process`, есть свойство `Modules`, содержащее список динамических библиотек, используемых соответствующими процессами. Для просмотра этого списка можно обрабатывать данные объекты командлетом `Select-Object` с параметром `-ExpandProperty`.

Например, следующий конвейер команд выводит список всех динамических библиотек, используемых оболочкой PowerShell (процесс с именем `powershell`):

```
PS C:\> Get-Process powershell | Select-Object -expandproperty modules |
Format-Table
```

Size(K)	ModuleName	FileName
452	powershell.exe	
C:\WINDOWS\System32\WindowsPowerShell\v1.0\powershell.exe		
2004	ntdll.dll	C:\WINDOWS\SYSTEM32\ntdll.dll
756	KERNEL32.DLL	C:\WINDOWS\System32\KERNEL32.DLL
2848	KERNELBASE.dll	C:\WINDOWS\System32\KERNELBASE.dll
632	msvcrt.dll	C:\WINDOWS\System32\msvcrt.dll
820	OLEAUT32.dll	C:\WINDOWS\System32\OLEAUT32.dll
628	msvc_p_win.dll	C:\WINDOWS\System32\msvc_p_win.dll
1024	ucrtbase.dll	C:\WINDOWS\System32\ucrtbase.dll
116	ATL.DLL	C:\WINDOWS\SYSTEM32\ATL.DLL
3412	combase.dll	C:\WINDOWS\System32\combase.dll
. . .		

Остановка процессов

Остановить процесс на локальной машине позволяет командлет Stop-Process, имеющий псевдоним kill. При этом по умолчанию используется параметр Id, требующий указания идентификатора останавливаемого процесса (напомним, что идентификатор процесса можно узнать с помощью командлета Get-Process). Например, следующая команда остановит процесс с идентификатором 764:

```
PS C:\> Stop-Process 764
```

Для остановки процесса с определенным именем нужно воспользоваться параметром -Name командлета Stop-Process. Например, следующая команда остановит все процессы с именем notepad:

```
PS C:\> Stop-Process -Name notepad
```

Параметр -Confirm командлета Stop-Process включает режим подтверждения при остановке процессов. При этом на экране отображается как имя, так и идентификатор останавливаемого процесса, например:

```
PS C:\Users\andrv> Stop-Process -Name notepad -Confirm
```

Подтверждение

Вы действительно хотите выполнить это действие?

Выполнение операции "Stop-Process" над целевым объектом "notepad (2812)".

[Y] Да - Y [A] Да для всех - A [N] Нет - N [L] Нет для всех - L [S]

Приостановить - S [?] Справка

(значением по умолчанию является "Y"):Y

Подтверждение

Вы действительно хотите выполнить это действие?

Выполнение операции "Stop-Process" над целевым объектом "notepad (13984)".

[Y] Да - Y [A] Да для всех - A [N] Нет - N [L] Нет для всех - L [S]

Приостановить - S [?] Справка

(значением по умолчанию является "Y"):Y

По умолчанию командлет `Stop-Process` не передает далее по конвейеру объекты, соответствующие останавливаемым процессам, и поэтому на экране ничего не отображается. Для остановки процессов с выводом информации нужно указать параметр `-PassThru`, например:

```
PS C:\> Stop-Process -Name notepad -PassThru
```

Handles	NPM(K)	PM(K)	WS(K)	VM(M)	CPU(s)	Id	ProcessName
26	2	624	2172	24		2224	notepad

ЗАМЕЧАНИЕ

Для остановки процессов на удаленных компьютерах можно воспользоваться утилитой командной строки `taskkill` или методом `Terminate()` WMI-объекта `Win32_Process`.

Запуск процессов

Для запуска процесса из оболочки PowerShell можно просто указать путь к соответствующему исполняемому файлу, например:

```
PS C:\> C:\WINDOWS\system32\notepad.exe
```

В результате выполнения этой команды запустится стандартный Блокнот.

ЗАМЕЧАНИЕ

Напомним, что для выполнения файла, путь к которому содержит пробелы, нужно использовать оператор запуска `&`, а путь к файлу заключить в кавычки.

Также запустить процесс можно с помощью командлета `Invoke-Item`, указав в качестве аргумента путь к нужному файлу. Параметр `-Confirm` позволяет запускать процесс только после подтверждения пользователя:

```
PS C:\> Invoke-Item C:\WINDOWS\system32\notepad.exe -Confirm
```

Подтверждение

Вы действительно хотите выполнить это действие?

Выполнение операции "Вызов элемента" над целевым объектом "Элемент:
C:\WINDOWS\system32\notepad.exe".

[Y] Да - Y [A] Да для всех - A [N] Нет - N [L] Нет для всех - L [S]

Приостановить - S [?] Справка

(значением по умолчанию является "Y"): Y

Однако командлет `Invoke-Item` не возвращает объект, соответствующий запущенному процессу, в силу чего мы не можем программно определить, скажем, идентификатор, назначенный системой данному процессу.

Поэтому процессы на локальной машине лучше запускать с помощью специального командлета `Start-Process`. Путь к исполняемому файлу или скрипту для запуска указывается в параметре `-FilePath`:

```
PS C:\Users\andr> Start-Process -FilePath notepad.exe
```


Параметры команды `Start-Process` позволяют передавать аргументы запускаемому процессу, перенаправлять его входные и выходные потоки, задавать учетную запись и определять тип окна для процесса и т. д.

По умолчанию `Start-Process` только запускает процесс, ничего при этом не возвращая. Если нужно вернуть объект, соответствующий новому процессу, то следует использовать ключ `-PassThru`:

```
PS C:\Users\andr> $process = Start-Process -FilePath notepad.exe -PassThru
PS C:\Users\andr> $process.GetType().FullName
System.Diagnostics.Process
```

Отметим, что с помощью `Start-Process` можно не только запускать исполняемые файлы, но и выполнять действия, ассоциированные с файлами других типов. Например, создадим текстовый файл `1.txt` и запустим его на печать, выполнив команду `Start-Process` с указанием действия `Print`:

```
PS C:\Users\andr> "File content" > 1.txt
PS C:\Users\andr> Start-Process -FilePath .\1.txt -Verb Print
```

Рассмотрим еще метод `Create()` WMI-класса `Win32_Process`, с помощью которого можно запускать процессы как на локальном, так и на удаленном компьютере. Данный метод является конструктором класса `Win32_Process`, для его вызова можно использовать спецификатор типа `[wmiClass]`. Например, запустим Блокнот:

```
PS C:\> $a = ([wmiClass] "win32_process").Create("notepad.exe")
```

С помощью командлета `Get-Member` посмотрим, какие свойства имеет объект, сохраненный в переменной `$a`:

```
PS C:\> $a | Get-Member
```

TypeName: 'System.Management.ManagementBaseObject#\'__PARAMETERS

Name	MemberType	Definition
-----	-----	-----
PSComputerName	AliasProperty	PSComputerName = __SERVER
ProcessId	Property	uint32 ProcessId {get;set;}
ReturnValue	Property	uint32 ReturnValue {get;set;}
__CLASS	Property	string __CLASS {get;set;}
__DERIVATION	Property	string[] __DERIVATION {get;set;}
__DYNASTY	Property	string __DYNASTY {get;set;}
__GENUS	Property	int __GENUS {get;set;}
__NAMESPACE	Property	string __NAMESPACE {get;set;}
__PATH	Property	string __PATH {get;set;}
__PROPERTY_COUNT	Property	int __PROPERTY_COUNT {get;set;}
__RELPATH	Property	string __RELPATH {get;set;}
__SERVER	Property	string __SERVER {get;set;}
__SUPERCLASS	Property	string __SUPERCLASS {get;set;}

Как видим, нам доступно свойство `ProcessId`, в котором хранится идентификатор запущенного процесса.

Изменение приоритетов выполнения процесса

У объекта `System.Diagnostics.Process` имеется метод `set_PriorityClass`, позволяющий изменять приоритет выполнения процесса. В качестве параметра данного метода следует указать одну из следующих строк: `Normal` (средний), `Idle` (низкий), `High` (высокий), `RealTime` (реального времени), `BelowNormal` (ниже среднего), `AboveNormal` (выше среднего).

Например, запустим новый экземпляр Блокнота и с помощью метода `get_PriorityClass` проверим текущий приоритет данного процесса:

```
PS C:\Users\andr> $b = Start-Process -FilePath notepad.exe -PassThru
PS C:\> $b.get_PriorityClass()
Normal
```

Повысим теперь приоритет до высокого:

```
PS C:\> $b.set_PriorityClass("High")
PS C:\> $b.get_PriorityClass()
High
```

Завершение неотвечающих процессов

У объектов `System.Diagnostics.Process`, соответствующих запущенным процессам, есть свойство `Responding`, которое принимает значение `False`, если процесс не реагирует на запросы. Для завершения всех неотвечающих приложений нужно выделить их из общей массы с помощью командлета `Where-Object`, после чего остановить с помощью командлета `Stop-Process`:

```
PS C:\> Get-Process | Where-Object -not Responding | Stop-Process
```

Управление службами

В операционной системе Windows *служба* (service) — это процесс, который запускается на машине в фоновом режиме для выполнения определенных действий в ответ на запросы пользователей. Например, служба веб-сервера работает в фоновом режиме, ожидая получения HTTP-запросов от клиентов. При получении такого запроса служба отвечает на него, посылая клиенту запрошенный файл или выполняя определенное действие. Состав служб зависит от версии операционной системы и установленных в ней приложений.

Основным инструментом для администрирования служб в графическом режиме на локальном компьютере является консоль **Службы**, которая находится в программной группе **Администрирование** панели управления (рис. 13.2).

В качестве утилиты, с помощью которой можно управлять службами как на локальных, так и на удаленных машинах, используется консоль **Управление компьютером**. Также в Windows входят стандартные утилиты командного интерпретатора `cmd.exe` для администрирования служб: `net start` и `net stop`.

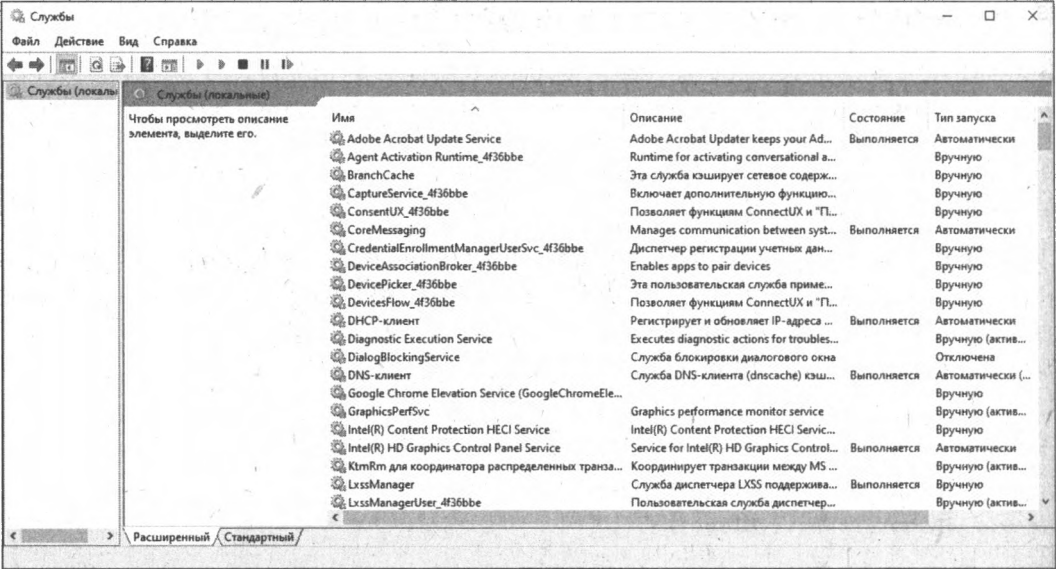


Рис. 13.2. Консоль для управления службами

Рассмотрим способы решения с помощью PowerShell некоторых типичных задач, связанных со службами.

Просмотр списка служб

Получить список служб, зарегистрированных на локальном компьютере, позволяет командлет `Get-Service`:

```
PS C:\> Get-Service

Status  Name                DisplayName
-----
Running Alerter             Оповещатель
Running ALG           Служба шлюза уровня приложения
Stopped AppMgmt        Управление приложениями
Stopped aspnet_state   ASP.NET State Service
Running AudioSrv       Windows Audio
Stopped BITS          Фоновая интеллектуальная служба пер...
Stopped Browser       Обозреватель компьютеров
Stopped cisvc         Служба индексирования
Stopped ClipSrv       Сервер папки обмена
. . .
```

Как видим, по умолчанию выводится имя службы (колонок `Name`), ее отображаемое имя (колонок `DisplayName`) и состояние (колонок `Status`).

Командлет `Get-Service` имеет параметры `-Name` и `-DisplayName`, поддерживающие шаблонные символы. Например, следующая команда выводит список служб, имена которых начинаются с символов `win`:

```
PS C:\Users\andrv> Get-Service -Name 'win*'
```

Status	Name	DisplayName
-----	----	-----
Running	WinDefend	Microsoft Defender Antivirus Service
Running	WinHttpAutoProx...	WinHTTP Web Proxy Auto-Discovery Se...
Running	Winmgmt	Windows Management Instrumentation
Stopped	WinRM	Windows Remote Management (WS-Manag...

Если нужно вывести на экран только работающие в данный момент службы, то нужно отфильтровать объекты, у которых значением свойства `Status` является строка `Running`:

```
PS C:\> Get-Service | Where-Object Status -eq "Running"
```

Status	Name	DisplayName
-----	----	-----
Running	Alerter	Оповещатель
Running	ALG	Служба шлюза уровня приложения
Running	AudioSrv	Windows Audio
Running	CryptSvc	Службы криптографии
. . .		

Для просмотра служб, зарегистрированных на удаленном компьютере, нужно указать его имя в качестве значения параметра `-ComputerName`.

Остановка и приостановка служб

Локальную службу можно остановить с помощью командлета `Stop-Service` (при этом PowerShell должен быть запущен от имени администратора). Параметр `-Name` задает имя службы для остановки (здесь можно использовать подстановочные знаки), параметр `-Force` позволяет остановить указанную службу вместе со всеми службами, зависящими от нее.

Например, следующая команда остановит службу сервера `LanmanServer` (эта служба поддерживает общий доступ к файлам и принтерам по сети) со всеми зависящими от нее службами:

```
PS C:\> Stop-Service -Name LanmanServer -Force
```

По умолчанию командлет `Stop-Service` не передает далее по конвейеру объекты, соответствующие останавливаемым службам, и поэтому на экране ничего не отображается. Для остановки служб с выводом информации нужно указать параметр `-PassThru`, например:

```
PS C:\> Stop-Service -Name LanmanServer -Force -PassThru
```

Status	Name	DisplayName
-----	----	-----
Stopped	LanmanServer	Сервер

Командлет `Suspend-Service` позволяет приостановить работу одной или нескольких служб, имена которых задаются в качестве значения параметра `-Name`. При приостановке

новке (временной остановке) службы она продолжает выполняться, однако ее действия приостанавливаются до поступления команды на возобновление работы (см. следующий раздел).

Следует учитывать, что не всякую службу можно приостановить. У объектов `System.ServiceProcess.ServiceController`, соответствующих службам, имеется логическое свойство `CanPauseAndContinue`, которое равно `True`, если служба может быть приостановлена. Следующая команда приостановит все службы, для которых это возможно:

```
PS C:\> Get-Service | Where-Object {$_.CanPauseAndContinue} | Suspend-Service -PassThru
```

Status	Name	DisplayName
-----	----	-----
Paused	Irmon	Монитор инфракрасной связи
Paused	lanmanserver	Сервер
Paused	lanmanworkstation	Рабочая станция
Paused	Schedule	Планировщик заданий
Paused	seclogon	Вторичный вход в систему
Paused	ShellHWDetection	Определение оборудования оболочки
Paused	TapiSrv	Телефония
Paused	winmgmt	Инструментарий управления Windows

ЗАМЕЧАНИЕ

Для остановки или приостановки служб на удаленном компьютере можно воспользоваться соответственно методами `StopService` и `PauseService` WMI-класса `Win32_Service`.

Запуск и перезапуск служб

Запустить службу на локальном компьютере можно с помощью командлета `Start-Service`. В качестве значения параметра `-Name` указывается имя запускаемой службы. Как и в предыдущих командлетах `*-Service`, после запуска службы на экран не выводится никакого сообщения; для исправления ситуации можно использовать параметр `-PassThru`. Например, следующая команда запускает службу `LanmanServer` и выводит на экран информацию об этой службе:

```
PS C:\> Start-Service -Name LanmanServer -PassThru
```

Status	Name	DisplayName
-----	----	-----
Running	Spooler	Диспетчер очереди печати

Командлет `Restart-Service` выполняет перезапуск (т. е. остановку и запуск) служб, указанных в качестве значения параметра `-Name` или полученных по конвейеру от других командлетов. Например, следующий конвейер команд перезапускает все сетевые службы, имя которых начинается со строки `net`:

```
PS C:\> Get-Service -Name net* | Restart-Service
```

Изменение параметров службы

В PowerShell имеется командлет `Set-Service`, позволяющий редактировать некоторые параметры служб на локальном или удаленном компьютере:

- ❑ отображаемое имя (параметр `DisplayName`);
- ❑ описание (параметр `Description`);
- ❑ тип запуска (параметр `StartupType`), возможные значения — `Automatic` (служба запускается автоматически), `Manual` (служба запускается вручную), `Disabled` (служба отключена).

Имена изменяемых служб указываются в качестве значения параметра `-Name` (в именах могут присутствовать подстановочные знаки).

Для примера мы изменим режим запуска службы сервера `LanmanServer`. Посмотрим все свойства этой службы:

```
PS C:\> Get-Service LanmanServer | Format-List *
```

```
Name                : LanmanServer
RequiredServices    : {SamSS, Srv2}
CanPauseAndContinue : False
CanShutdown         : False
CanStop             : True
DisplayName          : Сервер
DependentServices   : {}
MachineName         : .
ServiceName         : LanmanServer
ServicesDependedOn  : {SamSS, Srv2}
ServiceHandle       : SafeServiceHandle
Status              : Running
ServiceType         : Win32OwnProcess, Win32ShareProcess
StartType           : Automatic
Site                :
Container           :
```

Как видим, сейчас эта служба запускается автоматически (значение свойства `StartType` равно `Automatic`). Изменим с помощью командлета `Set-Service` режим запуска службы `LanmanServer` с автоматического на ручной:

```
PS C:\> Set-Service -Name LanmanServer -StartupType Manual
```

Еще раз обратимся к свойствам службы `LanmanServer` для проверки сделанных изменений:

```
PS C:\Users\andrv> Get-Service LanmanServer | Format-List Name, StartType
```

```
Name                : LanmanServer
StartType           : Manual
```

Как видим, значение свойства `StartType` успешно изменено.

Отметим, что описание службы не является свойством объектов `System.ServiceProcess.ServiceController`, возвращаемых командлетом `Get-Service`. Другими словами, командлет `Get-Service` не позволяет увидеть описание определенной службы.

Решить эту проблему можно с помощью WMI-класса `win32_Service`, который имеет свойство `Description` (описание службы). Посмотрим, чему равно значение этого свойства для службы `LanmanServer`:

```
PS C:\> Get-WmiObject win32_service -Filter "name = 'LanmanServer'" |  
Format-List Name, Description
```

```
Description Name : LanmanServer
```

```
Description : Поддерживает общий доступ к файлам, принтерам и именованным  
каналам для данного компьютера через сетевое подключение. Если служба  
остановлена, такие функции не удастся выполнить. Если данная служба отключена,  
не удастся запустить любые явно зависящие службы.
```

Работа с серверами автоматизации

В главе 11 мы рассматривали пример использования методов, предоставляемых внешними COM-объектами, для решения нужной нам задачи — создания ярлыка на рабочем столе. Этот стандарт программных компонентов `Component Object Model (COM)` лежал в основе технологии *связывания и внедрения объектов* (`Object Linking and Embedding, OLE`), которая изначально использовалась в Windows для обмена данными между приложениями.

Вначале технология OLE использовалась для создания составных документов, а затем для решения более общей задачи — предоставления приложениями друг другу собственных функций (служб) и правильного использования этих функций. Технология, позволяющая одному приложению (*клиенту автоматизации*) вызывать функции другого приложения (*сервера автоматизации*) была названа `OLE Automation`.

С середины 1990-х годов вместо термина OLE стал использоваться новый термин — *ActiveX*, первоначально обозначавший WWW-компоненты (объекты), созданные на базе технологии COM. До появления платформы .NET технология ActiveX была ключевой в продуктах Microsoft.

Одними из самых распространенных и часто используемых серверов автоматизации в Windows являются приложения пакета Microsoft Office. Мы рассмотрим на примерах, каким образом можно выводить из PowerShell информацию в две наиболее распространенные программы этого пакета: Microsoft Word и Microsoft Excel. Однако предварительно скажем несколько слов об объектных моделях, представляемых данными программами.

Объектные модели Microsoft Word и Excel

Для того чтобы использовать из PowerShell те возможности, которые поддерживают программы Word и Excel, необходимо знать, какие именно объекты предостав-

ляются для внешнего использования этими серверами автоматизации и как объекты соотносятся друг с другом.

Хотя объектные модели различных приложений Microsoft Office довольно обширны, они похожи друг на друга, причем для практических целей достаточно понять принцип работы с несколькими ключевыми объектами. Здесь мы не будем останавливаться на подробном рассмотрении свойств и методов объектов приложений Word и Excel, а лишь кратко упомянем, какие именно объекты будут использоваться в рассматриваемых далее примерах сценариев.

На самом верхнем уровне объектной модели Word находится объект `Application`, который представляет непосредственно само приложение Word и содержит (в качестве свойств) все остальные объекты. Таким образом, объект `Application` используется для получения доступа к любому другому объекту Word.

Семейство `Documents` является свойством объекта `Application` и содержит набор объектов `Document`, каждый из которых соответствует открытому в Word документу. Класс `Documents` понадобится нам в сценариях для создания новых документов. Объект `Document` содержит в качестве своих свойств семейства различных объектов документа: символов (`Characters`), слов (`Words`), предложений (`Sentences`), абзацев (`Paragraphs`), закладок (`Bookmarks`) и т. д.

Объект `Selection` позволяет работать с выделенным фрагментом текста (этот фрагмент может быть и пустым). Таким образом, можно сказать, что объект `Selection` открывает путь в документ, т. к. он предоставляет доступ к выделенному фрагменту документа. В частности, у объекта `Selection` имеется метод `TypeText(Text)`, с помощью которого можно вставлять текст в документ. Используя свойства этого объекта (которые, в свою очередь, могут являться объектами со своими свойствами), можно управлять параметрами выделенного фрагмента, например устанавливать нужный размер и гарнитуру шрифта, выравнивать абзацы по центру и т. п.

Объектная модель Excel построена по тому же принципу, что и объектная модель Word. Основным объектом, содержащим все остальные, является `Application`. Напомним, что отдельные файлы в Excel называются *рабочими книгами* (*workbooks*). Семейство `Workbooks` в Excel является аналогом семейства `Documents` в Word и содержит набор объектов `Workbook` (аналог объекта `Document` в Word), каждый из которых соответствует открытой в Excel рабочей книге. Новая рабочая книга создается с помощью метода `Add()` объекта `Workbooks`.

Для доступа к ячейкам активного *рабочего листа* (*worksheet*) в Excel используется свойство `Cells` объекта `Application`. Для получения или изменения значения отдельной ячейки применяется конструкция `Cells.Item(row, column).Value`, где `row` и `column` являются номерами строки и столбца соответственно, на пересечении которых находится данная ячейка.

В Excel, как и в Word, имеется объект `Selection`, позволяющий работать с выделенным фрагментом электронной таблицы. Самым простым способом выделить диапазон ячеек активного рабочего листа является использование метода `Select()` объекта `Range`. Например, выражение `Range("A1:C1").Select()` позволяет выделить три смежные ячейки: A1, B1 и C1.

Для того чтобы понять, какой именно объект Word или Excel нужно использовать для решения той или иной задачи, часто проще всего бывает проделать в соответствующем приложении необходимые манипуляции вручную, включив предварительно режим записи макроса. В результате мы получим текст макроса на языке VBA (Visual Basic for Applications), из которого будет ясно, какие методы и с какими параметрами нужно вызывать и какие значения нужно присваивать свойствам объектов.

Взаимодействие с Microsoft Word

Давайте научимся управлять с помощью PowerShell программой Microsoft Word: запустим из оболочки данную программу и напечатаем в окне Word строку текста.

Сделать это несложно. Сначала создаем экземпляр главного объекта сервера автоматизации Microsoft Word, который имеет программный идентификатор `Word.Application`, и сохраняем ссылку на этот объект в переменной `$word`:

```
PS C:\> $word = New-Object -ComObject Word.Application
```

С помощью метода `Add` коллекции `Documents` создаем новый документ и сохраняем ссылку на него в переменной `$doc`:

```
PS C:\> $doc = $word.Documents.Add()
```

Делаем окно с новым документом видимым. Для этого нужно в свойство `Visible` объекта `$word` записать `$true`:

```
PS C:\> $word.Visible = $true
```

В переменную `$sel` занесем ссылку на объект `Selection`, с помощью которого можно задавать тип и размер шрифта, тип выравнивания абзацев, а также печатать в документе строки текста:

```
PS C:\> $sel = $word.Selection
```

Установим размер шрифта:

```
PS C:\> $sel.Font.Size = 14
```

Установим полужирный шрифт:

```
PS C:\> $sel.Font.Bold = $true
```

Наконец, печатаем строку текста:

```
PS C:\> $sel.TypeText("Привет из PowerShell!")
```

Взаимодействие с Microsoft Excel

Давайте создадим из оболочки PowerShell новую рабочую книгу Microsoft Excel и выведем в нее заголовок и одну запись телефонной книги.

Как и в случае с Microsoft Word, сначала мы создаем экземпляр главного объекта `Excel.Application` и сохраняем ссылку на этот экземпляр в переменной `$XL`:

```
PS C:\> $XL = New-Object -ComObject Excel.Application
```

В результате запускается программа Microsoft Excel. Для того чтобы сделать окно приложения видимым, установим значение свойства `Visible` в `$true`:

```
PS C:\> $XL.Visible = $true
```

Создать новую рабочую книгу можно с помощью метода `Add` коллекции `WorkBooks`; при выполнении данного метода на экран выводится много ненужной технической информации, которую мы перенаправим на пустое устройство `$null`:

```
PS C:\> $XL.WorkBooks.Add() > $null
```

Теперь можно устанавливать нужную ширину столбцов (свойство `ColumnWidth`) и выводить текст заголовка:

```
PS C:\> #Устанавливаем нужную ширину колонок
```

```
PS C:\> $XL.Columns.Item(1).ColumnWidth = 40
```

```
PS C:\> $XL.Columns.Item(2).ColumnWidth = 40
```

```
PS C:\> $XL.Columns.Item(3).ColumnWidth = 40
```

```
PS C:\> #Печатаем в ячейках текст
```

```
PS C:\> $XL.Cells.Item(1,1).Value="Фамилия"
```

```
PS C:\> $XL.Cells.Item(1,2).Value="Имя"
```

```
PS C:\> $XL.Cells.Item(1,3).Value="Телефон"
```

С помощью метода `Select()` выделим три ячейки с введенным заголовком и установим в них полужирный шрифт:

```
PS C:\> #Выделяем три ячейки
```

```
PS C:\> $XL.Range("A1:C1").Select() > $null
```

```
PS C:\> #Устанавливаем полужирный шрифт для выделенного диапазона
```

```
PS C:\> $XL.Selection.Font.Bold = $true
```

Во вторую строку рабочей книги выводим данные об абоненте:

```
PS C:\> $XL.Cells.Item(2,1).Value="Иванов"
```

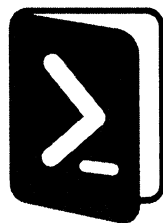
```
PS C:\> $XL.Cells.Item(2,2).Value="Иван"
```

```
PS C:\> $XL.Cells.Item(2,3).Value="555555"
```

Итоги

- ☐ Для управления процессами на локальной машине используются командлеты `Get-Process`, `Start-Process`, `Stop-Process`. Доступ к процессам на удаленной машине можно получить с помощью WMI-класса `Win32_Process`.
- ☐ Командлеты `Get-Service`, `Start-Service`, `Stop-Service` предназначены для работы со службами на локальном компьютере.
- ☐ С помощью командлета `New-Object` с параметром `-ComObject` можно обращаться к сервисам, предоставляемым серверами автоматизации.

ГЛАВА 14



HTTP-запросы к веб-ресурсам

В Интернете имеется огромное множество полезных сервисов, с которыми можно работать, обращаясь к их ресурсам по протоколу HTTP. Веб-разработчики постоянно работают с такими HTTP-запросами для доступа к функциям внешних API или для тестирования собственных приложений. Традиционно для этого используются специальные приложения (например, Postman), консольные утилиты типа cURL и wget, а также HTTP-клиенты, интегрированные в среды разработки.

PowerShell для обращения к веб-ресурсам по протоколу HTTP предлагает два стандартных командлета: `Invoke-WebRequest` и `Invoke-RestMethod`.

Командлет *Invoke-WebRequest*

С помощью командлета `Invoke-WebRequest` можно направить веб-серверу HTTP-запрос и получить от него ответ.

Анализ HTML-страниц

Командлет `Invoke-WebRequest` хорошо подходит для анализа HTML-страниц. Также этот командлет умеет сохранять страницы на локальном диске, в этом он похож на консольную утилиту `wget` и даже имеет такой псевдоним:

```
PS C:\Script> Get-Alias wget
```

CommandType	Name
Alias	wget -> Invoke-WebRequest

Обратимся с помощью данного командлета к какой-нибудь простой странице, например Example Domain (<https://example.com/index.html>) (рис. 14.1).

По умолчанию `Invoke-WebRequest` выполняет HTTP-запрос с методом `GET` к ресурсу на веб-сервере, адрес ресурса указывается в качестве значения параметра `-Uri`. В результате возвращается объект типа `HtmlWebResponseObject`, в котором хранится информация об ответе сервера:

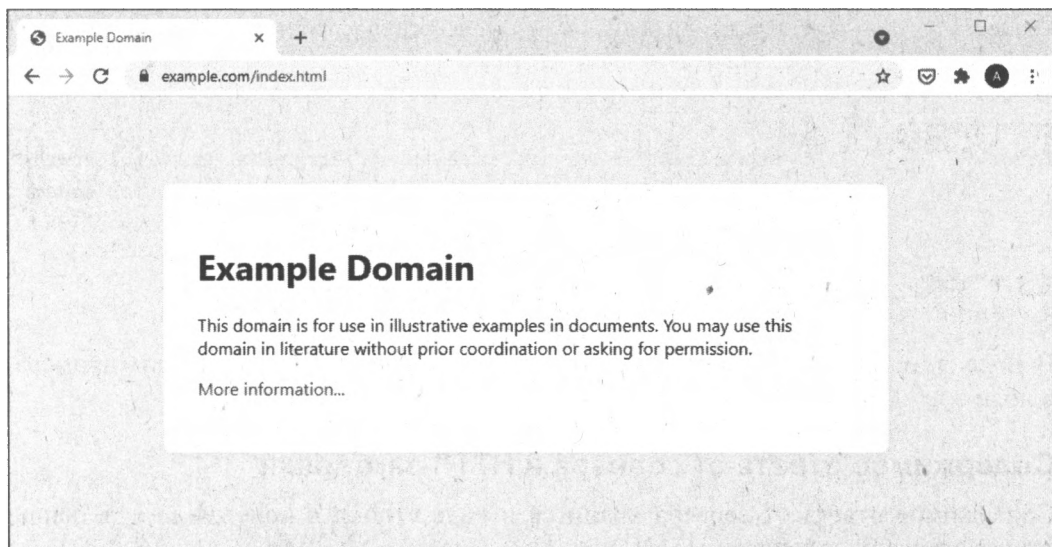


Рис. 14.1. Главная страница сайта Example Domain в браузере

```
PS C:\Script> $web = Invoke-WebRequest -Uri https://example.com/index.html
PS C:\Script> $web | Get-Member
```

```
TypeName: Microsoft.PowerShell.Commands.HtmlWebResponseObject
```

```
...
```

```
PS C:\Script> $web
```

```
StatusCodes       : 200
StatusDescription : OK
Content           : <!doctype html>
                   <html>
                   <head>
                       <title>Example Domain</title>
                       <meta charset="utf-8" />
                       <meta http-equiv="Content-type" content="text/html; char
                           set=utf-8" />
                       <meta name="viewport" conten...
RawContent        : HTTP/1.1 200 OK
                   Age: 497890
                   Vary: Accept-Encoding
                   X-Cache: HIT
                   Content-Length: 1256
                   Cache-Control: max-age=604800
                   Content-Type: text/html; charset=UTF-8
                   Date: Mon, 12 Jul 2021 16:05:14 GMT
                   Exp...
Forms             : {}
```

```

Headers      : {[Age, 497890], [Vary, Accept-Encoding], [X-Cache, HIT],
               [Content-Length, 1256]...}
Images       : {}
InputFields  : {}
Links        : {@{innerHTML=More information...; innerText=More information
               ...; outerHTML=<A href="https://www.iana.org/domains/example
               ">More information...</A>; outerText=More information...;
               tagName=A; href=https://www.iana.org/domains/example}}
ParsedHtml   : mshtml.HTMLDocumentClass
RawContentLength : 1256

```

В поле `StatusCode` содержится код ответа от сервера (200 для нашего примера), в поле `StatusDescription` — текстовое описание этого ответа (OK).

Содержимое ответа от сервера и HTTP-заголовки

Содержимое ответа от сервера хранится в виде строки в поле `Content`. В нашем случае здесь будет записан HTML-код:

```

PS C:\Script> $web.Content
<!doctype html>
<html>
<head>
  <title>Example Domain</title>

  <meta charset="utf-8" />
  <meta http-equiv="Content-type" content="text/html; charset=utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <style type="text/css">
body {
  background-color: #f0f0f2;
  margin: 0;
  padding: 0;
  font-family: -apple-system, system-ui, BlinkMacSystemFont, "Segoe UI",
  "Open Sans", "Helvetica Neue", Helvetica, Arial, sans-serif;

}
div {
  width: 600px;
  margin: 5em auto;
  padding: 2em;
  background-color: #fdfdff;
  border-radius: 0.5em;
  box-shadow: 2px 3px 7px 2px rgba(0,0,0,0.02);
}
a:link, a:visited {
  color: #38488f;
  text-decoration: none;
}

```

```
@media (max-width: 700px) {  
    div {  
        margin: 0 auto;  
        width: auto;  
    }  
}  
</style>  
</head>  
  
<body>  
<div>  
    <h1>Example Domain</h1>  
    <p>This domain is for use in illustrative examples in documents. You may  
    use this domain in literature without prior coordination or asking  
    for permission.</p>  
    <p><a href="https://www.iana.org/domains/example">More information...</a></p>  
</div>  
</body>  
</html>
```

В поле RawContent записывается полный ответ от сервера с HTTP-заголовками в начале:

```
PS C:\Script> $web.RawContent  
HTTP/1.1 200 OK  
Age: 497890  
Vary: Accept-Encoding  
X-Cache: HIT  
Content-Length: 1256  
Cache-Control: max-age=604800  
Content-Type: text/html; charset=UTF-8  
Date: Mon, 12 Jul 2021 16:05:14 GMT  
Expires: Mon, 19 Jul 2021 16:05:14 GMT  
ETag: "3147526947+ident"  
Last-Modified: Thu, 17 Oct 2019 07:18:26 GMT  
Server: ECS (dcb/7F83)
```

```
<!doctype html>  
<html>  
<head>  
    <title>Example Domain</title>  
    . . .
```

Также заголовки ответа выделяются отдельно и хранятся в виде хэш-таблицы в свойстве headers:

```
PS C:\Script> $web.headers
```

Key	Value
---	-----
Age	497890
Vary	Accept-Encoding

```

X-Cache      HIT
Content-Length 1256
Cache-Control max-age=604800
Content-Type  text/html; charset=UTF-8
Date         Mon, 12 Jul 2021 16:05:14 GMT
Expires      Mon, 19 Jul 2021 16:05:14 GMT
ETag         "3147526947+ident"
Last-Modified Thu, 17 Oct 2019 07:18:26 GMT
Server       ECS (dcb/7F83)

```

Сохранение веб-ресурсов

Для сохранения ответа от сервера в виде локального файла нужно при вызове `Invoke-WebRequest` использовать ключ `-OutFile` и указать путь к нужному файлу. Например, сохраним страницу <https://example.com/index.html> в файле `page.html` в текущем каталоге:

```
PS C:\Script> Invoke-WebRequest -Uri https://example.com/index.html
-OutFile page.html
```

Проверим содержимое файла `page.html` и убедимся, что в нем записана HTML-разметка сохраненной страницы:

```

PS C:\Script> type .\page.html
<!doctype html>
<html>
<head>
  <title>Example Domain</title>

  <meta charset="utf-8" />
  <meta http-equiv="Content-type" content="text/html; charset=utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <style type="text/css">
body {
  background-color: #f0f0f2;
  margin: 0;
  padding: 0;
  font-family: -apple-system, system-ui, BlinkMacSystemFont, "Segoe UI",
  "Open Sans", "Helvetica Neue", Helvetica, Arial, sans-serif;

}
div {
  width: 600px;
  margin: 5em auto;
  padding: 2em;
  background-color: #fdfdff;
  border-radius: 0.5em;
  box-shadow: 2px 3px 7px 2px rgba(0,0,0,0.02);
}

```

```

a:link, a:visited {
    color: #38488f;
    text-decoration: none;
}
@media (max-width: 700px) {
    div {
        margin: 0 auto;
        width: auto;
    }
}
</style>
</head>

<body>
<div>
    <h1>Example Domain</h1>
    <p>This domain is for use in illustrative examples in documents. You may
    use this domain in literature without prior coordination or asking
    for permission.</p>
    <p><a href="https://www.iana.org/domains/example">More information...</a></p>
</div>
</body>
</html>

```

Подобным образом можно сохранять не только HTML-файлы, но и ресурсы других типов (текстовые, графические или мультимедийные файлы и т. д.), к которым можно обратиться на сайте.

Поиск HTML-элементов на странице

В свойствах `Forms` (формы), `Images` (изображения), `InputFields` (поля ввода) и `Links` (ссылки) объекта `HtmlWebResponseObject` сохраняются массивы объектов, которые описывают соответствующие элементы HTML-разметки, полученной от сервера. Обработывая эти коллекции, можно получить информацию об интересующих нас элементах на загруженной странице.

Например, сохраним в переменной `$web` страницу результатов поиска слова `PowerShell` в `Yandex`:

```
PS C:\Script> $web = Invoke-WebRequest -Uri
https://yandex.ru/search/?text=PowerShell
```

Выделим на этой странице все ссылки на сайт **habr.com**. Для этого нужно отфильтровать массив `$web.Links`, оставив в нем объекты, у которых значение свойства `href` соответствует маске `*habr.com*`:

```
PS C:\Script> $habr_links = $web.Links | Where-Object href -like '*habr.com*'
```

В `$habr_links` находятся два объекта `PSCustomObject`, содержащие различные HTML-атрибуты для ссылок:


```
PS C:\Script> $habr_links.count
```

```
2
```

```
PS C:\Script> $habr_links | Get-Member
```

```
TypeName: System.Management.Automation.PSCustomObject
```

Name	MemberType	Definition
Equals	Method	bool Equals(System.Object obj)
GetHashCode	Method	int GetHashCode()
GetType	Method	type GetType()
ToString	Method	string ToString()
class	NoteProperty	string class=Link Link_theme_normal OrganicTitl...
data-counter	NoteProperty	string data-counter=["b"]
data-log-node	NoteProperty	string data-log-node=bf7fw01-00
href	NoteProperty	string href=https://habr.com/ru/company/ruvds/b...
innerHTML	NoteProperty	string innerHTML=<div class="Favicon Favicon_si...
innerText	NoteProperty	string innerText=...
outerHTML	NoteProperty	string outerHTML=<a tabindex="0" class="Link Li...
outerText	NoteProperty	string outerText=...
tabindex	NoteProperty	string tabindex=0
tagName	NoteProperty	string tagName=A
target	NoteProperty	string target=_blank

Например, выведем значение атрибутов href и innerText для этих ссылок:

```
PS C:\Script> $habr_links | ForEach-Object {$_.href + " - " + $_.innerText }
https://habr.com/ru/company/ruvds/blog/487876/ -
Что такое Windows PowerShell и с чем его едят? / Хабр
https://habr.com/ru/company/ruvds/blog/487876/ -
habr.com>ru/company/ruvds/blog/487876/
```

В свойстве `ParsedHtml` объекта `HtmlWebResponseObject` содержится объект типа `mshtml.HTMLDocumentClass`, который предоставляет доступ к DOM-дереву загруженной HTML-страницы.

```
PS C:\Users\andr> $html = $web.ParsedHtml
PS C:\Users\andr> Get-Member -InputObject $html
```

```
TypeName: mshtml.HTMLDocumentClass
```

```
...
```

ЗАМЕЧАНИЕ

Если в HTML-коде имеются сценарии JavaScript, то по умолчанию при построении DOM-дерева они будут выполнены. При необходимости выполнение сценариев можно отключить, указав параметр `-UseBasicParsing`.

В частности, используя методы `getElementById()`, `getElementsByName()` и `getElementsByTagName()`, можно получать объекты, соответствующие HTML-элементам с заданным идентификатором, именем или тегом соответственно.

Например, посмотрим, какой текст записан в первом заголовке второго уровня (HTML-тег `<h2>`):

```
PS C:\Users\andrv> $html.GetElementsByTagName('h2')[0].innerText
```

Документация по PowerShell - PowerShell | Microsoft Docs

Найдем элемент с идентификатором `search-result`:

```
PS C:\Users\andrv> $html.GetElementById('search-result')
```

```
className      : serp-list serp-list_left_yes
id              : search-result
tagName         : UL
parentElement   : System.__ComObject
style           : System.__ComObject
. . .
```

Метод `querySelector()` позволяет найти HTML-элемент по определенному CSS-селектору. Например, обратимся к элементу с классом `main__content`:

```
PS C:\Users\andrv> $html.querySelector('.main__content')
```

```
className      : main__content
id              :
tagName         : DIV
parentElement   : System.__ComObject
style           : System.__ComObject
. . .
```

Подробнее о свойствах, методах и событиях объекта `mshtml.HTMLDocumentClass` можно прочитать в документации на сайте Microsoft (<https://docs.microsoft.com/ru-ru/dotnet/api/system.windows.forms.htmldocument?view=net-5.0>).

Выполнение POST-запросов

Командлет `Invoke-WebRequest` позволяет не только выполнять GET-запросы, но и вызывать другие методы, определенные в протоколе HTTP (`DELETE`, `HEAD`, `MERGE`, `PATCH`, `POST`, `PUT`, `TRACE`). Для этого нужный метод указывается в качестве значения параметра `-Method`.

Рассмотрим пример выполнения запроса с HTTP-методом `POST`, который часто используется для передачи данных из веб-форм или загрузки файлов на сервер. Обращаться мы будем к ресурсу <http://httpbin.org/post>, в результате сервер должен сообщить нам о полученных данных.

Передавать мы будем два параметра с именами `name` и `lastName`, которые поместим в хэш-таблицу `$params`:

```
PS C:\Users\andrv> $params = @{name='Andrey'; lastName='Popov'}
```

Выполним `Invoke-WebRequest` с методом `POST`, поместив передаваемые параметры в тело запроса (параметр `-Body`):

```
PS C:\Users\andr> Invoke-WebRequest -Uri http://httpbin.org/post -Method POST
-Body $params
```

```
Statuscode      : 200
StatusDescription : OK
Content         : {
    "args": {},
    "data": "",
    "files": {},
    "form": {
        "lastName": "Popov",
        "name": "Andrey"
    },
    "headers": {
        "Content-Length": "26",
        "Content-Type": "application/x-www-form..
.
RawContent      : HTTP/1.1 200 OK
                  Connection: keep-alive
                  Access-Control-Allow-Origin: *
                  Access-Control-Allow-Credentials: true
                  Content-Length: 503
                  Content-Type: application/json
                  Date: Fri, 16 Jul 2021 03:03:13 GM...
Forms           : {}
Headers         : {[Connection, keep-alive], [Access-Control-Allow-Origin, *], [Access-Control-Allow-Credentials, true], [Content-Length, 503]...}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 503
```

Отправляемые на сервер параметры командлет `Invoke-WebRequest` автоматически приводит к формату `application/x-www-form-urlencoded`, который используется при передаче данных из веб-форм. Ответ в поле `Content` говорит о том, что сервер принял наши параметры и определил, что они были отправлены из формы.

Иногда бывает нужно передавать на сервер данные в формате JSON, а не в `application/x-www-form-urlencoded`. В этом случае нужно указать параметр `-ContentType` со значением `application/json`. Например:

```
PS C:\Users\andr> $json_params = "{ 'name': 'Andrey', 'lastName': 'Popov' }"
PS C:\Users\andr> Invoke-WebRequest -Uri http://httpbin.org/post -ContentType
"application/json" -Method POST -Body $json_params
```

```
StatusCode      : 200
StatusDescription : OK
```

```

Content          : {
    "args": {},
    "data": "{ 'name':'Andrey', 'lastName':'Popov'
' }",
    "files": {},
    "form": {},
    "headers": {
        "Content-Length": "39",
        "Content-Type": "application/json",
        "Host": "...
RawContent      : HTTP/1.1 200 OK
                  Connection: keep-alive
                  Access-Control-Allow-Origin: *
                  Access-Control-Allow-Credentials: true
                  Content-Length: 475
                  Content-Type: application/json
                  Date: Fri, 16 Jul 2021 03:43:04 GM...
Forms           : {}
Headers         : {[Connection, keep-alive], [Access-Control-Allo
w-Origin, *], [Access-Control-Allow-Credentials
, true], [Content-Length, 475]...}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 475

```

Как видим, при таком способе отправки запроса серверный скрипт, обрабатывающий обращения к ресурсу <http://httpbin.org/post>, извлек полученные данные и при формировании своего JSON-ответа поместил их в поле `data`, а не в поле `form`.

Командлет *Invoke-RestMethod*

Если мы обращаемся к веб-сервису, поддерживающему REST API, то ответ сервера скорее всего будет содержать структурированные данные в формате JSON или XML. Так, в предыдущем примере мы с помощью командлета `Invoke-WebRequest` посылали POST-запрос на ресурс <http://httpbin.org/post> и получали в ответ JSON-строку:

```

PS C:\Users\andr> $web = Invoke-WebRequest -Uri http://httpbin.org/
post -Method POST -Body @{name='Andrey'; lastName='Popov'}
PS C:\Users\andr> $web.Content
{
  "args": {},
  "data": "",
  "files": {},
  "form": {
    "lastName": "Popov",
    "name": "Andrey"
  },

```

```
"headers": {
  "Content-Length": "26",
  "Content-Type": "application/x-www-form-urlencoded",
  "Host": "httpbin.org",
  "User-Agent": "Mozilla/5.0 (Windows NT; Windows NT 10.0; ru-RU)
WindowsPowerShell/5.1.19041.1023",
  "X-Amzn-Trace-Id": "Root=1-60f2a42c-18f152db3b76b5d66d173e31"
},
"json": null,
"origin": "85.95.179.209",
"url": "http://httpbin.org/post"
}
```

Чтобы работать с полями ответа, мы должны преобразовать JSON в объект PowerShell с помощью командлета `ConvertFrom-Json` (мы рассматривали подобные преобразования в *главе 12*):

```
PS C:\Users\andr> $response = $web.Content | ConvertFrom-Json
PS C:\Users\andr> $response
```

```
args      :
data      :
files     :
form      : @{{lastName=Popov; name=Andrey}
headers   : @{{Content-Length=26; Content-Type=application/x-www-form-urlencoded;
              Host=httpbin.org; User-Agent=Mozilla/5.0 (Windows NT; Windows NT 10.0;
              ru-RU) Windows PowerShell/5.1.19041.1023;
              X-Amzn-Trace-Id=Root=1-60f2a42c-18f152db3b76b5d66d173e31}
json      :
origin    : 85.95.179.209
url       : http://httpbin.org/post
```

```
PS C:\Users\andr> $response.form.name
Andrey
```

В подобных случаях, когда от сервера мы получаем структурированные данные, удобнее пользоваться командлетом `Invoke-RestMethod`, который действует аналогично `Invoke-WebRequest` и имеет такие же параметры, но при этом автоматически преобразует ответ от сервера в объект PowerShell.

Выполним наш запрос с помощью `Invoke-RestMethod`, сохранив результат в переменной `$result`:

```
PS C:\Users\andr> $result = Invoke-RestMethod -Uri http://httpbin.org/
post -Method POST -Body @({name='Andrey'; lastName='Popov'})
```

Проверим тип и содержимое переменной `$result`:

```
PS C:\Users\andr> Get-Member -InputObject $result
```

```
TypeName: System.Management.Automation.PSCustomObject
```

Name	MemberType	Definition
Equals	Method	bool Equals(System.Object obj)
GetHashCode	Method	int GetHashCode()
GetType	Method	type GetType()
ToString	Method	string ToString()
args	NoteProperty	System.Management.Automation.PSCusto...
data	NoteProperty	string data=
files	NoteProperty	System.Management.Automation.PSCusto...
form	NoteProperty	System.Management.Automation.PSCusto...
headers	NoteProperty	System.Management.Automation.PSCusto...
json	NoteProperty	object json=null
origin	NoteProperty	string origin=85.95.179.209
url	NoteProperty	string url=http://httpbin.org/post

```
PS C:\Users\andrv> $result
```

```
args      :
data      :
files     :
form      : @{lastName=Popov; name=Andrey}
headers   : @{Content-Length=26; Content-Type=application/x-www-form-urlencoded;
              Host=httpbin.org; User-Agent=Mozilla/5.0 (Windows NT; Windows NT 10.0;
              ru-RU) Windows PowerShell/5.1.19041.1023;
              X-Amzn-Trace-Id=Root=1-60f2a897-6354ce272644b6d412560a9b}
json      :
origin    : 85.95.179.209
url       : http://httpbin.org/post
```

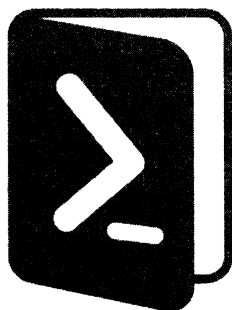
Как видим, вместо строки в формате JSON мы получаем PowerShell-объект типа `System.Management.Automation.PSCustomObject` и можем сразу обращаться к нужным свойствам:

```
PS C:\Users\andrv> $result.form.lastName
Popov
```

Итак, для выполнения HTTP-запросов к веб-ресурсам, возвращающим HTML-страницы, удобнее использовать командлет `Invoke-WebRequest`, а для работы с внешними REST API, возвращающими структурированные данные, лучше подойдет командлет `Invoke-RestMethod`.

Итоги

- ❑ Для обращения к веб-ресурсам по протоколу HTTP в PowerShell имеются два стандартных командлета: `Invoke-WebRequest` и `Invoke-RestMethod`.
- ❑ Действуют эти командлеты аналогично друг другу, за исключением того, что `Invoke-RestMethod` автоматически преобразует ответ от сервера в объект PowerShell.



ЧАСТЬ IV

Пишем сценарии

Глава 15. Разработка сценариев PowerShell

Глава 16. Отладка функций и сценариев

Глава 17. Графический интерфейс для сценариев

ГЛАВА 15



Разработка сценариев PowerShell

Удобная и однородная структура команд и конвейеров PowerShell помогает новичкам быстро освоиться в командной строке и без труда автоматизировать несложные задачи управления Windows. Можно сказать, что PowerShell идеально отвечает известному принципу KISS (*Keep it short and simple* или *Keep it simple, stupid*), утверждающему, что простые вещи должны делаться просто. Одно из главных преимуществ PowerShell состоит в том, что пользователи этой оболочки могут долго обходиться вообще без написания и применения специальных сценариев, что было бы невозможно при работе, например, с Windows Script Host.

Однако PowerShell — это не только командный интерпретатор и оболочка командной строки, но и современный скриптовый язык.

Язык PowerShell, с одной стороны, обладает мощными возможностями (он действительно оправдывает первую составляющую своего имени — Power), позволяющими создавать профессиональные сценарии, которые пользуются всей мощностью платформы .NET Framework и других объектных моделей операционной системы, могут взаимодействовать с внешними приложениями и службами с помощью структурированных данных (JSON, XML) и т. д. С другой стороны, код PowerShell зачастую выглядит проще, лаконичнее и интуитивно понятнее программы на традиционных скриптовых языках.

Чтобы начать писать сценарии PowerShell для автоматизации своей работы, не нужно быть профессиональным программистом, разбирающимся в тонкостях объектно-ориентированного программирования и устройстве платформы .NET Framework. Освоив базовые принципы PowerShell, перейти от интерактивной работы к написанию сценариев будет совсем не сложно. Это еще одно подтверждение принципа KISS для PowerShell: простые сценарии пишутся просто, для этого достаточно понимать принцип конвейерной обработки объектов и знать основы синтаксиса языка, которые мы рассматривали в предыдущих главах.

Переход от команд к сценариям

В каком случае стоит создавать отдельные сценарии и каких рекомендаций следует придерживаться при их написании?

Если с помощью PowerShell нужно выполнить какие-то разовые действия (например, создать определенный каталог на жестком диске и скопировать в него, например, все загруженные файлы с расширением jpg), то проще набрать отдельные команды непосредственно в оболочке, не создавая специальные файлы со сценариями.

Если же задача выполняется (пусть даже и с модификациями) периодически, то лучше создать для нее сценарий. Написав один раз сценарий, мы сможем запускать его в любое время, не вводя вновь и вновь вручную находящиеся в нем команды.

ЗАМЕЧАНИЕ

Количество стандартных командлетов, доступных из оболочки по умолчанию, увеличивается с каждой версией PowerShell. Поэтому прежде чем начинать писать собственный код, следует посмотреть, нет ли стандартного решения нашей задачи в самой оболочке или в общем репозитории модулей PowerShell Gallery.

Среды для разработки сценариев

В принципе, писать сценарии PowerShell можно в любом привычном для вас текстовом редакторе, например в стандартном Блокноте Windows. Однако намного удобнее это делать в специальных редакторах или средах разработки, которые поддерживают цветовое выделение синтаксиса и автоматическое дополнение вводимых команд, имеют возможности построчного запуска сценариев и просмотра значений переменных и промежуточных результатов во время выполнения.

Проще всего воспользоваться предустановленной в Windows интегрированной средой сценариев PowerShell (Integrated Scripting Environment, ISE), хотя сейчас PowerShell ISE не развивается и Microsoft рекомендует использовать свой бесплатный редактор Visual Studio Code с расширением для поддержки PowerShell. В данной главе мы кратко рассмотрим оба эти инструмента.

PowerShell ISE

Для запуска встроенной в систему интегрированной среды для написания сценариев PowerShell можно выбрать пункт **Windows PowerShell | Windows PowerShell ISE** в меню **Пуск** либо воспользоваться командной строкой или пунктом **Выполнить** в меню **Пуск** и выполнить команду `powershell-ise`.

В результате откроется окно PowerShell ISE, содержащее меню, панель инструментов, область для редактирования сценариев и консольную панель для вывода результатов, где также можно вводить команды PowerShell (рис. 15.1).

Запуск сценариев и фрагментов

Введем в области редактирования следующие строки:

```
'Привет из PowerShell!'  
"Текущий каталог: $(Get-Location)"  
'Пока!'
```

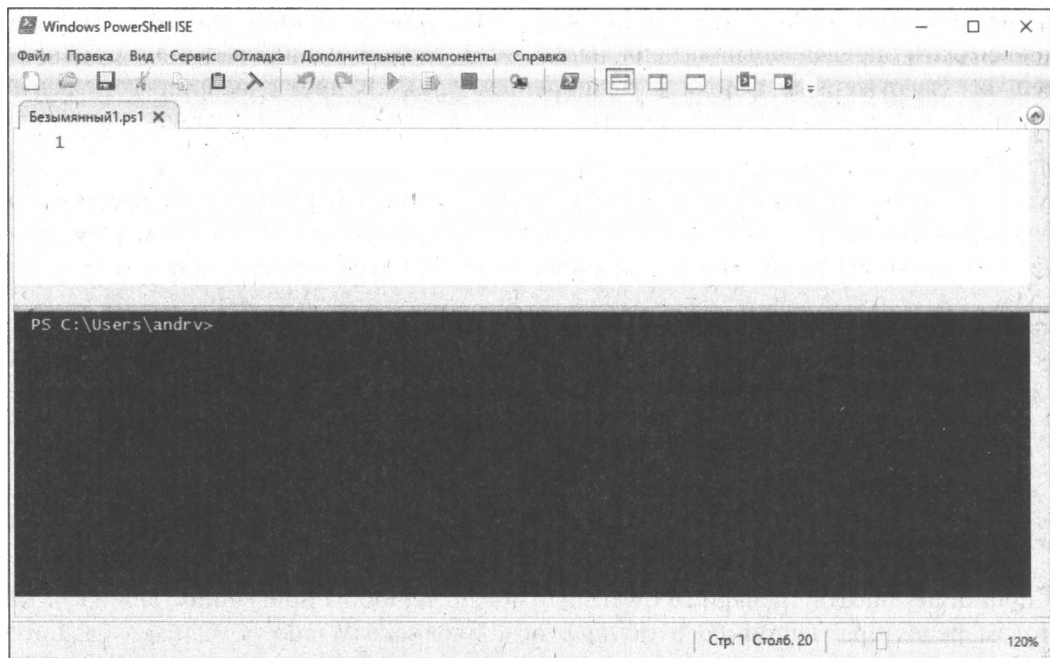


Рис. 15.1. Окно PowerShell ISE

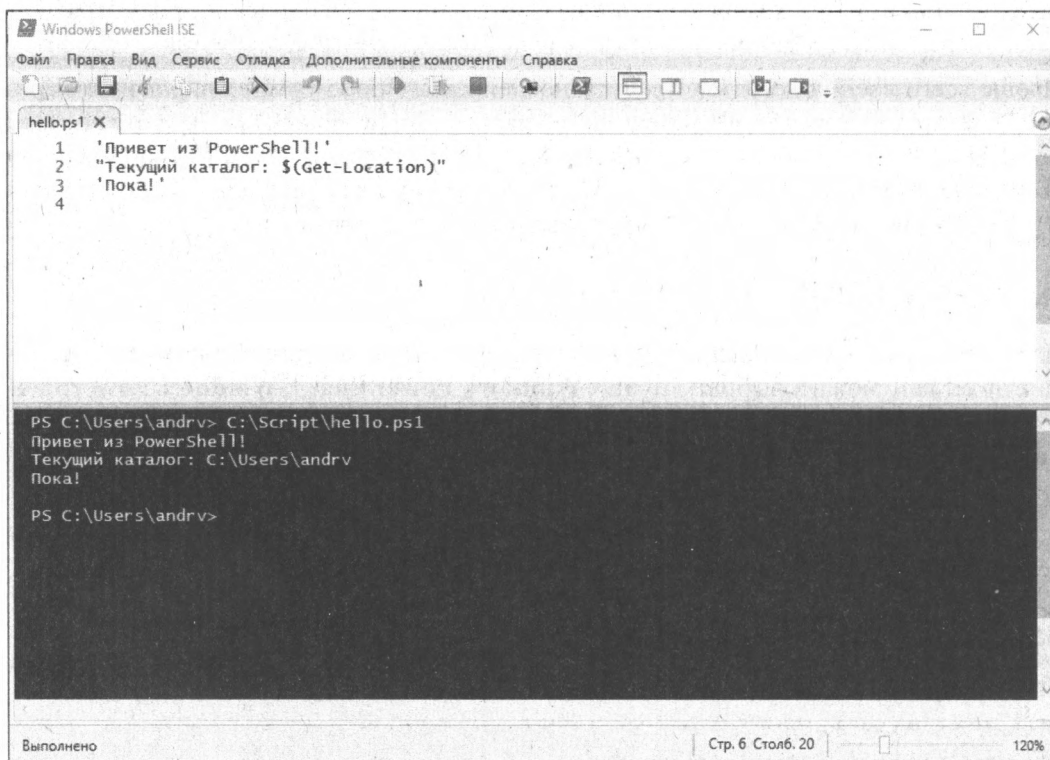


Рис. 15.2. Результат выполнения сценария hello.ps1

Сохраним сценарий под именем `hello.ps1` в каталоге `C:\Script`. Для запуска нашего сценария нужно щелкнуть на зеленом треугольнике на панели инструментов или нажать клавишу `<F5>`. Результат работы сценария будет выведен в консоль внизу экрана (рис. 15.2).

Можно запускать отлаживаемый скрипт по частям. Для этого следует выделить нужные строки и щелкнуть на значке с зеленым треугольником на листе бумаги или нажать клавишу `<F8>`. Например, выполним вторую и третью строки нашего сценария (рис. 15.3).

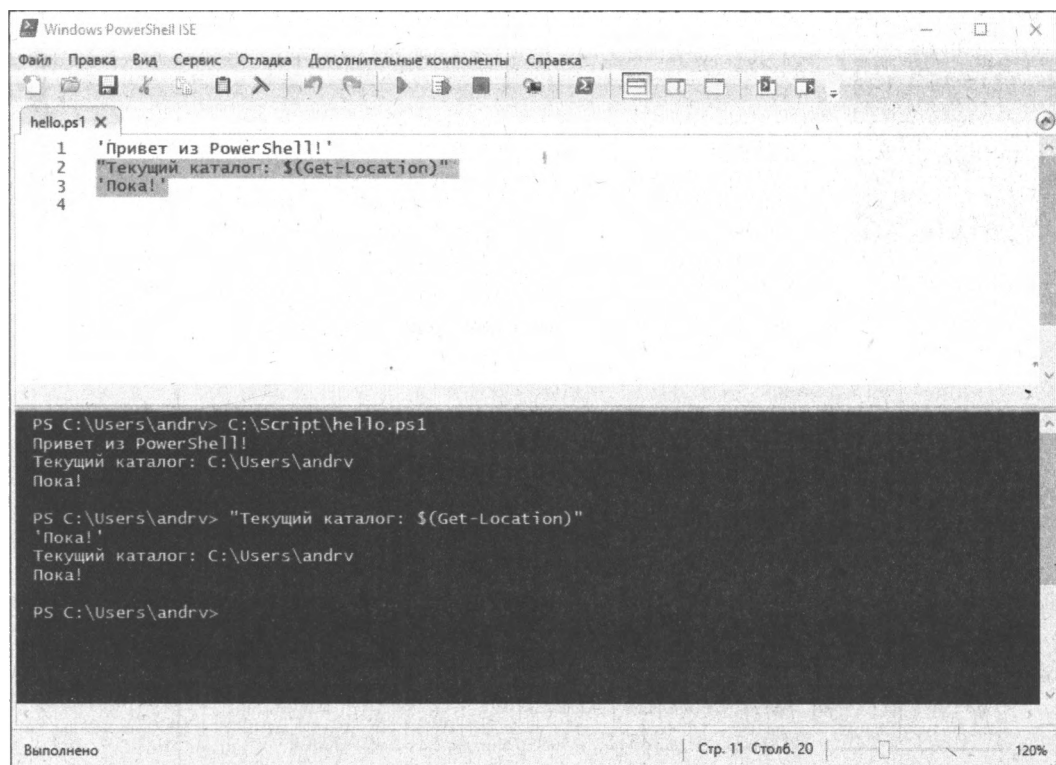


Рис. 15.3. Результат выполнения второй и третьей строк в `hello.ps1`

Справочная система

В PowerShell ISE доступна справочная система PowerShell. Для просмотра справки по команде достаточно поместить на нее курсор и нажать клавишу `<F1>`. При этом по умолчанию информация берется с локальной машины и выводится в отдельном окне (рис. 15.4).

При необходимости можно переключить источник получения и при нажатии клавиши `<F1>` получать онлайн-справку. Для этого нужно снять флажок **Использовать локальную справку вместо справки в Интернете** на вкладке **Общие параметры** диалогового окна с настройками PowerShell ISE (пункт меню **Сервис | Параметры**) (рис. 15.5).

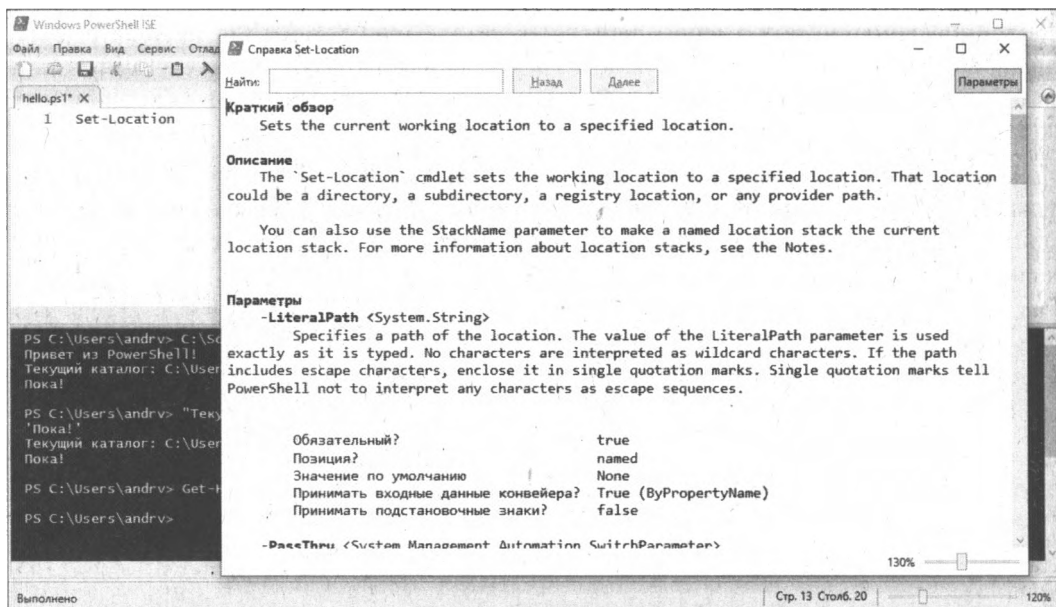


Рис. 15.4. Справка по командам в PowerShell ISE

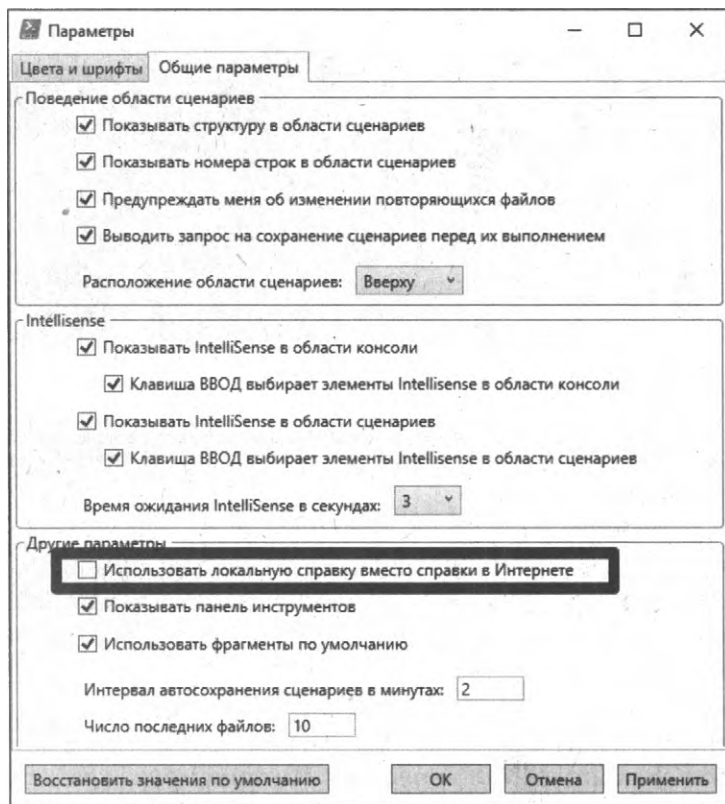


Рис. 15.5. Настройка общих параметров PowerShell ISE

Редактирование текста

Редактор кода, встроенный в PowerShell ISE, поддерживает базовые возможности для поиска и правки текста — соответствующие сочетания клавиш можно посмотреть в пункте меню **Правка** (рис. 15.6).

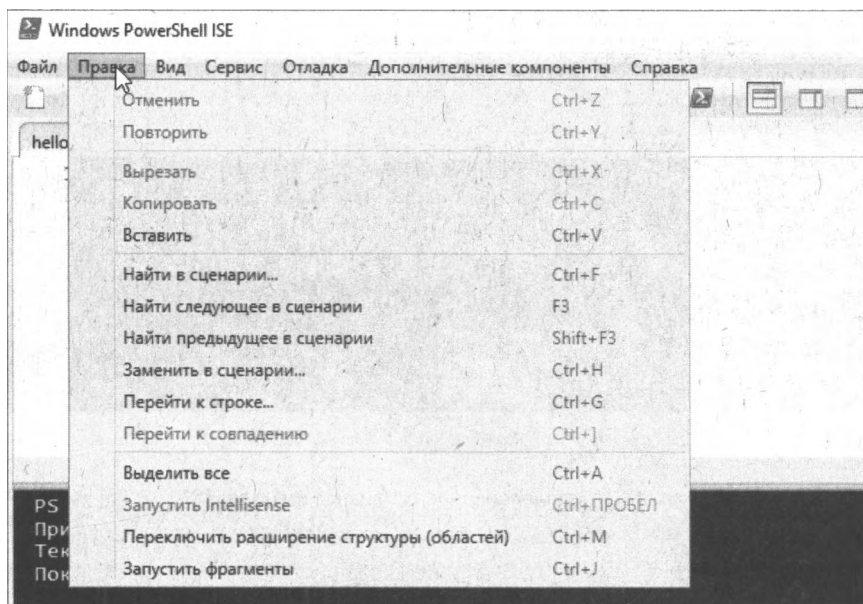


Рис. 15.6. Сочетания клавиш для поиска и редактирования текста в PowerShell ISE

В ISE имеется режим мультикурсора, когда изменения вносятся одновременно в несколько строк, в которых установлен курсор. Для установки дополнительного курсора на строку выше или ниже нужно одновременно зажать клавиши <Alt> и <Shift> и нажать клавишу <↑> или <↓> соответственно.

При вводе команд PowerShell в окне редактора срабатывает автодополнение, появляется список подходящих команд или параметров, из которого можно выбрать нужный пункт (рис. 15.7).

Также поддерживается режим автодополнения, аналогичный механизму в командной строке, когда подходящие команды и параметры подставляются по нажатию клавиши <Tab>.

Работая в PowerShell ISE, можно воспользоваться стандартными фрагментами кода (сниппетами). Для этого нужно нажать комбинацию клавиш <Ctrl>+<j> и выбрать нужный сниппет из списка (рис. 15.8).

Например, если выбрать сниппет `Cmdlet`, то в текст сценария будет вставлена расширенная функция с названием `Verb-Noun`, содержащая пустые секции `Begin`, `Process` и `End`. Для этой функции будут указаны некоторые параметры и комментарии:

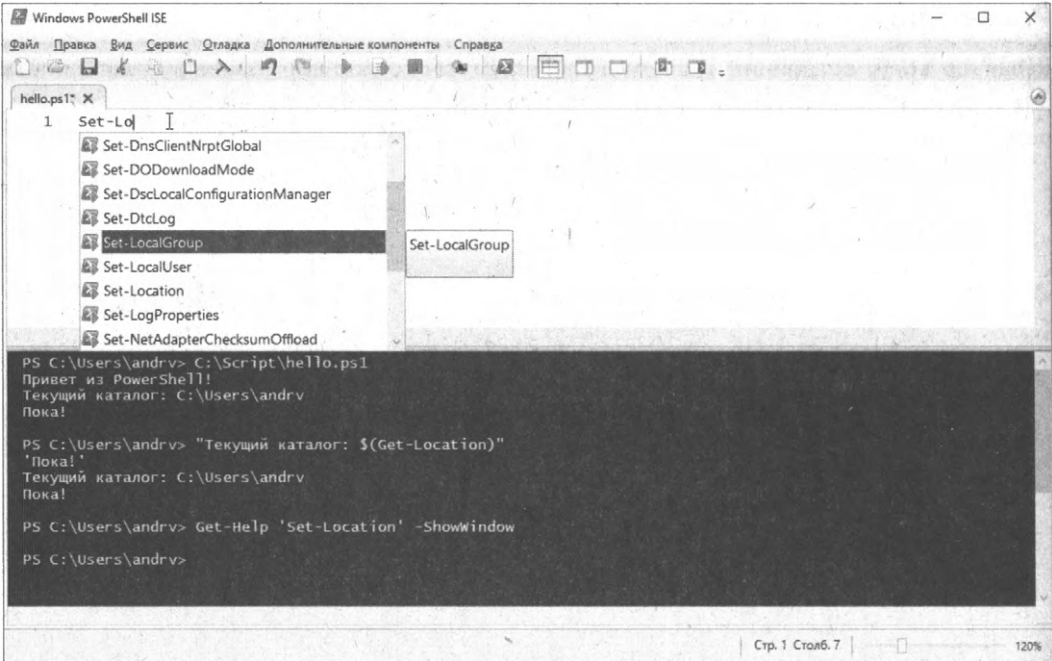


Рис. 15.7. Автодополнение команд в PowerShell ISE

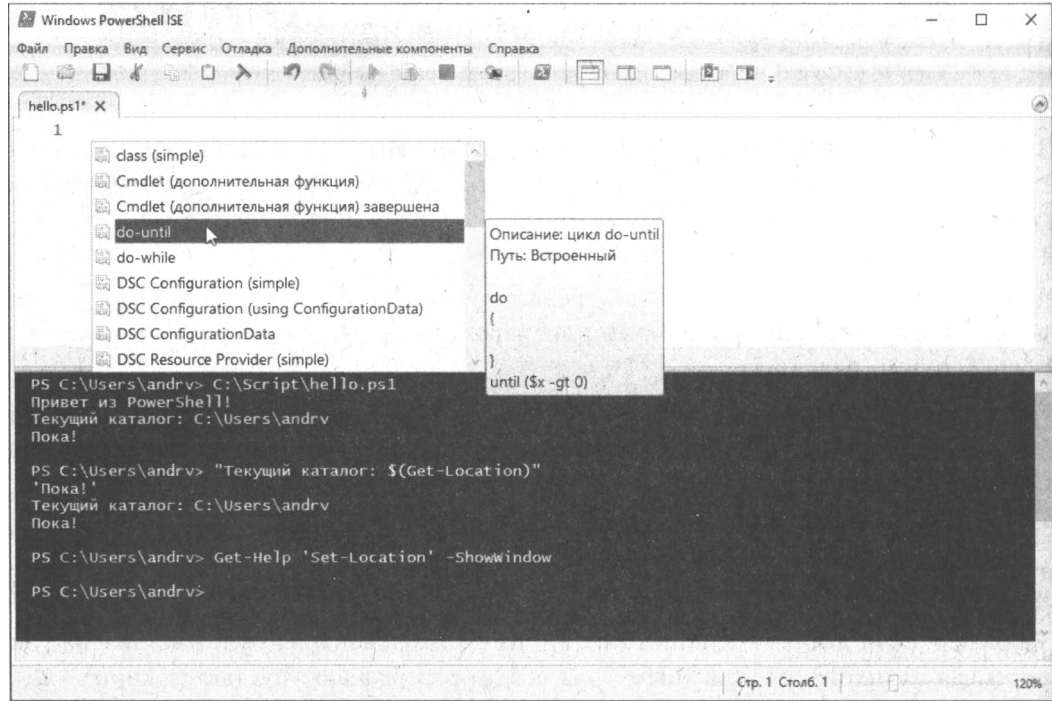


Рис. 15.8. Вставка фрагмента кода в PowerShell ISE

```
<#
.Synopsis
    Краткое описание
.DESCRIPTION
    Длинное описание
.EXAMPLE
    Пример использования этого командлета
.EXAMPLE
    Еще один пример использования этого командлета
#>
function Verb-Noun
{
    [CmdletBinding()]
    [Alias()]
    [OutputType([int])]
    Param
    (
        # Справочное описание параметра 1
        [Parameter(Mandatory=$true,
                    ValueFromPipelineByPropertyName=$true,
                    Position=0)]
        $Param1,

        # Справочное описание параметра 2
        [int]
        $Param2
    )

    Begin
    {
    }
    Process
    {
    }
    End
    {
    }
}
```

Исправить и дополнить эту заготовку намного проще и быстрее, чем писать подобную функцию с нуля.

К сожалению, редактор ISE не поддерживает некоторые возможности редактирования программного кода, которые в других редакторах и средах разработки относятся к стандартным. Например, в ISE нет функций автоматического отступа или форматирования для придания коду структурированности, поэтому выделять пробелами вложенный код внутри условий или циклов придется вручную.

Отладка сценариев

В PowerShell ISE сценарии можно выполнять пошагово: если какой-то сценарий работает некорректно, его можно запустить в ISE и проверить, что он делает на каждом шаге, чтобы выявить ошибку.

Команды и комбинации клавиш для отладки сценариев можно посмотреть в пункте меню **Отладка** (рис. 15.9).

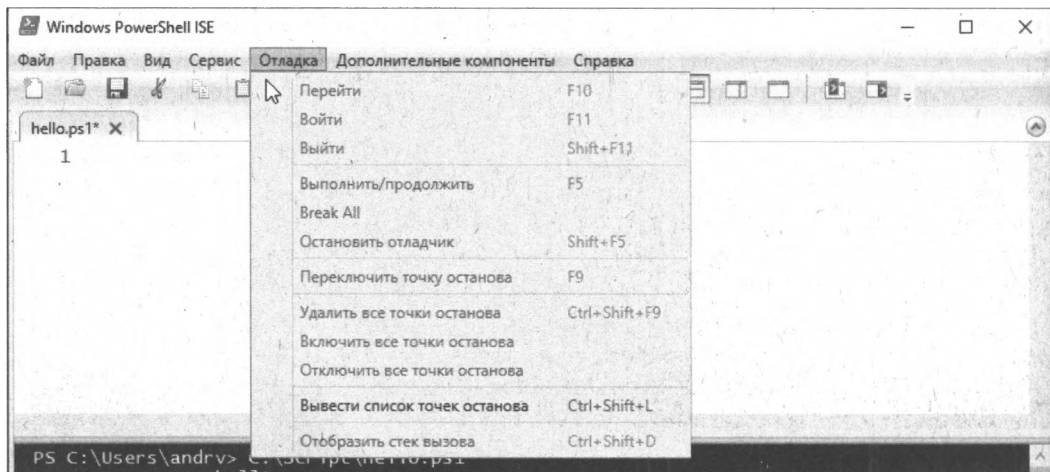


Рис. 15.9. Команды для отладки сценариев в PowerShell ISE

Для того чтобы сценарий приостановил свое выполнение перед определенной строкой, нужно поставить на ней *точку останова* (также их называют контрольными точками или точками прерывания). В ISE точки останова устанавливаются и снимаются нажатием клавиши <F9>.

Рассмотрим пример. Создадим сценарий C:\Script\sum.ps1, в котором будет вычисляться сумма чисел от 1 до 10:

```
"Начало сценария"
$sum = 0
foreach ($i in 1..10)
{
    $sum += $i
}
"Сумма равна $sum"
```

Поставим точку останова на пятой строке, где вычисляется новое значение переменной `$sum`, и запустим сценарий нажатием клавиши <F5>. В консоли мы увидим сообщение о начале сценария и новое приглашение командной строки с префиксом [DBG], указывающим на то, что мы находимся в режиме отладки (рис. 15.10).

Во вложенной командной строке можно выполнять команды PowerShell, проверять или изменять значения переменных, используемых в сценарии. Посмотрим, чему равны переменные `$sum` и `$i` перед первым выполнением пятой строки:

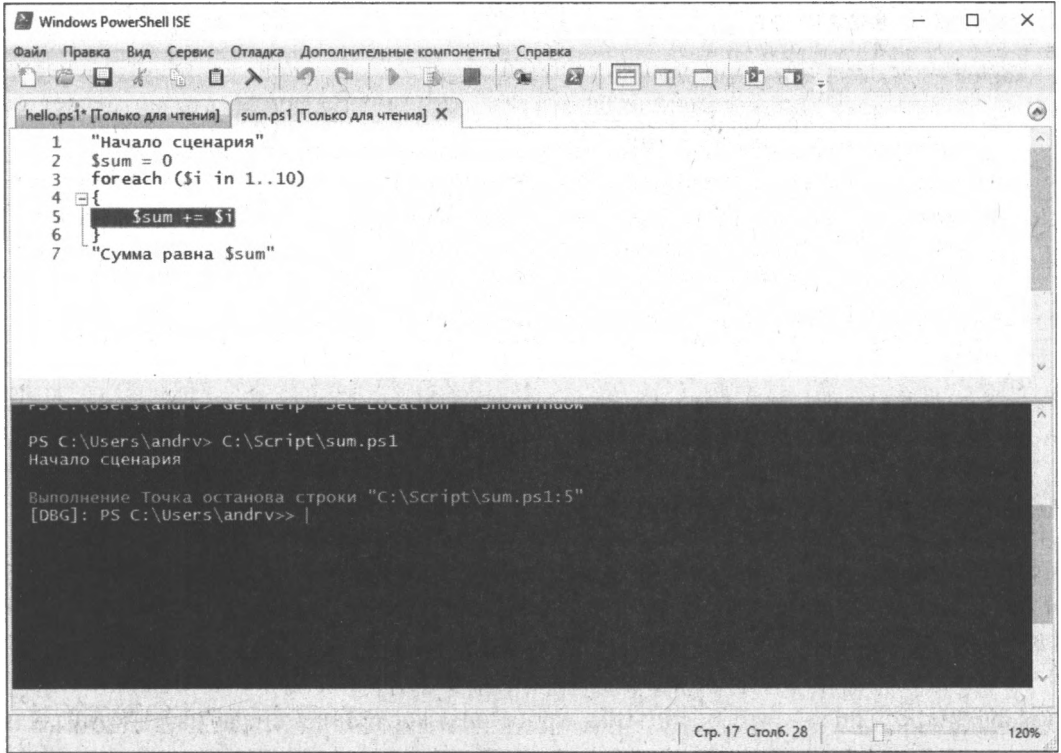


Рис. 15.10. Вложенная командная строка в режиме отладки

```
[DBG]: PS C:\Users\andrv>> $sum
0
[DBG]: PS C:\Users\andrv>> $i
1
```

Для продолжения сценария нажмем клавишу <F5>. Выполнение вновь прервется на пятой строке сценария (это будет вторая итерация цикла). Убедимся, что значения переменных `$sum` и `$i` изменились:

```
[DBG]: PS C:\Users\andrv>> $sum
1
[DBG]: PS C:\Users\andrv>> $i
2
```

После приостановки сценарий можно продолжать выполнять в разных режимах (табл. 15.1).

Таблица 15.1. Варианты пошагового выполнения сценария

Клавиша	Режим выполнения
<F10>	Пропуск блока. Запускается только следующий оператор. Если оператор является вызовом функции, определенной пользователем, то PowerShell запускает эту функцию и останавливается после ее завершения.

Таблица 15.1 (окончание)

Клавиша	Режим выполнения
<F11>	Вход в блок. Запускается только следующий оператор. Если оператор является вызовом функции, определенной пользователем, то PowerShell переходит на первый оператор в этой функции и там останавливается.
<Shift>+<F11>	Выход из блока. Если отладка производится внутри функции, то эта функция выполняется до конца, после чего управление передается в строку сценария, где данная функция была вызвана, и выполнение останавливается.
<F5>	Выполнение до следующей точки останова или до конца файла.

Более подробно процесс отладки сценариев из командной строки мы рассмотрим в *главе 16*.

Visual Studio Code

Сейчас компания Microsoft перестала развивать встроенную среду PowerShell ISE и официально рекомендует разрабатывать сценарии PowerShell с помощью другой своей разработки — мощного, гибкого и универсального редактора Visual Studio Code.

Visual Studio Code — это бесплатная кросс-платформенная среда разработки. Для нее существует огромное количество расширений, позволяющих быстро и удобно писать код практически на любом языке программирования в операционных системах Windows, macOS и Linux.

Устанавливается Visual Studio Code стандартным образом, скачать установочный файл для Windows можно с официального сайта <https://code.visualstudio.com/> (рис. 15.11).

Сразу после установки Visual Studio Code поддерживает подсветку синтаксиса для сценариев на PowerShell и автоматическое форматирование и структурирование кода (напомним, что в PowerShell ISE этой возможности нет). При открытии файла с расширением `ps1` редактор поймет, что это сценарий на языке PowerShell, и порекомендует установить расширение для поддержки этого языка, разработанное Microsoft.

Управлять расширениями Visual Studio Code можно на специальной панели, которая открывается комбинацией клавиш <Ctrl>+<Shift>+<x>. Установим здесь нужное нам расширение (рис. 15.12).

Также это расширение можно установить непосредственно из PowerShell, выполнив следующую команду:

```
PS C:\Users\andrv> code --install-extension ms-vscode.powershell
```

Помимо подсветки синтаксиса после установки расширения PowerShell в Visual Studio Code появляются дополнительные возможности для редактирования и отладки сценариев. Перечислим основные из них.

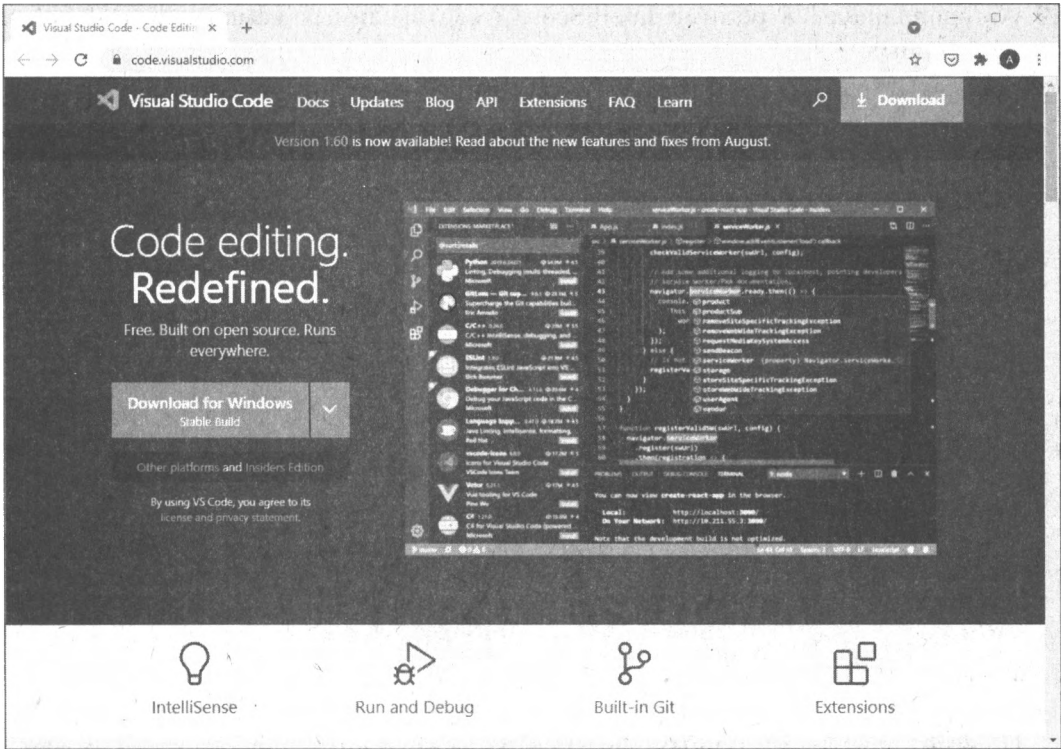


Рис. 15.11. Страница для установки Visual Studio Code

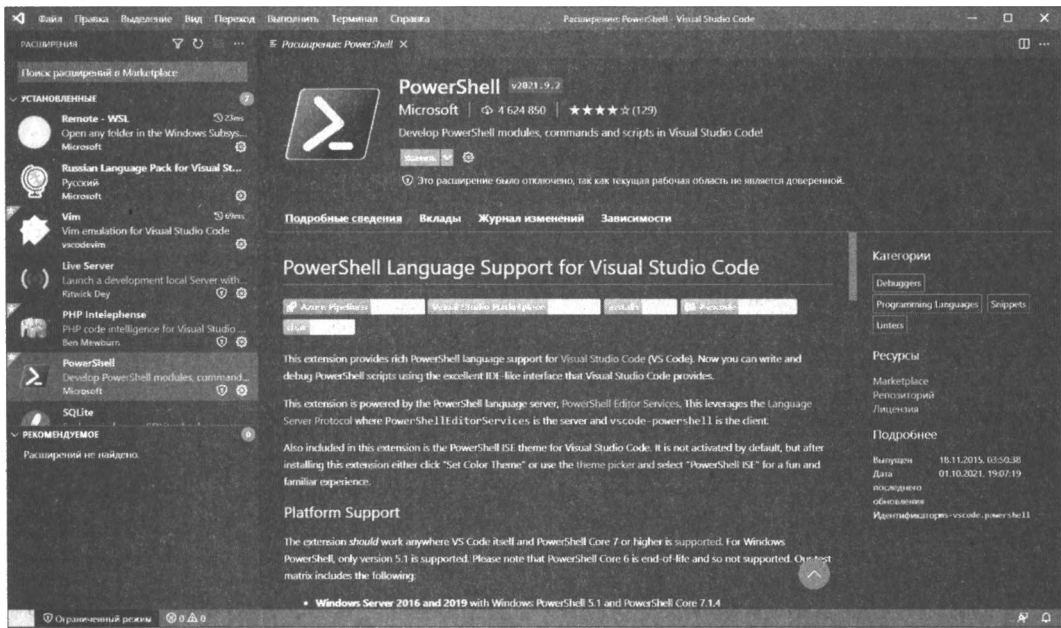


Рис. 15.12. Расширение для поддержки языка PowerShell в Visual Studio Code

- ❑ Автодополнение в режиме IntelliSense имен вводимых команд, параметров и переменных.
- ❑ Вставка в сценарий фрагментов стандартного кода (сниппетов). Нужный фрагмент можно выбрать из списка доступных вариантов, который появляется при вводе. Например, если выбрать сниппет `comment-help`, то в редактор вставится следующий код:

```
<#
.SYNOPSIS
    Short description
.DESCRPTION
    Long description
.EXAMPLE
    PS C:\> <example usage>
    Explanation of what the example does
.INPUTS
    Inputs (if any)
.OUTPUTS
    Output (if any)
.NOTES
    General notes
#>
```

Полный список доступных фрагментов открывается при нажатии клавиш <Ctrl>+<Alt>+<j>.

- ❑ Переход к строке в сценарии, где была инициализирована определенная переменная или описана функция. Для выполнения этого действия нужно щелкнуть правой кнопкой мыши на имени переменной или функции и в контекстном меню выбрать пункт **Перейти к определению** (Go To Definition). Также можно поместить курсор на нужное имя и нажать клавишу <F12>.
- ❑ Поиск всех строк в коде, где используется определенная переменная или функция. Для выполнения этого действия нужно щелкнуть правой кнопкой мыши на имени переменной или функции и в контекстном меню выбрать пункт **Перейти к ссылкам** (Find References). Также можно поместить курсор на нужное имя и нажать клавиши <Shift>+<F12>.
- ❑ Поиск в онлайн-справке по PowerShell для слова, в котором находится курсор, при нажатии клавиш <Ctrl>+<F1>.
- ❑ Автоматическая проверка сценария на соответствие определенным правилам написания кода с помощью *линтера* (статического анализатора) PowerShell Script Analyzer. Это позволяет писать более качественный код. Например, линтер выдаст предупреждение, если вы в тексте сценария используете псевдоним вместо полного наименования командлета или определили функцию, в имени которой используется нестандартный глагол.
- ❑ Проблемный код выделяется в редакторе волнистой линией, при наведении на него мыши можно посмотреть описание проблемы и исправить ее. Полный

список сообщений от линтера выводится на вкладке **Проблемы** (Problems) (рис. 15.13).

- ❑ Сценарии можно запускать в режиме отладки как полностью (клавиша <F5>), так и частично (выделив нужные строки и нажав клавишу <F8>). При этом отлаживаемый код будет запускаться в интегрированной консоли PowerShell (PowerShell Integrated Console), которая связана с редактируемым сценарием.

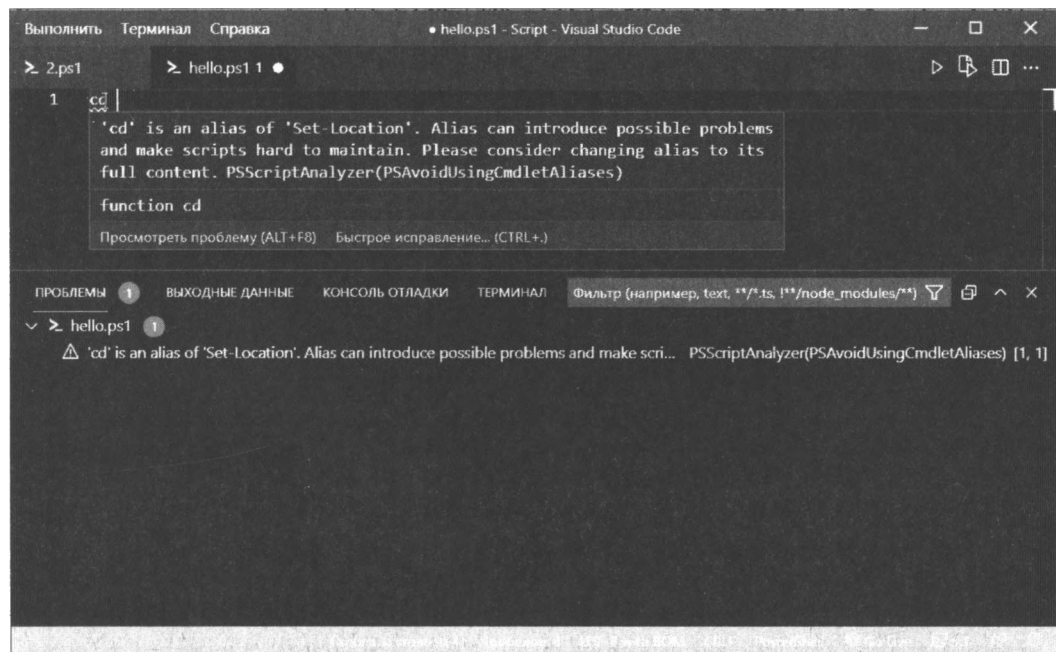


Рис. 15.13. Сообщения о проблемах в коде сценария от PowerShell Script Analyzer

Как и в PowerShell ISE, при отладке сценариев в Visual Studio Code можно работать с точками останова, которые устанавливаются на нужных строках клавишей <F9> (для снятия точки останова нужно нажать клавишу <F9> еще раз). Для продолжения приостановленного сценария используются те же горячие клавиши, что и в PowerShell ISE (см. табл. 15.1).

Все команды, используемые при отладке сценария, доступны в пункте меню **Выполнить** (Debug) (рис. 15.14).

В Visual Studio Code есть удобный инструмент — командная панель, с помощью которой можно, в частности, вызвать любую функцию редактора. Если открыть эту панель комбинацией клавиш <Ctrl>+<Shift>+<p> и ввести слово powershell, то будет выведен список всех команд расширения для поддержки PowerShell (рис. 15.15). При выборе команды из этого списка она будет выполнена.

Настройки самого редактора Visual Studio Code и установленных в нем расширений можно изменять в соответствии со своими предпочтениями с помощью пункта меню **Файл** (File) | **Настройки** (Preferences) | **Параметры** (Settings). В частности,

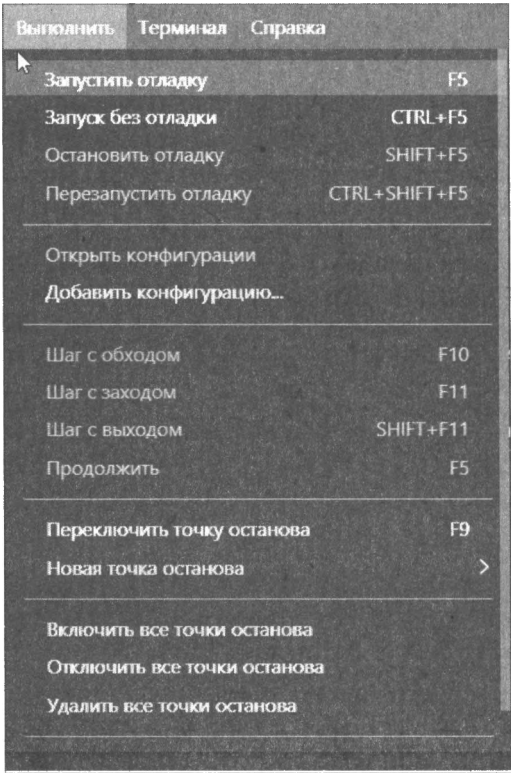


Рис. 15.14. Команды для отладки сценариев в Visual Studio Code

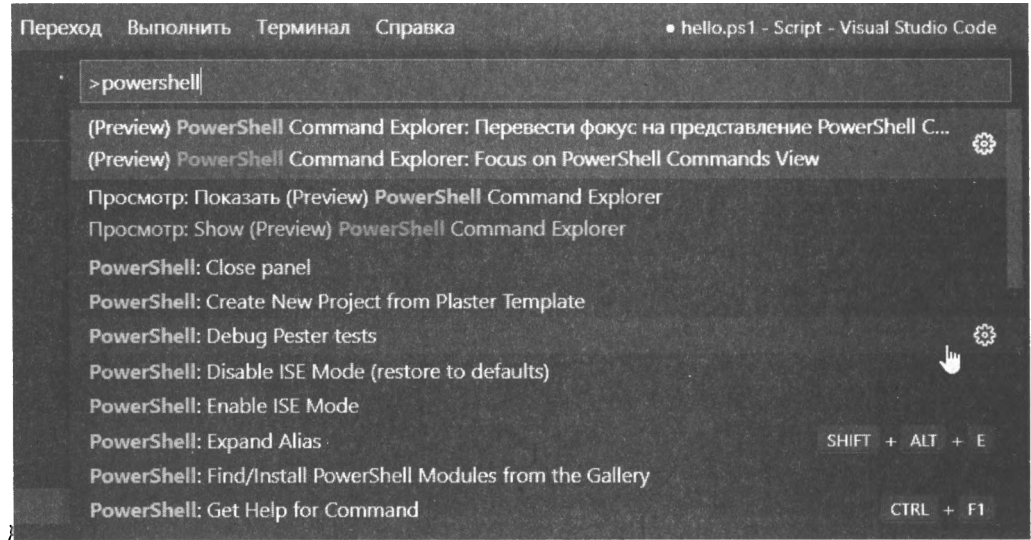


Рис. 15.15. Команды для отладки сценариев в Visual Studio Code

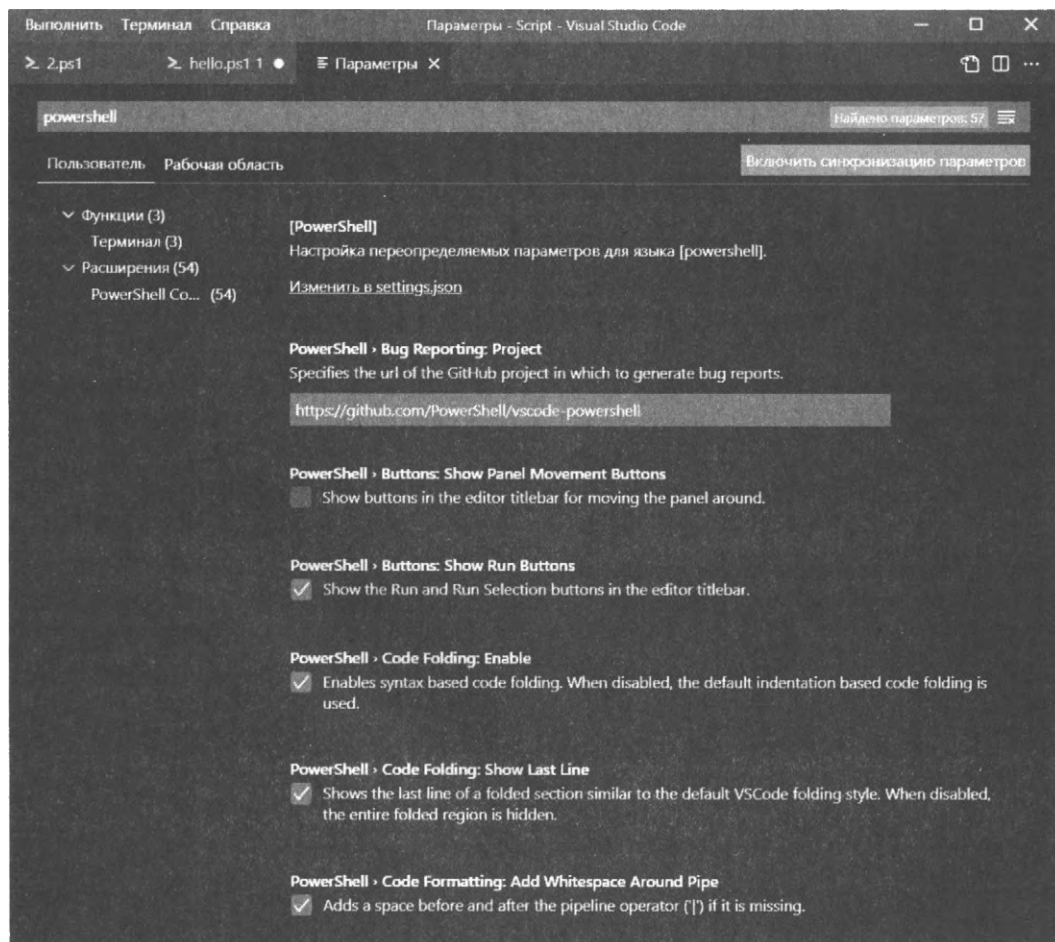


Рис. 15.16. Параметры расширения PowerShell в Visual Studio Code

здесь можно настроить различные параметры, связанные с форматированием кода и другими возможностями расширения PowerShell (рис. 15.16).

Другие редакторы и среды разработки

Несмотря на то, что PowerShell ISE и Visual Studio Code являются рекомендованными инструментами для разработки сценариев PowerShell, мы не ограничены только этими редакторами.

Существуют среды разработки (IDE), предназначенные специально для сценариев. К таким продуктам относятся, например, PowerShell Plus компании Idera (<https://www.idera.com/productssolutions/freetools/powershellplus>) или PrimalScript компании Sapien (<https://www.sapien.com/software/primalscript>).

Отдельно можно выделить очень мощную среду разработки Sapien PowerShell Studio (https://www.sapien.com/software/powershell_studio), позволяющую в том числе быстро создавать графический интерфейс для сценариев.

Конечно, прибегать к специальным IDE для PowerShell (как правило, это коммерческие платные продукты) имеет смысл, только если вы занимаетесь профессиональной разработкой больших и сложных сценариев. В большинстве случаев для написания и отладки скриптов будет достаточно бесплатного текстового редактора, с которым вы привыкли работать. Для большинства популярных универсальных редакторов (например, Notepad++, Sublime Text, Atom и Vim) можно найти плагин или расширение для поддержки языка PowerShell.

Рекомендации по разработке сценариев

Когда человек пишет программу, он общается не только с компьютером, который должен выполнить эту программу, но и с другими людьми, которые будут разбираться в его коде. Профессионализм программиста в большой степени связан с умением передать через исходный код свои знания другим людям. Как говорил Мартин Фаулер: "Любой дурак может написать код, понятный компьютеру. Хорошие программисты пишут код, понятный людям".

Оформляя программу или сценарий, мы объясняем и описываем, что же именно мы хотели тут сделать. Поэтому при написании сценария важно придерживаться определенных рекомендаций по оформлению и документированию кода. Это требует дополнительных усилий, однако если об этом не задумываться, то даже вам самим через месяц будет трудно разобраться в этом сценарии и вспомнить, что и как вы в нем делали.

Общая структура сценария

На простоту чтения и понимания сценариев влияет их структура и стиль, в котором написаны эти сценарии. Две программы, выполняющие одну и ту же задачу, могут восприниматься человеком по-разному. Приведем простой пример. Пусть первый сценарий `hello1.ps1` состоит всего из одной строки:

```
"Привет всем!"
```

Второй сценарий `hello2.ps1` напомним в следующем виде:

```
function WriteHello {  
    $s = "Привет всем!"  
    $s  
}  
WriteHello
```

Запустив эти сценарии, мы получим одинаковый результат — сценарий `hello2.ps1` выводит на экран ту же строку, что и сценарий `hello1.ps1`. Однако второй сценарий намного больше, он содержит описание функции, поэтому для восприятия и анализа он сложнее.

Понятно, что структуры простейших сценариев, состоящих из одного или нескольких конвейеров команд, и сложных больших сценариев (с управляющими структурами, функциями, множеством операторов и т. п.) должны отличаться друг от дру-

га. Небольшие сценарии можно писать в "плоском" формате, без использования функций. Большие сценарии лучше разделять на логические части, определять функции, отдельно инициализировать все использующиеся переменные. Достоинства и недостатки возможных вариантов структур сценариев кратко описаны в табл. 15.2.

Таблица 15.2. Сравнение сценариев, имеющих различную структуру

Структура	Достоинства	Недостатки
В сценарии не используются функции	Самый простой вариант написания сценариев. Хорошо подходит для небольших сценариев (до 100 строк), где отсутствует многократный вызов одного и того же кода.	Тяжело выделить ключевые части сценария. Могут возникнуть трудности с созданием библиотек сценариев, так как в сценариях нет функций.
В сценарии используются функции, которые расположены в тексте неупорядоченно	Никаких достоинств	Подобный сценарий тяжело читать и изменять.
Сценарий организован следующим образом: 1 Раздел инициализации 2 Основное тело сценария 3 Все функции	Подобный сценарий легко читается. Сценарий имеет ясную логическую структуру, поэтому нетрудно понять, как он работает.	При разборе сценария постоянно приходится перемещаться по тексту от описания функций в основное тело сценария. В небольших сценариях, где отсутствует многократный вызов одного и того же кода, данную структуру лучше не использовать.
Сценарий организован следующим образом: 1 Раздел инициализации 2 Все функции 3 Основное тело сценария	Подобная структура упрощает поиск основных операций, выполняемых в сценарии.	При чтении кода приходится искать основное тело сценария, что может несколько осложнить разбор логики сценария.
Сценарий организован следующим образом: 1 Раздел инициализации 2 Вызов главной функции Main, содержащей основное тело сценария 3 Все остальные функции	Данный подход соответствует структуре приложений, принятой во многих языках программирования.	При разборе сценария постоянно приходится перемещаться по тексту от описания функций в основное тело сценария.

Имена и псевдонимы команд и параметров

Краткими псевдонимами удобно пользоваться в интерактивном сеансе оболочки для быстрого набора и однократного выполнения команды. С помощью псевдонимов можно быстро написать компактный и лаконичный конвейер из нескольких команд. Однако при создании сценариев не так важна скорость написания кода, как его понятность.

Поэтому нужно стараться использовать псевдонимы командлетов и сокращенные имена параметров только при интерактивной работе, а в сценариях всегда писать их полные имена. Вы потратите несколько лишних секунд при написании кода, но сэкономите намного больше времени человеку (или самому себе в будущем), который будет читать этот сценарий и разбираться в нем.

```
# Короткая команда с псевдонимами - быстрее набирать
ps | ? id -gt 1000
```

```
# Та же команда с полными названиями - проще читать и понимать
Get-Process | Where-Object id -gt 1000
```

По той же причине в сценариях рекомендуется явно указывать имена даже для тех параметров, которые используются в функциях и командлетах по умолчанию.

```
# Короткий вариант, подходит для интерактивного режима
dir C:\Windows
```

```
# Полный вариант команды для сценария
Get-ChildItem -Path C:\Windows
```

При использовании в функции позиционных параметров может возникнуть проблема: если сигнатура функции (т. е. состав и порядок перечисления ее параметров) в будущем изменится, то старый код может "сломаться". Поэтому позиционных параметров нужно избегать и стараться везде указывать именованные параметры.

При выборе наименования для расширенных функций и сценариев нужно придерживаться стандартного для PowerShell шаблона "действие — объект". При этом глаголы, обозначающие действие, лучше выбирать из списка рекомендованных глаголов, который выводится командой `Get-Verb`:

```
PS C:\Users\andrv> Get-Verb | Format-Wide -Column 4
```

Add	Clear	Close	Copy
Enter	Exit	Find	Format
Get	Hide	Join	Lock
Move	New	Open	Optimize
Pop	Push	Redo	Remove
Rename	Reset	Resize	Search
Select	Set	Show	Skip
Split	Step	Switch	Undo
Unlock	Watch	Backup	Checkpoint
Compare	Compress	Convert	ConvertFrom
ConvertTo	Dismount	Edit	Expand
Export	Group	Import	Initialize
Limit	Merge	Mount	Out
Publish	Restore	Save	Sync
Unpublish	Update	Approve	Assert
Complete	Confirm	Deny	Disable
Enable	Install	Invoke	Register
Request	Restart	Resume	Start

Stop	Submit	Suspend	Uninstall
Unregister	Wait	Debug	Measure
Ping	Repair	Resolve	Test
Trace	Connect	Disconnect	Read
Receive	Send	Write	Block
Grant	Protect	Revoke	Unblock
Unprotect	Use		

Имена параметров в собственных функциях и сценариях должны соответствовать аналогичным параметрам в стандартных командлетах. Например, если вам нужен параметр с именем компьютера, то назвать его нужно `-ComputerName`, как это сделано в командлетах `*-Computer`, `*-Process` или `*-EventLog`, а не `-Comp`, `-PC` или `-Station`.

Расширенные и базовые функции

Напомним, что PowerShell поддерживает два типа функций: базовые и расширенные (см. гл. 9). Базовые функции имеет смысл применять только в простых случаях, когда функция состоит из нескольких строк. В остальных случаях используйте в своих сценариях расширенные функции и пользуйтесь их дополнительными возможностями.

Если в функцию могут поступать какие-либо параметры по конвейеру, то в ней обязательно нужно определить раздел `Process`.

Для возврата значения из функций не нужно использовать оператор `return`, достаточно просто указать в строке нужную переменную или объект. Для эффективного использования конвейеров возвращать значения в расширенных функциях следует в разделе `Process` (напомним, что он выполняется каждый раз при получении по конвейеру нового объекта), а не в разделах `Begin` или `End`.

В расширенных функциях всегда нужно указывать атрибут `[CmdletBinding()]`, в этом случае функция будет вести себя как командлет, в том числе будет поддерживать флаги `-Verbose` (вывод подробной информации о выполняемых командах) и `-Debug` (вывод отладочной информации).

Проверять на корректность (валидировать) значения параметров в расширенных функциях нужно с помощью специальных атрибутов `[Validate*]`, `[Allow*]`, а не писать для этого свой код внутри тела функции.

Вообще, всегда следует использовать встроенные возможности языка или платформы, а не изобретать велосипед. Это относится и к выбору команд PowerShell. Если задачу можно решить как с помощью "родных" команд PowerShell, так и через обращения к методам объектов COM, WMI или .NET, то предпочтение следует отдавать стандартным командлетам оболочки.

Комментарии

Комментарии помогают пояснить назначение и логику работы сценария целиком и отдельных его фрагментов. Рекомендуется комментировать следующие части сценария:

- ☐ весь сценарий в целом;
- ☐ все функции, объявленные в сценарии;
- ☐ большие управляющие структуры, в том числе не входящие в функции;
- ☐ все строки сценария, требующие дополнительного пояснения.

В языке PowerShell комментарии могут быть однострочными и блочными, состоящими из нескольких строк. Однострочные комментарии начинаются со знака #; все символы в строке, стоящие после этого знака, рассматриваются как часть комментария и не интерпретируются системой. Например:

```
PS C:\> # Вся строка - комментарий. Выражение 2+2 не вычисляется
PS C:\> 2 + 2 # Комментарий в конце строки. Выражение до комментария
вычисляется
4
```

Блочные (многострочные) комментарии располагаются между символами <# и #>:

```
<#
Пример
    комментарий
из трех строк
#>
```

Комментарии нужно поддерживать в актуальном состоянии, обновлять их при внесении изменений в программный код. Если комментарий не соответствует коду, то это даже хуже полного отсутствия комментариев.

Если вы создаете сценарий не только для собственного использования, то комментарии нужно писать на английском языке. Текст комментария должен представлять собой законченное предложение. В случае короткого предложения точку в конце можно не ставить.

Справка, основанная на комментариях

При работе с PowerShell мы с помощью команды `Get-Help` постоянно обращаемся к справочной системе для просмотра синтаксиса стандартных встроенных команд-летов, поддерживаемых ими параметров и примеров их использования.

Аналогичную справку можно реализовать и для своих функций и сценариев. Для этого достаточно добавить в сценарий или функцию многострочный комментарий, содержащий определенные теги (ключевые слова). После этого данная справка будет доступна в справочной системе PowerShell, ее можно будет просматривать через вызов `Get-Help`. Такой вид документирования работы сценария или функции называется *справкой, основанной на комментариях* (comment-based help).

В функциях комментарии для генерации справки можно размещать в одном из трех мест:

- ☐ В начале тела функции после открывающей фигурной скобки.
- ☐ В конце тела функции. В этом случае между последней строкой комментария и закрывающей фигурной скобкой, указывающей на завершение функции, должна быть добавлена пустая строка.

❑ Перед ключевым словом `function`. Между последней строкой комментария и словом `function` не должно быть более одной пустой строки.

В сценариях справочные комментарии располагаются либо в самом начале файла (в этом случае перед ними могут быть только обычные комментарии и пустые строки), либо в конце файла.

В качестве примера рассмотрим сценарий `Out-Sum.ps1`, который выводит на экран сумму двух своих аргументов:

```
param ( [int]$a, [int]$b )
$c = $a + $b
"$a + $b = $c"
```

Добавим к этому сценарию справочный комментарий с тегами `.SYNOPSIS` и `.DESCRIPTION`:

```
<#
.SYNOPSIS
    Сценарий Out-Sum
.DESCRIPTION
    Этот сценарий складывает два своих аргумента
и выводит сумму на экран
#>
param ( [int]$a, [int]$b )
$c = $a + $b
"$a + $b = $c"
```

Запросим у PowerShell справочную информацию о сценарии `Out-Sum`:

```
PS C:\Script> Get-Help .\Out-Sum.ps1
```

```
ИМЯ
    C:\Script\Out-Sum.ps1

ОПИСАНИЕ
    Сценарий Out-Sum

СИНТАКСИС
    C:\Script\Out-Sum.ps1 [[-a] <Int32>] [[-b] <Int32>] [<CommonParameters>]

ОПИСАНИЕ
    Этот сценарий складывает два своих аргумента
и выводит сумму на экран

ССЫЛКИ ПО ТЕМЕ

ЗАМЕЧАНИЯ
    Для просмотра примеров введите: "get-help C:\Script\Out-Sum.ps1 -examples".
    Для получения дополнительных сведений введите: "get-help C:\Script\
        Out-Sum.ps1 -detailed"
    Для получения технических сведений введите: "get-help C:\Script\Out-Sum.ps1
        -full".
```

Как видим, PowerShell показывает справку для сценария `Out-Sum.ps1`, содержащую краткое и полное описание сценария (текст берется из содержимого разделов `.SYNOPSIS` и `.DESCRIPTION` справочных комментариев)

Некоторые часто используемые в справочных комментариях теги приведены в табл. 15.3.

Таблица 15.3. Теги в справочных комментариях

Тег	Описание
<code>.SYNOPSIS</code>	Краткое описание функции или сценария
<code>.DESCRIPTION</code>	Детальное описание функции или сценария
<code>.PARAMETER</code>	Описание отдельного параметра
<code>.EXAMPLE</code>	Пример использования команды
<code>.INPUTS</code>	Тип объекта, который может поступать в команду по конвейеру
<code>.OUTPUTS</code>	Типы возвращаемых командой объектов
<code>.NOTES</code>	Дополнительная информация о функции или сценарии
<code>.LINK</code>	Имя связанного раздела в справочной системе или ссылка на сетевой ресурс, содержащий онлайн-версию данной справки

Во многих случаях нет необходимости в справочных комментариях использовать все имеющиеся теги. При написании сценариев и функций нужно стремиться к тому, чтобы код был самодокументируемым, понятным без дополнительных комментариев. Для этого самим сценариям, функциям и параметрам нужно давать полные, осмысленные имена.

Расположение и форматирование кода

Во многих языках программирования применяются стандарты и рекомендации по оформлению кода (например, PEP 8 для Python или PSR-1 и PSR-12 для PHP). Для чего они нужны, ведь оформление исходного кода никак не влияет на работу программы или сценария? Дело в том, что код, отформатированный привычным и стандартным для данного языка образом, легче читать и понимать.

Хотя Microsoft не предлагает официального руководства по стилю написания кода PowerShell, сообщество PowerShell поддерживает сборник лучших практик и рекомендация по оформлению кода, который доступен на GitHub: <https://github.com/PoshCode/PowerShellPracticeAndStyle>.

Регистр символов в именах

Для именования публичных идентификаторов, в том числе модулей, командлетов, функций, методов, констант и глобальных переменных, в PowerShell используется стиль `PascalCase`. В этом случае первая буква каждого слова, входящего в имя, пишется с заглавной буквы. Например: `$GlobalVariable`, `MyFunction`, `Sum-Arguments`.

Имена локальных переменных можно записывать в стиле camelCase, когда с заглавной буквы пишутся имена всех слов, входящих в имя, кроме первого слова. Например, `$args` или `$mySuperVariable`.

Названия ключевых слов и операторов языка PowerShell записываются строчными буквами (`if`, `for`, `switch`, `process`, `-eq`, `-match` и т. д.).

Скобки в коде

Фигурные скобки в коде PowerShell рекомендуется расставлять "единственно верным способом" (также его называют стилем Кернигана и Ритчи), когда открывающая скобка ставится в конце строки, а закрывающая — в начале. Например:

```
enum Color {
    Black,
    White
}

function Test-Code {
    [CmdletBinding()]
    param (
        [int]$ParameterOne
    )
    end {
        if (10 -gt $ParameterOne) {
            "Greater"
        } else {
            "Lesser"
        }
    }
}
```

Исключением из этого правила являются небольшие блоки кода, которые передаются в командлеты в качестве параметров. В этом случае и открывающую, и закрывающую скобку можно указать на одной строке, например:

```
Get-ChildItem | Where-Object { $_.Length -gt 10mb }
```

Отступы, пробелы и пустые строки

Вложенные структуры в коде PowerShell рекомендуется выделять четырьмя пробелами. Например:

```
function Add-Numbers {
    $sum = 0
    foreach ($number in 1..10) {
        $sum += $number
    }
    $sum
}
```


При наборе исходного кода для смещения начала строки на один уровень вправо удобнее пользоваться клавишей табуляции, поэтому в редакторе должна быть настроена автоматическая замена символа табуляции на четыре пробела.

Код будет лучше читаться, если в командах имена параметров или переменных отделять одним пробелом от операторов (в том числе от операторов присваивания и сравнения). Например:

```
# Запись без пробелов
```

```
$a=2+3
```

```
$a-eq5
```

```
# С пробелами код становится понятнее
```

```
$a = 2 + 3
```

```
$a -eq 5
```

Есть исключение из этого правила: дополнительными пробелами не следует выделять унарные операторы (т. е. операторы, имеющие один параметр). Например:

```
# Лишние пробелы в унарных операторах смотрятся плохо
```

```
$yesterdaysDate = (Get-Date).AddDays( - 1)
```

```
$i = 0
```

```
$i ++
```

```
# Без пробелов выглядит лучше
```

```
$yesterdaysDate = (Get-Date).AddDays(-1)
```

```
$i = 0
```

```
$i++
```

Одиночные пробелы нужно вставлять после запятой, точки с запятой, вокруг фигурных скобок и вокруг содержимого внутри скобок. Например:

```
# Вариант без пробелов
```

```
$Numbers = 1,2,3,10
```

```
Get-Service | Where-Object{$_.Status -eq "Stopped"}
```

```
# С дополнительными пробелами читается лучше
```

```
$Numbers = 1, 2, 3, 10
```

```
Get-Service | Where-Object { $_.Status -eq "Stopped" }
```

При описании простой функции рекомендуется разделять пробелом имя функции и список ее параметров в круглых скобках, например:

```
function MyFunction ($param1, $param2) {
```

```
...
```

```
}
```

Отдельные функции в сценарии нужно окружать двумя пустыми строками. Также рекомендуется ставить пустую строку в конце каждого файла-сценария.

В конце строки не должно быть "висячих" пробелов, т. е. пробельных неотображаемых символов.

Точка с запятой как разделитель строк и значений

С помощью точки с запятой можно разделять команды, операторы или выражения внутри одной строки. Например:

```
$a = 1; $b = 2; $c = $a + $b
```

В принципе, можно поставить точку с запятой и в конце строки, ошибки при этом не возникнет. Однако в PowerShell делать это не принято, специального символа для разделения строк указывать не нужно.

Разделитель можно убрать и при перечислении элементов хэш-таблицы, если они расположены на разных строках. Например:

```
$Options = @{  
    Margin    = 2  
    Padding   = 2  
    FontSize  = 24  
}
```

Обратный апостроф для многострочных команд

С помощью обратного апострофа ` можно записать одну команду на нескольких строках. Это позволяет, например, более наглядно оформить вызов командлета или функции с несколькими именованными параметрами:

```
Get-WmiObject -Class Win32_LogicalDisk  
              -Filter "DriveType=3"  
              -ComputerName SERVER2
```

Однако рекомендуется по возможности избегать подобного использования обратного апострофа, т. к. он усложняет отладку сценариев. Этот символ трудно заметить при чтении, а если после него в строке будут стоять пробельные символы, то при выполнении сценария возникнет ошибка.

В подобных случаях лучше предварительно поместить имена и значения параметров в хэш-таблицу, присвоив ее переменной, а при вызове функции использовать сплаттинг этой переменной (см. гл. 9):

```
$GetWmiObjectParams = @{  
    Class = "Win32_LogicalDisk"  
    Filter = "DriveType=3"  
    ComputerName = "SERVER2"  
}  
Get-WmiObject @GetWmiObjectParams
```

Производительность сценариев и продуктивность разработчика

В качестве недостатков PowerShell иногда указывают большое потребление оперативной памяти и медленную работу сценариев по сравнению с другими оболочками и сценариями. Да, это действительно так, ведь за все в этом мире нужно платить.

Повышенное потребление ресурсов — это обратная сторона удобства и мощных возможностей PowerShell. Команды и сценарии PowerShell не сразу выполняются интерпретатором, как это происходит в других оболочках. Они проходят через дополнительный слой .NET Framework, при этом создаются объекты, которые постоянно преобразуются к единому формату для передачи по конвейеру между командами.

Однако на практике вопросы производительности запускаемых сценариев редко являются основными критериями при выборе инструмента автоматизации. Сами базовые понятия языков сценариев выбираются из соображений выразительной мощности и простоты, а не в расчете на максимально эффективную проекцию на архитектуру компьютера.

Основной приоритет — повышение продуктивности труда разработчиков сценариев за счет упрощения используемого языка и применения в сценариях готовых компонентов. По сути сценарии — это склеивающие приложения, где сложность скрыта в используемых компонентах (внешних утилитах и объектах, имеющих в операционной системе).

К сценариям для автоматизации своих действий в операционной системе Windows часто обращаются люди, не занимающиеся программированием профессионально. Для них самым главным является простота освоения языка и скорость написания сценариев, которые будут работать только на их компьютере. В этом случае оболочка PowerShell будет идеальным инструментом, ведь она позволяет практически со скоростью мысли создавать конвейеры из простых и понятных команд, которые потом можно легко оформить в виде сценария. При этом не нужно отвлекаться на сооружение дополнительных конструкций, как в сценариях WSH, или вспоминать названия утилит и их разнородных параметров и аргументов, как в традиционных оболочках командной строки.

К PowerShell в полной мере можно отнести описание языков сценариев из статьи "Природа и эволюция сценарных языков" Р. Богатырева: "Сценарный язык должен ориентироваться на скорость и простоту освоения базовых возможностей, быстро дающих видимый результат. Но из этого не следует, что язык должен быть примитивным. Путь к решению обычных типовых задач даже для начинающего должен быть коротким, простым и понятным. В этом отношении он должен напоминать родной язык: освоить несколько слов и фраз бывает достаточно, чтобы тебя понимали, однако научиться четко, грамотно и красиво выражать мысли на родном языке не всем и не всегда удается даже к глубокой старости".

Пример. Статистика по объектам файловой системы (cmd и PowerShell)

PowerShell легко и изящно справляется с автоматизацией задач вида "Получить множество каких-то объектов, выбрать из них нужные и обработать их" — человек должен только сказать, что нужно сделать, не задумываясь о деталях того, как это будет сделано. Для иллюстрации эффективности сценариев PowerShell с точки зрения времени и сложности разработки рассмотрим один практический пример.

Несколько филиалов банка ежедневно формируют операционные дневники в виде файлов `od*.htm`, которые записываются в папки `ГГГГММДД\NNN`, где `ГГГГММДД` — текущая дата, `NNN` — номер филиала. Например:

```
├──20201201
│   ├──001
│   ├──008
│   ├──011
│   ├──049
│   ├──067
│   ├──076
│   └──099
├──20201202
│   ├──001
│   ├──008
│   ├──011
│   ├──026
│   ├──067
│   ├──076
│   └──100
```

Нужно создать сценарий для сбора статистики, который для каждого филиала подсчитывал бы количество операционных дневников, созданных в течение месяца.

Сначала эта задача решалась в "полуавтоматическом" режиме с помощью командных файлов и электронных таблиц Microsoft Excel. Первый шаг — подготовка файла для анализа:

1. Формируем текстовый файл `temp.txt` с путями к каталогам, в которых есть дневники.
2. Сортируем строки в файле `temp.txt` по номеру филиала и выделяем из них этот номер.
3. Записываем результат в файл `odb.txt`.

Эти действия выполнялись командным файлом интерпретатора `cmd.exe`, в котором указывался нужный год и месяц:

```
@echo off
if exist temp.txt del temp.txt
if exist odb.txt del odb.txt
```

```
rem Формирование файла со списком каталогов, в которых есть
rem дневники за декабрь 2020 года
```

```
for /D %%f in (d:\odb\202012*) do (  
    dir /s/b %%f\od*.htm >> temp.txt)  
  
rem Сортировка строк в файле по номеру филиала (17-й столбец),  
rem выделение из путей этих номеров и запись их в файл odb.txt  
for /f "delims=\ tokens=4" %%i in ('sort /+17 temp.txt') do (  
    echo %%i >> odb.txt)  
del temp.txt
```

Затем полученный файл odb.txt открывался Microsoft Excel, где с помощью стандартной функции формировались промежуточные итоги с подсчетом количества филиалов.

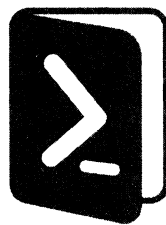
С помощью PowerShell данная задача решается в одну строку с помощью всего двух команд, объединенных в конвейер. Первая команда, `Get-ChildItem`, формирует список объектов-файлов, содержащихся в нужных каталогах, а вторая команда, `Group-Object`, группирует полученные объекты-файлы по имени и выводит на экран нужную нам информацию:

```
Get-ChildItem -Path D:\odb\202012*\*\od*.htm | Group-Object -Property  
($_.DirectoryName) -NoElement
```

Итоги

- ❑ В Windows имеется интегрированная среда сценариев PowerShell ISE, в которой можно писать и отлаживать сценарии PowerShell.
- ❑ Встроенный в ISE редактор предоставляет только базовые возможности по редактированию и форматированию кода, развивать этот продукт Microsoft перестала и рекомендует пользоваться своим бесплатным редактором Visual Studio Code с расширением для поддержки языка PowerShell.
- ❑ Существуют специальные среды разработки для сценариев PowerShell, которыми можно пользоваться при профессиональной работе с большими и сложными сценариями.
- ❑ При написании сценариев желательно придерживаться рекомендаций и лучших практик, принятых в сообществе PowerShell (<https://github.com/PoshCode/PowerShellPracticeAndStyle>).
- ❑ Для документирования сценариев в PowerShell используется механизм справки, основанный на комментариях, благодаря которому справка по конкретному сценарию включается в общую справочную систему PowerShell.

ГЛАВА 16



Отладка функций и сценариев

В PowerShell поддерживается несколько механизмов отладки сценариев, помогающих находить ошибки в программном коде, а также анализировать ход выполнения сценариев и состояние системы при этом. Мы рассмотрим лишь некоторые из них: вывод на консоль диагностических сообщений, трассировка выполняемых команд и установка точек прерывания.

Вывод диагностических сообщений

Самый простой метод отладки — это вывод на экран сообщений о ходе выполнения сценария. С помощью таких сообщений можно проверить значения переменных или убедиться, что определенная строка сценария выполнена. При этом желательно, чтобы диагностические сообщения выделялись от других цветом и не попадали в основной выходной поток `Output`, куда записываются все объекты, генерируемые командами PowerShell.

Такую функциональность предоставляет несколько командлетов с глаголом `Write-*`, которые мы рассматривали в *главе 6*, когда говорили о дополнительных потоках PowerShell.

Попробуем вывести сообщение с помощью `Write-Host`:

```
PS C:\Users\andr> Write-Host "Тестовое сообщение"
Тестовое сообщение
```

Как видим, на экран выводится строка текста. Проверим, что при этом не генерируется никакой объект. Для этого присвоим переменной `$a` результат выполнения командлета `Write-Host` и убедимся, что переменная останется неопределенной (`$a` совпадает с `$null`):

```
PS C:\Users\andr> $a = (Write-Host "Тестовое сообщение")
Тестовое сообщение
PS C:\Users\andr> $a -eq $null
True
```

Командлет `Write-Host` имеет параметры `-ForegroundColor` и `-BackgroundColor`, позволяющие задать цвет текста и цвет фона соответственно. Например, следующая команда выведет желтое сообщение на черном фоне:

```
PS C:\Users\andrv> Write-Host "Тест" -ForegroundColor Yellow -BackgroundColor Black
```

Тест

Если после вывода сообщения нужно приостановить выполнение сценария до нажатия клавиши `<Enter>`, то можно воспользоваться командлетом `Read-Host`:

```
PS C:\Users\andrv> Read-Host "Нажмите Enter"
```

Нажмите Enter

```
PS C:\Users\andrv>
```

Можно также приостановить выполнение сценария до нажатия произвольной клавиши, вызвав следующий метод:

```
PS C:\Users\andrv> $Host.UI.RawUI.ReadKey("NoEcho,IncludeKeyDown")
```

VirtualKeyCode	Character	ControlKeyState	KeyDown
-----	-----	-----	-----
32		NumLockOn	True

Как видим, в этом случае на экран выводится информация о нажатой на клавиатуре клавише. Для подавления этой информации можно перенаправить результат выполнения метода `ReadKey` на пустое устройство:

```
PS C:\Users\andrv> $Host.UI.RawUI.ReadKey("NoEcho,IncludeKeyDown") > $null
```

```
PS C:\Users\andrv>
```

Сообщения, которые выводятся с помощью `Write-Host`, будут отображаться на экране всегда. В PowerShell также есть командлеты `Write-Verbose` и `Write-Debug`, которые выводят сообщения в потоки `Verbose` и `Debug` соответственно. Сообщения из этих потоков отображаются на экране только при определенных режимах запуска функции или сценария с атрибутом `[CmdletBinding()]`.

В поток `Debug` записываются сообщения, используемые при отладке сценариев. Отображаются они только в том случае, если функция или сценарий запускаются с общим параметром `-Debug`. Например, введем следующую функцию `Test-Debug`:

```
PS C:\Users\andrv> function Test-Debug {
>>     [CmdletBinding()]
>>     param ()
>>     $a = 1
>>     Write-Debug "a = $a"
>>     $b = $a + 2
>>     Write-Host "b = $b"
>> }
```

Если просто вызывать эту функцию, то на экран будет выведено сообщение команды `Write-Host`, т. е. значение переменной `$b`:

```
PS C:\Users\andrv> Test-Debug
b = 3
```

Теперь запустим функцию Test-Debug с параметром -Debug:

```
PS C:\Users\andrv> Test-Debug -Debug
ОТЛАДКА: a = 1
Подтверждение
Продолжить выполнение текущей операции?
[Y] Да - Y  [A] Да для всех - A  [H] Прервать команду - H  [S]
Приостановить - S  [?] Справка
(значением по умолчанию является "Y"):
b = 3
```

Как видим, теперь на экран выводится отладочное сообщение из Write-Debug (значение переменной \$a), после чего функция приостанавливается и у пользователя запрашивается режим дальнейшего выполнения.

Поток Verbose предназначен для вывода подробной информации о выполняемой операции. Например:

```
PS C:\Users\andrv> function Sum-TwoArgs {
>>     [CmdletBinding()]
>>     param (
>>         $a, $b
>>     )
>>     Write-Verbose "функция вычисляет сумму двух аргументов: a=$a и b=$b"
>>     $a + $b
>> }
PS C:\Users\andrv> Sum-TwoArgs 1 2
3
PS C:\Users\andrv> Sum-TwoArgs 1 2 -Verbose
ПОДРОБНО: функция вычисляет сумму двух аргументов: a=1 и b=2
3
```

Командлет Set-PSDebug

В PowerShell есть несколько командлетов для отладки сценариев. Самым первым из них появился командлет Set-PSDebug, параметры которого позволяют включить режимы трассировки и пошагового выполнения команд, а также режим обязательного объявления переменных (табл. 16.1).

Таблица 16.1. Параметры командлета Set-PSDebug

Параметр	Описание
-Trace 0	Отключение трассировки
-Trace 1	Включение основного режима трассировки
-Trace 2	Включение полного режима трассировки

Таблица 16.1 (окончание)

Параметр	Описание
-Step	Включение режима пошагового выполнения
-Strict	Включение режима обязательного объявления переменных
-Off	Отключение всех механизмов отладки

Рассмотрим режимы отладки более подробно.

Трассировка выполнения команд

Основной режим трассировки включается следующим образом:

```
PS C:\Users\andr> Set-PSDebug -Trace 1
```

В данном режиме каждая команда, выполняемая интерпретатором, будет отображаться на экране. Например:

```
PS C:\Users\andr> 3+2
ОТЛАДКА: 1+ >>>> 3+2
5
PS C:\Users\andr> $s="abc"
ОТЛАДКА: 1+ >>>> $s="abc"
PS C:\Users\andr> pwd
ОТЛАДКА: 1+ >>>> pwd
```

```
Path
----
C:\Users\andr
```

Отладочная информация выводится на экран другим цветом и с префиксом ОТЛАДКА. Определим теперь функцию MyFunc:

```
PS C:\Users\andr> function MyFunc {
>> $x = $args[0]
>> "x=$x"
>> }
>>
ОТЛАДКА: 1+ function MyFunc {
```

Вызовем данную функцию в цикле:

```
PS C:\Users\andr> foreach ($i in 1..3) {MyFunc $i}
ОТЛАДКА: 1+ foreach ($i in >>>> 1..3) {MyFunc $i}
ОТЛАДКА: 1+ foreach ( >>>> $i in 1..3) {MyFunc $i}
ОТЛАДКА: 1+ foreach ($i in 1..3) { >>>> MyFunc $i}
ОТЛАДКА: 1+ function MyFunc >>>> {
ОТЛАДКА: 2+ >>>> $x = $args[0]
ОТЛАДКА: 3+ >>>> "x=$x"
x=1
```

```

ОТЛАДКА: 4+ >>>> }
ОТЛАДКА: 1+ foreach ( >>>> $i in 1..3) {MyFunc $i}
ОТЛАДКА: 1+ foreach ($i in 1..3) { >>>> MyFunc $i}
ОТЛАДКА: 1+ function MyFunc >>>> {
ОТЛАДКА: 2+ >>>> $x = $args[0]
ОТЛАДКА: 3+ >>>> "x=$x"
x=2
ОТЛАДКА: 4+ >>>> }
ОТЛАДКА: 1+ foreach ( >>>> $i in 1..3) {MyFunc $i}
ОТЛАДКА: 1+ foreach ($i in 1..3) { >>>> MyFunc $i}
ОТЛАДКА: 1+ function MyFunc >>>> {
ОТЛАДКА: 2+ >>>> $x = $args[0]
ОТЛАДКА: 3+ >>>> "x=$x"
x=3
ОТЛАДКА: 4+ >>>> }
ОТЛАДКА: 1+ foreach ( >>>> $i in 1..3) {MyFunc $i}

```

В данном случае на экран выводятся все выполняемые команды, однако вызовы функций и присваивания значений переменным явным образом не выделяются. Включим теперь полный режим трассировки и вновь вызовем функцию `MyFunc` в цикле:

```

PS C:\Users\andrv> Set-PSDebug -Trace 2
ОТЛАДКА: 1+ >>>> Set-PSDebug -Trace 2
PS C:\> foreach ($i in 1..3) {MyFunc $i}
PS C:\Users\andrv> foreach ($i in 1..3) {MyFunc $i}
ОТЛАДКА: 1+ foreach ($i in >>>> 1..3) {MyFunc $i}
ОТЛАДКА: ! CALL function '<ScriptBlock>'
ОТЛАДКА: ! SET $foreach = 'IEnumerator'.
ОТЛАДКА: 1+ foreach ( >>>> $i in 1..3) {MyFunc $i}
ОТЛАДКА: ! SET $i = '1'.
ОТЛАДКА: 1+ foreach ($i in 1..3) { >>>> MyFunc $i}
ОТЛАДКА: 1+ function MyFunc >>>> {
ОТЛАДКА: ! CALL function 'MyFunc'
ОТЛАДКА: 2+ >>>> $x = $args[0]
ОТЛАДКА: ! SET $x = '1'.
ОТЛАДКА: 3+ >>>> "x=$x"
x=1
ОТЛАДКА: 4+ >>>> }
ОТЛАДКА: 1+ foreach ( >>>> $i in 1..3) {MyFunc $i}
ОТЛАДКА: ! SET $i = '2'.
ОТЛАДКА: 1+ foreach ($i in 1..3) { >>>> MyFunc $i}
ОТЛАДКА: 1+ function MyFunc >>>> {
ОТЛАДКА: ! CALL function 'MyFunc'
ОТЛАДКА: 2+ >>>> $x = $args[0]
ОТЛАДКА: ! SET $x = '2'.
ОТЛАДКА: 3+ >>>> "x=$x"
x=2

```

```

ОТЛАДКА:      4+ >>>> }
ОТЛАДКА:      1+ foreach ( >>>> $i in 1..3) {MyFunc $i}
ОТЛАДКА:      ! SET $i = '3'.
ОТЛАДКА:      1+ foreach ($i in 1..3) { >>>> MyFunc $i}
ОТЛАДКА:      1+ function MyFunc >>>> {
ОТЛАДКА:      ! CALL function 'MyFunc'
ОТЛАДКА:      2+ >>>> $x = $args[0]
ОТЛАДКА:      ! SET $x = '3'.
ОТЛАДКА:      3+ >>>> "x=$x"
x=3
ОТЛАДКА:      4+ >>>> }
ОТЛАДКА:      1+ foreach ( >>>> $i in 1..3) {MyFunc $i}
ОТЛАДКА:      ! SET $foreach = ''.

```

Как видим, в полном режиме трассировки отдельно выделяются сообщения о вызове функций (префикс CALL) и о фактическом присвоении значений переменным (префикс SET).

Пошаговое выполнение команд

Рассмотрим еще один режим отладки, позволяющий запускать сценарии в пошаговом режиме (с подтверждением выполнения каждой строки). Включается режим пошагового выполнения команд следующим образом:

```

PS C:\Users\andr> Set-PSDebug -Step
ОТЛАДКА:      1+ >>>> Set-PSDebug -Step

```

Вызовем в цикле функцию MyFunc, определенную в предыдущем разделе:

```

PS C:\Users\andr> foreach ($i in 1..3) {MyFunc $i}

```

Продолжить выполнение текущей операции?

```

      1+ foreach ($i in >>>> 1..3) {MyFunc $i}
[Y] Да - Y [A] Да для всех - A [N] Нет - N [L] Нет для всех - L [S]
Приостановить - S [?] Справка
(значением по умолчанию является "Y"):Y
ОТЛАДКА:      1+ foreach ($i in >>>> 1..3) {MyFunc $i}

```

Продолжить выполнение текущей операции?

```

      1+ foreach ( >>>> $i in 1..3) {MyFunc $i}
[Y] Да - Y [A] Да для всех - A [N] Нет - N [L] Нет для всех - L [S]
Приостановить - S [?] Справка
(значением по умолчанию является "Y"):A
ОТЛАДКА:      1+ foreach ( >>>> $i in 1..3) {MyFunc $i}
x=1
x=2
x=3

```

В данном режиме на экран выводится текущая строка для исполнения, после чего система запрашивает действие, которое необходимо выполнить. Обратите внимание, что в пошаговом режиме, в отличие от режима трассировки, подробная отладочная информация о выполняемых командах не выводится.

Вложенная командная строка

В PowerShell реализована возможность рекурсивного вызова интерпретатора во время интерактивного сеанса работы в оболочке или при выполнении сценария. При этом запускается новый (назовем его вложенным) сеанс оболочки в том же командном окне, а текущий сеанс приостанавливается.

Другими словами, можно приостановить выполнение команд в текущем сеансе или сценарии и общаться с PowerShell, используя так называемую *вложенную командную строку*. Это позволяет выполнять команды PowerShell, анализирующие или изменяющие состояние приостановленного сеанса (не нужны отдельные специальные команды для отладки).

Создать новый вложенный сеанс можно в режиме пошагового выполнения команд, выбирая в запросе на продолжение выполнения текущей операции пункт **Приостановить**. Включим данный режим:

```
PS C:\Users\andr> Set-PSDebug -Step
```

Запустим цикл, который будет выводить числа от 1 до 10:

```
PS C:\Users\andr> $i=0; while ($i++ -lt 10) { $i }
```

Продолжить выполнение текущей операции?

```
1+ >>>> $i=0; while ($i++ -lt 10) { $i }
```

[Y] Да - Y [A] Да для всех - A [N] Нет - N [L] Нет для всех - L [S]

Приостановить - S [?] Справка

(значением по умолчанию является "Y"):

Нажимая клавишу <Enter>, продолжим выполнение шагов до тех пор, пока не будет в первый раз выведено значение переменной \$i:

Продолжить выполнение текущей операции?

```
1+ $i=0; while ( >>>> $i++ -lt 10) { $i }
```

[Y] Да - Y [A] Да для всех - A [N] Нет - N [L] Нет для всех - L [S]

Приостановить - S [?] Справка

(значением по умолчанию является "Y"):

```
ОТЛАДКА: 1+ $i=0; while ( >>>> $i++ -lt 10) { $i }
```

Продолжить выполнение текущей операции?

```
1+ $i=0; while ($i++ -lt 10) { >>>> $i }
```

[Y] Да - Y [A] Да для всех - A [N] Нет - N [L] Нет для всех - L [S]

Приостановить - S [?] Справка

(значением по умолчанию является "Y"):

```
ОТЛАДКА: 1+ $i=0; while ($i++ -lt 10) { >>>> $i }
```

Продолжить выполнение текущей операции?

```
1+ $i=0; while ( >>>> $i++ -lt 10) { $i }
[Y] Да - Y [A] Да для всех - A [N] Нет - N [L] Нет для всех - L [S]
Приостановить - S [?] Справка
(значением по умолчанию является "Y"):
```

На этом шаге мы приостановим выполнение команды, нажав вместо <Enter> клавишу <S>:

Продолжить выполнение текущей операции?

```
1+ $i=0; while ( >>>> $i++ -lt 10) { $i }
[Y] Да - Y [A] Да для всех - A [N] Нет - N [L] Нет для всех - L [S]
Приостановить - S [?] Справка
(значением по умолчанию является "Y"):S
PS C:\Users\andrv>>
```

Итак, мы попали во вложенную командную строку (обратите внимание, что изменился вид приглашения). Другим индикатором работы во вложенном сеансе является ненулевое значение переменной `$NestedPromptLevel` (уровень вложения командной строки):

```
PS C:\Users\andrv>> $NestedPromptLevel
1
```

Во вложенном сеансе можно выполнять те же команды, что и в обычном сеансе PowerShell. Например, проверим, чему равно значение счетчика цикла `$i`:

```
PS C:\Users\andrv>> $i
1
```

На самом деле можно не только проверять значения переменных, но и изменять их. Например, запишем в переменную `$i` число 8:

```
PS C:\Users\andrv>> $i=8
PS C:\Users\andrv>>
```

Выйдем теперь из вложенной командной строки с помощью инструкции `Exit`. На экран вновь будет выдано предложение продолжить выполнение текущей операции:

```
PS C:\Users\andrv>> exit
```

Продолжить выполнение текущей операции?

```
1+ $i=0; while ( >>>> $i++ -lt 10) { $i }
[Y] Да - Y [A] Да для всех - A [N] Нет - N [L] Нет для всех - L [S]
Приостановить - S [?] Справка
(значением по умолчанию является "Y"):
```

Запустим все команды, нажав клавишу <A>:

Продолжить выполнение текущей операции?

```
1+ $i=0; while ( >>>> $i++ -lt 10) { $i }
[Y] Да - Y [A] Да для всех - A [N] Нет - N [L] Нет для всех - L [S]
Приостановить - S [?] Справка
(значением по умолчанию является "Y"):A
```

```
ОТЛАДКА:    1+ $i=0; while ( >>>> $i++ -lt 10) { $i }  
9  
10  
PS C:\>
```

Как видим, переменная `$i` принимает значения 9 и 10, т. е. изменения, произведенные во вложенном сеансе, повлияли на приостановленный сеанс.

Итак, режим пошагового выполнения команд позволяет в любой момент вызвать вложенную командную строку для анализа или изменения состояния интерпретатора. Однако отлаживать более-менее большие сценарии с помощью данного метода часто оказывается неудобно, т. к. для запуска вложенного сеанса в определенной строке сценария придется каждый раз добираться до этой строки с самого начала сценария.

Гораздо удобнее было бы использовать *точки останова*, позволяющие запускать сценарий в автоматическом режиме и приостанавливать выполнение или запускать определенный код при достижении нужной строки.

Управление точками останова (командлеты **-PSBreakPoint*)

Работать с точками останова в сценариях можно с помощью графического интерфейса в редакторах PowerShell ISE (пункт меню **Отладка** (Debug), рис. 16.1) или Visual Studio Code с установленным плагином поддержки PowerShell (пункт меню **Выполнить** (Run), рис. 16.2).

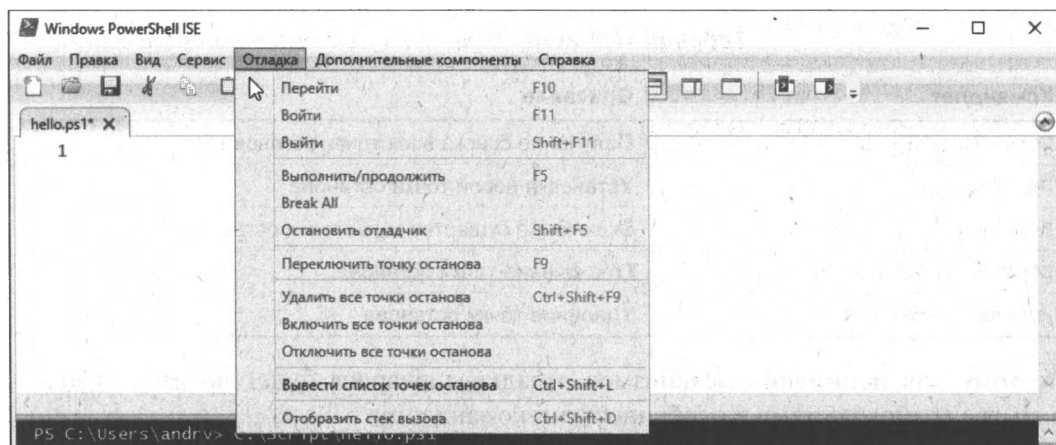


Рис. 16.1. Пункты меню для управления точками останова в PowerShell ISE

В обоих редакторах нажатием клавиши <F9> можно переключить точку останова на строке сценария, запустить выполнение сценария клавишей <F5>, пошагово выполнять команды с помощью клавиш <F10> и <F11>.



Рис. 16.2. Пункты меню для управления точками останова в Visual Studio Code

На самом деле инструменты для отладки скриптов, которые встроены в среды разработки, используют функциональность нескольких командлетов PowerShell, предназначенных для управления точками останова (табл. 16.2).

Таблица 16.2. Командлеты для управления точками останова

Командлет	Описание
Get-PSBreakPoint	Получение списка всех точек останова
Set-PSBreakPoint	Установка новой точки останова
Enable-PSBreakPoint	Включение существующей точки останова
Disable-PSBreakPoint	Отключение точки останова
Remove-PSBreakPoint	Удаление точки останова

Поэтому для понимания механизмов отладки сценариев будет полезно познакомиться с возможностями и особенностями командлетов *-PSBreakPoint.

Создание точки останова для сценария

Создадим в каталоге C:\Script сценарий fact.ps1 для вычисления факториала числа:

```
PS C:\Script> @'  
>> $n = 8
```

```
>> "Вычисляем факториал $n"
>> $fact = 1
>> foreach ($i in 1..$n) {
>>     $fact = $fact * $i
>> }
>> "Факториал $n равен $fact"
>> '@ > fact.ps1
```

Установим с помощью команды `Set-PSBreakPoint` новую точку останова, которая сработает на пятой строке сценария `fact.ps1`:

```
PS C:\Script> $bp1 = Set-PSBreakpoint -Script .\fact.ps1 -Line 5 `
>> -Action {
>>     if ($i -gt 2 -and $i -lt 5)
>>     {
>>         Write-Host "*** ОТЛАДКА: i = $i, fact = $fact"
>>     }
>> }
```

Блок кода, указанный в качестве значения параметра `-Action`, будет срабатывать всякий раз перед выполнением пятой строки:

```
PS C:\Script> .\fact.ps1
Вычисляем факториал 8
*** ОТЛАДКА: i = 3, fact = 2
*** ОТЛАДКА: i = 4, fact = 6
Факториал 8 равен 40320
```

Команда `Set-PSBreakPoint` возвращает объект типа `System.Management.Automation.LineBreakpoint`, который соответствует созданной точке останова:

```
PS C:\Script> $bp1 | Get-Member

TypeName: System.Management.Automation.LineBreakpoint
```

Name	MemberType	Definition
Equals	Method	bool Equals(System.Object obj)
GetHashCode	Method	int GetHashCode()
GetType	Method	type GetType()
ToString	Method	string ToString()
Action	Property	scriptblock Action {get;}
Column	Property	int Column {get;}
Enabled	Property	bool Enabled {get;}
HitCount	Property	int HitCount {get;}
Id	Property	int Id {get;}
Line	Property	int Line {get;}
Script	Property	string Script {get;}

Посмотрим значение свойств объекта `$bp1`:

```
PS C:\Script> $bp1 | Format-List *
```

```
Column      : 0
Line        : 5
Action      :
              if ($i -gt 2 -and $i -lt 5)
              {
                  Write-Host "*** ОТЛАДКА: i = $i, fact = $fact"
              }

Enabled     : True
HitCount    : 8
Id          : 0
Script      : C:\Script\fact.ps1
```

Как видим, каждая точка останова имеет свой целочисленный идентификатор `Id` (нумерация начинается с нуля). В объекте хранятся: путь к отлаживаемому сценарию (свойство `Script`), номер строки в сценарии (свойство `Line`), статус точки останова (свойство `Enabled`). В свойстве `Action` записан блок кода, который будет выполнен при срабатывании точки останова, а в свойство `HitCount` заносится количество срабатываний данной точки останова в сценарии.

Если в блок кода `-Action` поместить команду `Break`, то выполнение сценария будет приостановлено. Создадим такую точку останова для нашего скрипта `fact.ps1`:

```
PS C:\Script> $bp2 = Set-PSBreakpoint -Script .\fact.ps1 -Line 5 `
>> -Action {
>>     if ($i -eq 3)
>>     {
>>         Write-Host "!!! ОТЛАДКА: i = $i, fact = $fact"
>>         break
>>     }
>> }
```

Запустим сценарий `fact.ps1`:

```
PS C:\Script> .\fact.ps1
```

Вычисляем факториал 8

*** ОТЛАДКА: i = 3, fact = 2

!!! ОТЛАДКА: i = 3, fact = 2

Выполняется запуск в режиме отладки. Введите "h" или "?" для получения справки.

Выполнение Точка останова строки "C:\Script\fact.ps1:5"

```
C:\Script\fact.ps1:5 знак:5
```

```
+     $fact = $fact * $i
```

```
+     ~~~~~
```

```
[DBG]: PS C:\Script>>
```

В этом режиме мы видим строку сценария, перед которой выполнение приостановилось. Вид приглашения командной строки изменился, мы находимся в режиме отладки во вложенной командной строке. Список команд для отладки можно посмотреть, введя `h` или `?`:

```
[DBG]: PS C:\Script>> h
```

<code>s, stepInto</code>	Выполнение одного шага (выполнение шага со входом в функции, сценарии и т. д.)
<code>v, stepOver</code>	Выполнение шага с переходом к следующей инструкции (шаг с обходом функций, сценариев и т. д.)
<code>o, stepOut</code>	Выполнение шага с выходом из текущей функции, сценария и т. д.
<code>c, 'continue</code>	Продолжение операции
<code>q, quit</code>	Прекращение операции и завершение работы отладчика
<code>d, detach</code>	Продолжить операцию и отсоединить отладчик.
<code>k, Get-PSCallStack</code>	Отображение стека вызова
<code>l, list</code>	Вывод на экран исходного кода текущего сценария. Используйте команду "list", чтобы перейти в начало текущей строки, команду "list <m>", чтобы перейти в начало строки <m>, или команду "list <m> <n>", чтобы вывести <n> строк, начиная со строки <m>
<code><ввод></code>	Повторение предыдущей команды, если это была команда <code>stepInto</code> , <code>stepOver</code> или <code>list</code>
<code>?, h</code>	Выводит эту справку.

Чтобы получить указания о настройке командной строки отладчика, введите "help about_prompt".

```
[DBG]: PS C:\Script>>
```

Итак, во вложенной командной строке отладчика доступны все стандартные операции отладки. Например, чтобы продолжить выполнение сценария, нужно ввести команду `c` (Continue):

```
[DBG]: PS C:\Script>> c
*** ОТЛАДКА: i = 4, fact = 6
Факториал 8 равен 40320
PS C:\Script>
```

Здесь мы видим еще одно отладочное сообщение от первой точки останова и итоговое значение вычисленного факториала. Отладка и выполнение сценария завершены, командная строка вернулась в обычный режим ввода команд PowerShell.

Создание точки останова для команды

Можно установить точку останова не только на строку в файле сценария, но и на определенную команду.

Например, создадим функцию Say-Hello и создадим для нее точку останова (имя команды указывается в параметре -Command):

```
PS C:\Script> function Say-Hello {'Привет!'}
PS C:\Script> Set-PSBreakpoint -Command Say-Hello
```

ID Script	Line Command
-- -----	----
2	Say-Hello

Теперь при выполнении команды Say-Hello мы сразу войдем в режим отладки:

```
PS C:\Script> Say-Hello
Выполнение Точка останова команды на "Say-Hello"
```

```
строка:1 знак:20
+ function Say-Hello {'Привет!'}
+
[DBG]: PS C:\Script>>
```

Здесь доступны те же команды для отладки, которые мы рассматривали в предыдущем разделе. Например, выполним код нашей функции в пошаговом режиме (команда s):

```
[DBG]: PS C:\Script>> s
строка:1 знак:21
+ function Say-Hello {'Привет!'}
+
[DBG]: PS C:\Script>> s
Привет!
строка:1 знак:30
+ function Say-Hello {'Привет!'}
+
[DBG]: PS C:\Script>> s
PS C:\Script>
```

Создание точки останова для переменной

В PowerShell можно создать точку останова, которая будет срабатывать при чтении или записи определенной переменной.

Например, создадим подобную точку для входа в режим отладки в момент записи значения в переменную fact в сценарии fact.ps1:

```
PS C:\Script> $bp3 = Set-PSBreakpoint -Script .\fact.ps1
>> -Variable fact -Mode Write -Action {
```

```
>>     if ($fact -gt 6)
>>     {
>>         Write-Host ">>> В переменную fact записано значение $fact"
>>     }
>> }
PS C:\Script>
```

Запустим сценарий `fact.ps1`, на который действуют уже три точки останова (первые две мы установили ранее для пятой строки):

```
PS C:\Script> .\fact.ps1
Вычисляем факториал 8
*** ОТЛАДКА: i = 3, fact = 2
!!! ОТЛАДКА: i = 3, fact = 2
Выполнение Точка останова строки "C:\Script\fact.ps1:5"

C:\Script\fact.ps1:5 знак:5
+      $fact = $fact * $i
+      ~~~~~
[DBG]: PS C:\Script>>
```

После приостановки сценария введем в режиме отладки команду `c`, чтобы продолжить выполнение:

```
[DBG]: PS C:\Script>> c
*** ОТЛАДКА: i = 4, fact = 6
>>> В переменную fact записано значение 24
>>> В переменную fact записано значение 120
>>> В переменную fact записано значение 720
>>> В переменную fact записано значение 5040
>>> В переменную fact записано значение 40320
Факториал 8 равен 40320
```

Как видим, блок кода, заданный для точки останова при записи нового значения в переменную `$fact`, срабатывает, сообщения на экран выводятся.

Просмотр точек останова

Команда `Get-PSBreakPoint` выводит список точек останова, созданных в текущей сессии (при следующем запуске оболочки их не будет):

```
PS C:\Script> Get-PSBreakpoint | Format-List *

Column      : 0
Line        : 5
Action      :
              if ($i -gt 2 -and $i -lt 5)
              {
                  Write-Host "*** ОТЛАДКА: i = $i, fact = $fact"
              }
```

```

Enabled : True
HitCount : 24
Id       : 0
Script   : C:\Script\fact.ps1
Column   : 0
Line     : 5
Action   :
        if ($i -eq 3)
        {
            Write-Host "!!! ОТЛАДКА: i = $i, fact = $fact"
            break
        }

Enabled : True
HitCount : 16
Id       : 1
Script   : C:\Script\fact.ps1

Command  : Say-Hello
Action   :
Enabled  : True
HitCount : 1
Id       : 2
Script   :

AccessMode : Write
Variable   : fact
Action     :
        if ($fact -gt 6)
        {
            Write-Host ">>> В переменную fact записано значение $fact"
        }

Enabled : True
HitCount : 9
Id       : 3
Script   : C:\Script\fact.ps1

```

Как видим, установлены три точки останова с идентификаторами 0, 1 и 2. Параметр `-Type` командлета `Get-PSBreakpoint` позволяет получить список точек останова определенного типа. Например, следующая команда покажет точки останова для команд:

```
PS C:\Script> Get-PSBreakpoint -Type Command | Format-List *
```

```

Command : Say-Hello
Action   :
Enabled  : True
HitCount : 1

```

```
Id      : 2
Script  :
```

Посмотреть точки останова, установленные для строк сценария fact.ps1, можно так:

```
PS C:\Script> Get-PSBreakpoint -Script fact.ps1 -Type Line | Format-List *
```

```
Column   : 0
Line     : 5
Action   :
            if ($i -gt 2 -and $i -lt 5)
            {
                Write-Host "*** ОТЛАДКА: i = $i, fact = $fact"
            }

Enabled   : True
HitCount  : 24
Id        : 0
Script    : C:\Script\fact.ps1

Column   : 0
Line     : 5
Action   :
            if ($i -eq 3)
            {
                Write-Host "!!! ОТЛАДКА: i = $i, fact = $fact"
                break
            }

Enabled   : True
HitCount  : 16
Id        : 1
Script    : C:\Script\fact.ps1
```

Удаление точек останова

Удалить точки останова можно командой `Remove-PSBreakPoint`.

Например, удалим точку останова с идентификатором 2:

```
PS C:\Script> Remove-PSBreakpoint -Id 2
PS C:\Script> Get-PSBreakpoint | Format-Table Id, Script, Line, Variable
```

Id	Script	Line	Variable
0	C:\Script\fact.ps1	5	
1	C:\Script\fact.ps1	5	
3	C:\Script\fact.ps1		fact

Для удаления нескольких (или всех) точек останова нужно сформировать их список с помощью команды `Get-PSBreakPoint` и передать его по конвейеру команде `Remove-PSBreakPoint`. Удалим все наши точки останова:

```
PS C:\Script> (Get-PSBreakpoint).count
3
PS C:\Script> Get-PSBreakpoint | Remove-PSBreakpoint
PS C:\Script> (Get-PSBreakpoint).count
0
```

Итоги

- ❑ Диагностические сообщения для отладки сценариев можно выводить с помощью командлетов `Write-Host`, `Write-Verbose`, `Write-Debug`. В основной выходной поток `Output` эти сообщения попадать не будут.
- ❑ Режимы трассировки или пошагового выполнения сценариев можно включить командой `Set-PSDebug`.
- ❑ Мощным и удобным инструментом для отладки являются точки останова, работать с которыми можно в графических средах разработки сценариев PowerShell ISE и Visual Studio Code.
- ❑ Для управления точками останова из командной строки используются командлеты `*-PSBreakPoint`.

ГЛАВА 17



Графический интерфейс для сценариев

Работая в терминале, мы управляем сценариями с помощью параметров, которые указываются в командной строке при их вызове. Если с нашим сценарием будут работать обычные пользователи, то, добавив к сценарию простой графический интерфейс (GUI), можно избавить их от необходимости запускать терминал и вводить названия параметров.

В Windows PowerShell доступны два варианта создания GUI с помощью объектов .NET Framework. Классические элементы интерфейса Windows можно добавить с помощью библиотеки .NET Windows Forms (WinForms), которая поддерживалась в .NET с самого начала. Более современный вариант дизайна реализуется с помощью технологии Windows Presentation Foundation (WPF), которая была добавлена позже в .NET Framework 3.0.

Построение GUI с помощью Windows Forms

Давайте создадим простейшую форму с надписью и кнопкой, выполняя команды непосредственно в консоли PowerShell.

Как мы знаем, платформа .NET построена таким образом, что для обращения к тем или иным объектам нужно предварительно загрузить в операционную память соответствующую *сборку* (assembly) — динамическую библиотеку определенного вида. Наиболее часто использующиеся сборки загружаются в PowerShell автоматически. Для обращения к объектам WinForms из PowerShell нужно сначала загрузить сборку, поддерживающую эти объекты:

```
PS C:\Users\andr> Add-Type -Assemblyname System.Windows.Forms
```

Графический интерфейс в терминах WinForms — это набор *элементов управления* (кнопки, поля ввода, списки, флажки и т. д.), которые располагаются в *контейнерах* (формы, панели, вкладки и т. д.). Изменяя значения свойств объектов, соответствующих элементам управления, мы определяем внешний вид этих элементов (размер, цвет фона, используемый шрифт и т. д.).

Любое графическое приложение WinForms должно иметь главную форму — объект типа `Windows.Forms.Form`. Создадим нашу главную форму и сохраним соответствующий объект в переменной `$form`:

```
PS C:\Users\andr> $form = New-Object System.Windows.Forms.Form
```

Наша форма пока находится только в оперативной памяти, на экране мы ее не видим.

Зададим заголовок формы (свойство `Text` объекта `$form`):

```
PS C:\Users\andr> $form.Text = 'Первая форма'
```

Поместим на форму надпись — для этого нужно создать объект типа `System.Windows.Forms.Label` и присвоить значение свойству `Text` этого объекта:

```
PS C:\Users\andr> $label = New-Object System.Windows.Forms.Label
PS C:\Users\andr> $label.Text = 'Привет!'
```

Кроме надписи на форму можно помещать другие элементы управления, которым соответствуют объекты из пространства имен `System.Windows.Forms`. Некоторые из этих объектов представлены в табл. 17.1.

Таблица 17.1. Объекты, соответствующие элементам управления

Объект	Описание
Button	Кнопка на форме. Надпись на кнопке задается свойством <code>Text</code>
CheckBox	Флажок (чекбокс) — элемент управления для выбора одного или нескольких пунктов из предложенных вариантов
CheckedListBox	Состоит из списка элементов (<code>ListBox</code>), перед каждым из которых помещен флажок (чекбокс) для возможности выбора
ComboBox	Состоит из поля для ввода текста (<code>TextBox</code>) и связанного с ним раскрывающегося списка значений (<code>ListBox</code>)
DataGridView	Основной элемент управления для представления табличных данных (поддерживает связь с базой данных)
GroupBox	Используется для выделения групп элементов управления (чаще всего радиокнопок), возле которых отображается рамка
ImageList	Контейнер для размещения коллекции изображений, используемых другими элементами управления (например, <code>ListView</code> и <code>TreeView</code>)
ListBox	Список элементов, которые могут быть выбраны. Может содержать простой текст или объекты
Panel	Видимый или невидимый контейнер для группировки элементов управления
PictureBox	Используется для размещения графических изображений в нескольких стандартных форматах
ProgressBar	Индикатор хода выполнения какого-либо процесса
RadioButton	Переключатель (радиокнопка) для выбора одного варианта из списка предложенных

Таблица 17.1 (окончание)

Объект	Описание
TextBox	Элемент для ввода текста в одной или нескольких строках
TreeView	Отображение данных в виде узлов дерева

Для задания размеров и расположения элементов управления используются объекты типа `System.Drawing.Point`, в которые передаются координаты: первая — по горизонтали, вторая — по вертикали (начало координат размещается в верхнем левом углу, ось `X` направлена вправо, ось `Y` — вниз). Нашу надпись мы сместим от верхнего левого угла на 20 пикселей вправо и 10 пикселей вниз. Объект с нужными координатами записывается в свойство `Location` объекта `$label`:

```
PS C:\Users\andrv> $label.Location = New-Object System.Drawing.Point 20,10
```

Пока надпись представляет собой отдельный объект, который не связан с основной формой. Поместим надпись на форму, добавив объект `$label` в коллекцию `$form.Controls` с помощью метода `Add`:

```
PS C:\Users\andrv> $form.Controls.Add($label)
```

Теперь создадим кнопку (объект типа `Windows.Forms.Button`) с надписью "Нажми!":

```
PS C:\Users\andrv> $button = New-Object Windows.Forms.Button
PS C:\Users\andrv> $button.Text = "Нажми!"
```

Зададим координаты для кнопки:

```
PS C:\Users\andrv> $button.Location = New-Object System.Drawing.Point 20,40
```

Поместим кнопку на форму:

```
PS C:\Users\andrv> $form.Controls.Add($button)
```

Теперь добавим нашей форме функциональность — по нажатии кнопки `$button` текст в надписи `$label` будет меняться на "Пока!". Для этого с помощью метода `Add_Click` нужно определить обработчик события `Click` на кнопке. В этот метод передается блок кода, который будет выполнен при нажатии кнопки. В нашем случае достаточно изменить значение свойства `$label.Text`:

```
PS C:\Users\andrv> $button.Add_Click({$label.Text='Пока!' })
```

Если бы обработчик события был более объемным, то удобнее было бы заранее сохранить соответствующий блок кода в переменной, которую затем передать в метод `Add_Click`.

Теперь с помощью метода `ShowDialog` можно отобразить созданную форму с элементами управления на экране (рис. 17.1).

```
PS C:\Users\andrv> $form.ShowDialog()
```

По нажатии кнопки надпись на форме изменится (рис. 17.2).

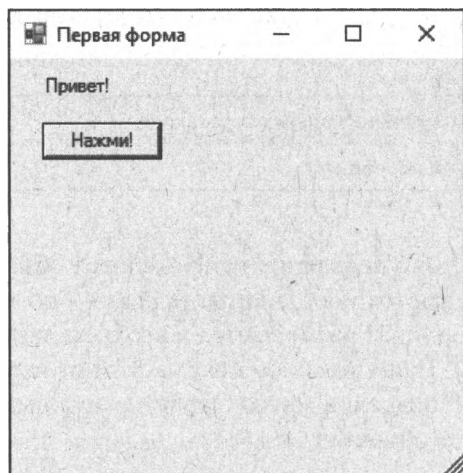


Рис. 17.1. Форма, созданная в PowerShell

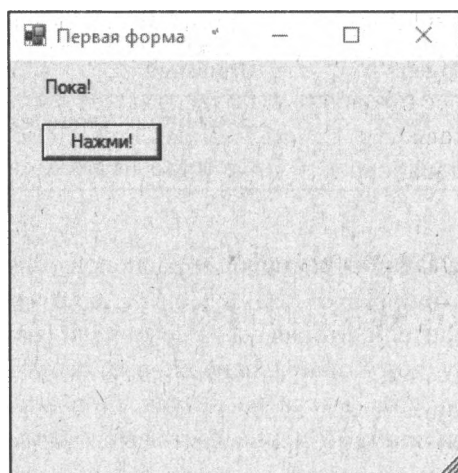


Рис. 17.2. Изменения в элементе на форме

Итак, мы рассмотрели простейший пример создания формы в PowerShell с помощью библиотеки WinForms. Более сложные интерфейсы реализуются по той же схеме:

1. Загружаем сборку `System.Windows.Forms`.
2. Создаем объекты, соответствующие главной форме, контейнерам и элементам управления.
3. Привязываем элементы управления к нужным контейнерам и форме.
4. Задаем свойства элементов управления, определяющие их внешний вид и расположение внутри контейнеров.
5. Пишем блоки кода для обработчиков событий и привязываем их к элементам управления.
6. Показываем главную форму на экране.

Как видим, при создании графического интерфейса на PowerShell с помощью WinForms значительная часть работы будет приходиться на написание шаблонного кода для создания элементов управления и задания их свойств. Конечно, намного удобнее и проще было бы строить интерфейс с помощью визуального конструктора с дальнейшей автоматической генерацией соответствующего кода на языке PowerShell. Подобные конструкторы есть в некоторых платных профессиональных средах разработки для PowerShell, например в Sapien PowerShell Studio (https://www.sapien.com/software/powershell_studio), или в сервисе PowerShell GUI Designer (<https://poshgui.com/>). Из бесплатных инструментов можно выделить утилиту PowerShell WinForms Creator (<https://www.pswinformscreator.com/>), которая сама написана на PowerShell.

Таким образом, при работе с WinForms код для описания интерфейса (создание объектов для элементов управления) оказывается тесно связанным с кодом, определяющим функциональность сценария (обработчики событий на элементах управ-

ления и другие функции). Вносить изменения в такие сценарии неудобно, они быстро разрастаются и становятся слишком объемными и сложными для понимания.

Более правильный подход, когда декларативное описание графического интерфейса отделяется от остального кода, обеспечивает технология Windows Presentation Foundation (WPF).

Построение GUI с помощью Windows Presentation Foundation

При использовании WPF графический интерфейс описывается с помощью базирующегося на XML языка разметки XAML (Extensible Application Markup Language). Таким образом, дизайн интерфейса отделяется от логики сценария, работающего с этим интерфейсом.

Для работы с объектами WPF в PowerShell нужно загрузить сборку PresentationFramework:

```
PS C:\Users\andr> Add-Type -AssemblyName PresentationFramework
```

Создадим сначала простейшее диалоговое окно без элементов размером 300×200 пикселей и с заголовком "Форма WPF". На первом шаге с помощью here-string создадим строку `$xaml` с XML-разметкой, содержащей один тег `<Window/>` с атрибутами `Width`, `Height` и `Title`, и преобразуем эту строку в объект XML:

```
PS C:\Users\andr> [xml]$xaml = @"
>> <Window
>>     xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
>>     xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
>>     x:Name="Window" Title="Форма WPF" Height="200" Width="300"
>> />
>> "@
```

Атрибут `xmlns` здесь определяет пространство имен, содержащее имена элементов и атрибутов, которые можно использовать в разметке Windows Presentation Foundation. Префикс `x` перед атрибутом указывает на то, что данный атрибут относится к данному пространству имен.

Создадим теперь экземпляр класса `XmlNodeReader`, передав ему объект `$xaml` в качестве аргумента. Полученный объект сохраним в переменной `$reader`:

```
PS C:\Users\andr> $reader = (New-Object System.Xml.XmlNodeReader $xaml)
```

Объект, соответствующий диалоговому окну, создается с помощью статического метода `Load()` класса `XamlReader`. В этот метод передается объект `$reader`:

```
PS C:\Users\andr> $window = [Windows.Markup.XamlReader]::Load($reader)
```

Теперь окно можно отобразить на экране с помощью метода `ShowDialog` (рис. 17.3):

```
PS C:\Users\andr> $window.ShowDialog()
```

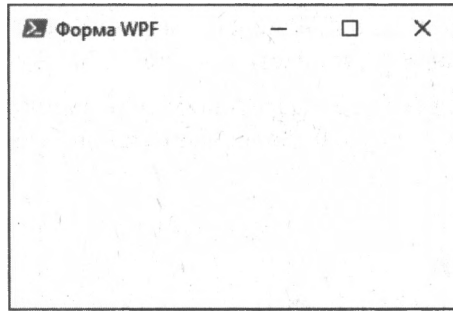


Рис. 17.3. Пустая форма WPF

При этом работа оболочки PowerShell приостановится, для ввода следующей команды нужно закрыть наше диалоговое окно.

Добавим в нашу форму элемент `<Grid>`, задающий сетку для размещения других элементов управления.

```
PS C:\Users\andrv> [xml]$xaml = @"
>> <Window
>>     xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
>>     xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
>>     x:Name="Window" Title="Форма WPF" Height="200" Width="300"
>> >
>>     <Grid x:Name="Grid">
>>         <Grid.RowDefinitions>
>>             <RowDefinition Height="Auto"/>
>>             <RowDefinition Height="Auto"/>
>>         </Grid.RowDefinitions>
>>         <Grid.ColumnDefinitions>
>>             <ColumnDefinition Width="Auto"/>
>>             <ColumnDefinition Width="Auto"/>
>>         </Grid.ColumnDefinitions>
>>     </Grid>
>> </Window>
>> @"
```

Сетка не отображается на экране, поэтому внешний вид главной формы (диалогового окна) не изменится, форма останется пустой.

Поместим теперь на форму надпись (тег `<Label>` с именем `Message`) и кнопку (тег `<Button>` с именем `ChangeMessage`). Надписи в этих элементах задаются с помощью атрибута `Content`, расположение элемента в ячейке сетки определяется с помощью атрибутов `Grid.Column` и `Grid.Row`.

```
PS C:\Users\andrv> [xml]$xaml = @"
>> <Window
>>     xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
>>     xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
```

```

>> x:Name="Window" Title="$Форма WPF" Height="200" Width="300"
>> >
>> <Grid x:Name="Grid">
>>     <Grid.RowDefinitions>
>>         <RowDefinition Height="Auto"/>
>>         <RowDefinition Height="Auto"/>
>>     </Grid.RowDefinitions>
>>     <Grid.ColumnDefinitions>
>>         <ColumnDefinition Width="Auto"/>
>>         <ColumnDefinition Width="Auto"/>
>>     </Grid.ColumnDefinitions>
>>     <Label x:Name="Message"
>>         Content="Привет!"
>>         Grid.Column="0"
>>         Grid.Row="0"
>>     />
>>     <Button x:Name="ChangeMessage"
>>         Content="Нажми!"
>>         Grid.Column="1"
>>         Grid.Row="1"
>>     />
>> </Grid>
>> </Window>
>> "@"

```

Отообразим форму на экране (рис. 17.4):

```

C:\Users\andr> $reader = (New-Object System.Xml.XmlNodeReader $xaml)
PS C:\Users\andr> $window = [Windows.Markup.XamlReader]::Load($reader)
PS C:\Users\andr> $window.ShowDialog()

```

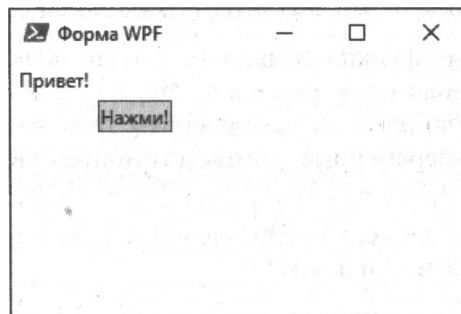


Рис. 17.4. WPF-форма с элементами управления

С помощью команды `Set-Content` поместим XAML-разметку для нашей формы в файл `wpf_test.xaml`:

```

PS C:\Users\andr> @"
>> <Window
>>     xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"

```

```
>> xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
>> x:Name="Window" Title="Форма WPF" Height="200" Width="300"
>> >
>> <Grid x:Name="Grid">
>>     <Grid.RowDefinitions>
>>         <RowDefinition Height="Auto"/>
>>         <RowDefinition Height="Auto"/>
>>     </Grid.RowDefinitions>
>>     <Grid.ColumnDefinitions>
>>         <ColumnDefinition Width="Auto"/>
>>         <ColumnDefinition Width="Auto"/>
>>     </Grid.ColumnDefinitions>
>>     <Label x:Name="Message"
>>         Content="Привет!"
>>         Grid.Column="0"
>>         Grid.Row="0"
>>     />
>>     <Button x:Name="ChangeMessage"
>>         Content="Нажми!"
>>         Grid.Column="1"
>>         Grid.Row="1"
>>     />
>> </Grid>
>> </Window>
>> "% | Set-Content -Path ./wpf_test.xaml -Encoding UTF8
```

Теперь для отображения формы мы можем прочитать ее разметку из файла:

```
PS C:\Users\andr> [xml]$xaml = Get-Content -Path ./wpf_test.xaml -Encoding UTF8
PS C:\Users\andr> $reader = (New-Object System.Xml.XmlNodeReader $xaml)
PS C:\Users\andr> $window = [Windows.Markup.XamlReader]::Load($reader)
```

Осталось добавить форме функциональность, создав обработчики нужных событий, возникающих на элементах управления. Это делается практически так же, как и при использовании WinForms, за исключением того, что нам нужно предварительно создать объекты-переменные, соответствующие элементам управления загруженной формы.

Напомним, что для тегов элементов управления мы задавали атрибут `Name` из пространства имен XAML-разметки для WPF:

```
<Label x:Name="Message"
    . . .
/>
<Button x:Name="ChangeMessage"
    . . .
/>
```

По значению атрибута `Name` с помощью метода `$window.FindName()` получим ссылки на соответствующие элементы управления:

```
PS C:\Users\andrv> $messageLabel = $window.FindName("Message")
PS C:\Users\andrv> $changeButton = $window.FindName("ChangeMessage")
```

Пусть, как и в примере с WinForms, при нажатии кнопки текст надписи в нашей форме должен измениться на "Пока!". Для этого зададим обработчик события Click на кнопке с помощью метода Add_Click:

```
PS C:\Users\andrv> $changeButton.Add_Click({$messageLabel.Content='Пока!'})
```

Теперь форму можно отобразить на экране и проверить результат нажатия кнопки (рис. 17.5).

```
PS C:\Users\andrv> $window.ShowDialog()
```

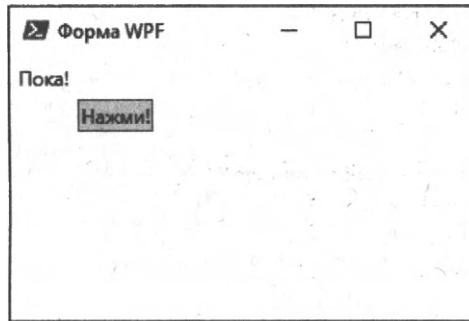


Рис. 17.5. Изменение содержимого элемента управления на WPF-форме

Итак, при работе с графическим интерфейсом с помощью WPF нужно выполнить следующие шаги:

1. Создать XAML-файл с описанием интерфейса.
2. Подключить сборку PresentationFramework.
3. Создать объект, соответствующий форме, загрузив ее описание в формате XAML.
4. Определить переменные (объекты), соответствующие элементам управления в форме.
5. Написать блоки кода для обработчиков событий и привязать их к объектам, созданным на предыдущем шаге.
6. Отобразить главную форму на экране.

Таким образом, при работе с WPF мы можем создавать и изменять дизайн интерфейса независимо от его функционала.

Кроме того, XAML-описание формы не зависит от специфики PowerShell, как это было в случае интерфейса на базе WinForms, поэтому вариантов выбора визуального конструктора для построения форм становится больше. Кроме платных специализированных сред разработки сценариев PowerShell (Sapien PowerShell Studio, PowerShell GUI Designer) можно воспользоваться бесплатной платформой Microsoft — Visual Studio Community Edition. Скачать Visual Studio можно с сайта Microsoft по адресу <https://visualstudio.microsoft.com/ru/vs/community/>.

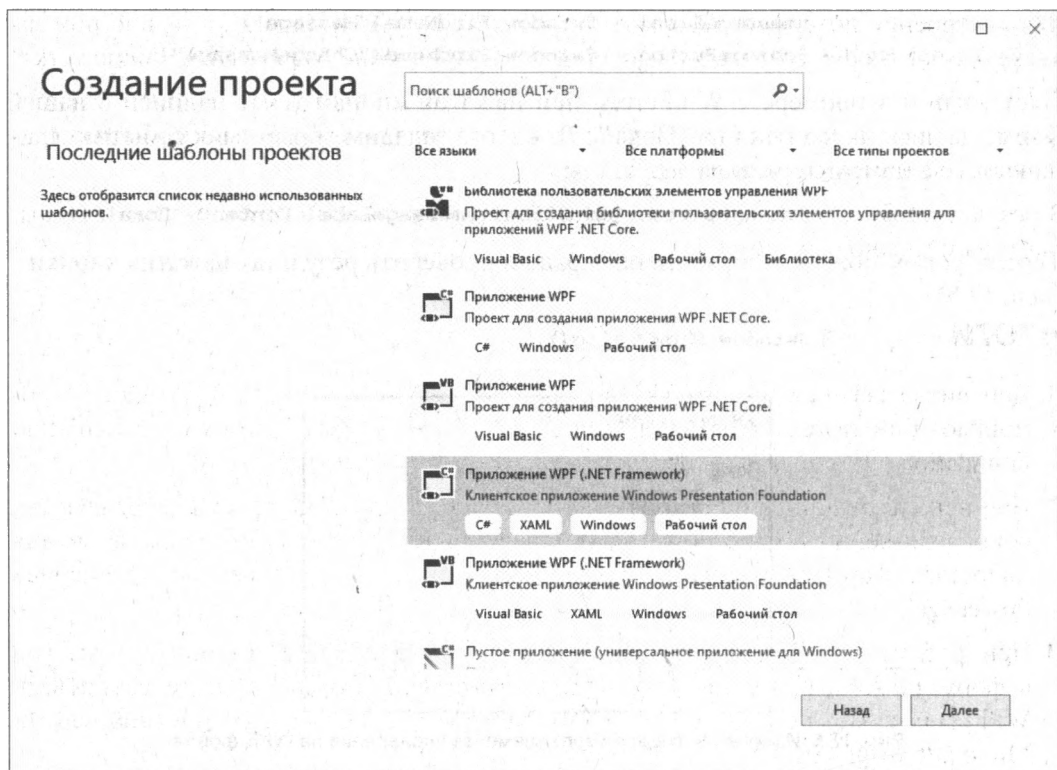


Рис. 17.6. Создание WPF-приложения в Visual Studio

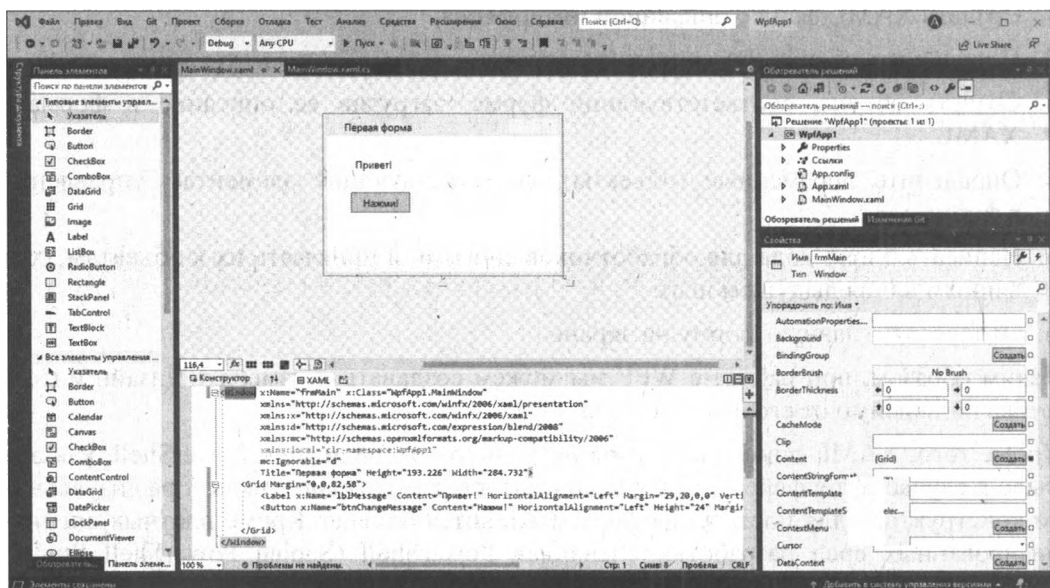


Рис. 17.7. Конструктор для WPF-форм в Visual Studio

Для построения интерфейса формы и получения ее XAML-разметки нужно при запуске Visual Studio создать новый проект "Приложение WPF (.NET Framework)" (рис. 17.6).

При создании такого приложения можно настроить дизайн формы с помощью графического конструктора (рис. 17.7).

В результате будет автоматически сформирован XAML-файл с описанием формы, с которым можно работать в PowerShell.

Итоги

- ❑ Добавить графический интерфейс к сценариям Windows PowerShell можно с помощью библиотек Windows Forms (WinForms) или Windows Presentation Foundation (WPF).
- ❑ При использовании WinForms элементы интерфейса создаются и настраиваются непосредственно в коде PowerShell. Средства для визуального редактирования подобных форм встроены в профессиональные среды разработки сценариев PowerShell.
- ❑ При работе с WPF дизайн интерфейса, описываемый с помощью разметки в формате XAML, не связан с кодом PowerShell. Создавать и редактировать WPF-формы можно с помощью визуального конструктора в бесплатной версии Microsoft Visual Studio.

Что дальше?

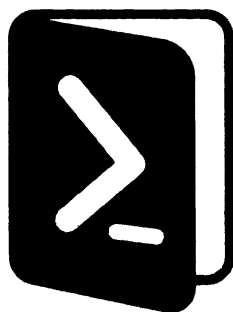
PowerShell для профессионалов

Мы рассмотрели лишь базовые аспекты PowerShell, которых достаточно для уверенной работы в оболочке на своем компьютере и написания несложных сценариев. Профессионалам, использующим PowerShell для решения задач системного администрирования и управления серверами, могут пригодиться следующие мощные возможности, оставшиеся за рамками данной книги.

- ❑ Подключение и работа с оболочкой PowerShell на удаленных компьютерах с помощью службы удаленного управления Windows (WinRM) или по протоколу SSH.
- ❑ Удаленное управление одним или несколькими компьютерами с помощью объектов .NET, CIM, WMI.
- ❑ Использование рабочих процессов (workflows) PowerShell, с помощью которых можно параллельно или последовательно запускать несколько заданий.
- ❑ Асинхронное выполнение команд с помощью фоновых заданий (background jobs).
- ❑ Обработка синхронных и асинхронных событий от различных источников в операционной системе.
- ❑ Настройка среды выполнения для приложений с помощью технологии Desired State Configuration (DSC), реализующей подход "инфраструктура как код".

Если вы часто пишете на PowerShell сценарии, которые становятся все сложнее и объемнее, то для их создания и поддержки лучше придерживаться испытанных подходов из традиционной разработки программного обеспечения.

- ❑ Обязательно следует применять систему контроля версий Git, с которой вы никогда не потеряете свой код и всегда сможете вернуться к любой сохраненной версии сценариев (способ интеграции Git с командной строкой PowerShell описан в *приложении 3*).
- ❑ Очень полезно будет разобраться с тем, как в PowerShell можно создавать и применять тесты, позволяющие удостовериться в том, что после внесения изменений код продолжает корректно работать. Подобно другим языкам программирования, для PowerShell существуют специальные фреймворки (frameworks), облегчающие работу с тестами, например Pester (<https://github.com/pester/Pester>).
- ❑ Связанные друг с другом функции и сценарии можно организовывать в виде модулей (напомним, что работу с готовыми модулями мы рассматривали в *главе 9*).



ПРИЛОЖЕНИЯ

Приложение 1. Что значат эти символы

Приложение 2. PowerShell в macOS и Linux

Приложение 3. Дополнительная настройка командной строки

ПРИЛОЖЕНИЕ 1

Что значат эти символы

В языке PowerShell используются различные специальные символы, которые в зависимости от контента могут иметь разный смысл и иногда применяются не так, как в других языках программирования.

- ❑ Знак ``` (обратный апостроф или *гравис*) является в PowerShell экранирующим символом. Внутри строки в двойных кавычках обратный апостроф задает escape-последовательность (например, ``n` означает перевод строки). Использование его перед пробелом внутри параметров команды позволяет не указывать кавычки для путей к файлам или каталогам, содержащим пробелы (например, `cd C:\Program` Files`). Если обратный апостроф является последним символом в командной строке, то ввод команды продолжается со следующей строки (это позволяет выполнять в PowerShell многострочные команды).
- ❑ Знак `~` (тильда) используется в путях к объектам файловой системы, он соответствует пути к домашнему каталогу активного пользователя. Например, по команде `cd ~` мы перейдем в этот каталог (в системе данный путь хранится в переменной среды `USERPROFILE`).
- ❑ Круглые скобки `()` в PowerShell используются двумя способами.
 - Скобки задают порядок вычисления выражений или выполнения команд. Например, в команде `Get-Process -ComputerName (Get-Content .\comps.txt)` сначала выполняется команда `Get-Content .\comps.txt`, а ее результат (содержимое файла `comps.txt`) передается с качестве значения параметра `-ComputerName`.
 - Скобки, указанные после имени метода, приводят к его выполнению. Например: `"Привет".Trim()`. Скобки после имени метода нужно писать обязательно, даже если в метод не передается ни один параметр.
- ❑ Квадратные скобки `[]` используются двумя способами.
 - Квадратные скобки позволяют получить элемент массива по его индексу. Например, `$services[3]` означает четвертый элемент массива `$services` (первый элемент всегда имеет индекс 0).

- Внутри квадратных скобок можно указать тип, к которому требуется привести переменную или результат вычисления выражения. Например, значением выражения `[string]1.1 + [string]0.9` будет строка `'1.10.9'`.
- ❑ Фигурные скобки `{ }` могут применяться в нескольких вариантах.
 - В фигурных скобках указывается блок исполняемого кода (аналог анонимных функций в других языках программирования). Например: `Get-Service | Where-Object { $_.Status -eq "Stopped" }`.
 - При создании хэш-таблиц внутри фигурных скобок, перед которыми стоит знак `@`, перечисляются пары "ключ=значение". Например: `$user = @(name="James"; lastName="Bond"; age=30)`.
 - С помощью фигурных скобок можно давать переменным имена, содержащие пробелы или другие недопустимые для имен символы. Например: `$(My Var)`. Интересный эффект получается, если в фигурных скобках указана переменная из пространства имен файловой системы, т. е. путь к файлу с указанием имени диска. В этом случае значением переменной будет содержимое этого файла, например `$content = $(C:\Script\test.txt)`. Если подобную переменную указать в левой части оператора присваивания, то содержимое файла будет изменено на новое значение, например `$(C:\Script\test.txt) = "Новое содержимое файла"`.
- ❑ Одинарными кавычками `' '` выделяют нерасширяемые строки. В таких строках не действуют `escape`-последовательности и не происходит подстановка значений переменных.
- ❑ Двойные кавычки `" "` используются для создания расширяемых строк. Если внутри такой строки встречается имя переменной или другое выражение, которое может быть вычислено, то в строку подставляется значение данной переменной или результат вычисления выражения. Также в расширяемых строках действуют `escape`-последовательности. Например, если в переменной `$name` хранится слово "Иван", то в переменной `$hello = "Привет, $name!\n"` будет записана строка "Привет, Иван!" и символ перевода строки.
- ❑ Знак доллара `$` в PowerShell ставится перед именами переменных.
- ❑ Знак процента `%` используется в качестве псевдонима командлета `ForEach-Object`. В арифметических выражениях `%` означает оператор целочисленного деления.
- ❑ Вопросительный знак `?` используется в качестве псевдонима командлета `Where-Object`.
- ❑ Знак больше `>` фактически является псевдонимом командлета `Out-File`. С помощью этого символа можно перенаправить вывод какого-либо командлета во внешний файл, и выглядит это как перенаправление вывода в оболочках `cmd.exe` или `bash`.
- ❑ Знак `+` используется для обозначения операции сложения чисел и конкатенации строк.

- ❑ Дефис – ставится перед именами параметров команд (например, `-id`) и перед многими операторами (например, `-eq`). Также дефис используется для разделения глагола и существительного в именах командлетов (например, `Get-Process`).
- ❑ Для знака коммерческого ат (знака "собачки") `@` возможны четыре варианта использования.
 - Ставится перед фигурными скобками при создании хэш-таблицы. Например:
`$server = @{comp = "Server1"; ip = "10.170.1.1"}.`
 - Записывается перед круглыми скобками, внутри которых через запятую перечисляется список значений, формирующих массив. Например: `$numbers = @(1, 2, 45, 3).`
 - Определяет строку типа `here-string` вида `@"блок текста"@` или `@'блок текста'@`. Подобные строки обычно используются для вставки в сценарий больших блоков текста или при генерации текстовой информации для других программ.
 - Обозначает `splat`-оператор, с помощью которого можно передавать в команду параметры, упакованные в массив, элементы которого будут соответствовать позиционным параметрам, или хэш-таблицу, ключи в которой соответствуют именам параметров. Например: `$params = @{Path = "test.txt"; Destination = "test2.txt"; WhatIf = $true}; Copy-Item @params.`
- ❑ Знак амперсанда `&` соответствует оператору вызова и сообщает оболочке о том, что следующая за этим знаком строка является командной, которую нужно выполнить. Например: `$command = "dir"; & $command.`
- ❑ Точка с запятой `;` является разделителем, который позволяет записать и выполнить несколько команд в одной строке. Например: `dir ; Get-Process`. При такой записи команды являются независимыми, результаты выполнения одной команды следующей команде не передаются.
- ❑ Знак решетки `#` используется в комментариях. Оболочка игнорирует все символы, которые идут после этого знака до конца строки. Блочные комментарии, состоящие из нескольких строк, помещаются внутри символов `<# ... #>`.
- ❑ Знак равенства `=` обозначает оператор присваивания значения переменной: `$a = 1`. В операторе сравнения знак равенства не используется, вместо него нужно писать оператор `-eq`. Также знак равенства может применяться совместно с математическими операторами: `$a += 10` добавит число 10 к текущему значению переменной `$a`.
- ❑ Вертикальная черта `|` используется для организации конвейера, когда выходные данные одного командлета (т. е. объекты, которые этот командлет помещает в выходной поток) передаются на вход другому командлету.
- ❑ Символ косой черты (слэш) `/` в математических выражениях обозначает операцию деления. Слэш и обратный слэш `\` также используются в качестве разделителя в путях к объектам файловой системы. Например: `C:\Windows` или `C:/Windows`.

- ❑ Точка `.` может использоваться в нескольких вариантах.
 - Точка указывается после имени объекта для получения доступа к его свойству или методу. Например, с помощью `$_ .Name` мы обращаемся к свойству `Name` объекта, которому соответствует переменная `$_`.
 - Точка и пробел перед именем скрипта или перед путем к нему приводят к выполнению этого сценария в режиме `dot-sourcing`. Скрипт запускается в области видимости окна командной строки, все функции и переменные из него приобретают глобальную область видимости и остаются доступными в оболочке после завершения работы сценария.
 - Две точки соответствуют оператору диапазона. Например, выражение `1 .. 10` генерирует числа от 1 до 10.
 - Две точки в файловых путях соответствуют родительскому каталогу. Например, если текущим каталогом является `C:\Script\PowerShell`, то путь `..\..` будет соответствовать корневому каталогу на диске `C:\`.
- ❑ Запятая `,` является разделителем элементов в массиве или списке. Например: `$arr = "Иванов", "Петров", "Сидоров", 23`. Также запятая может разделять несколько значений параметров командлета или функции, например: `Get-Services -computername Server1,Server2,Server3`.
- ❑ Двойное двоеточие `::` используется для доступа к статическим членам класса .NET Framework. Например: `[math]::Sqrt(25)`.
- ❑ Восклицательный знак `!` является кратким псевдонимом оператора логического отрицания `-not`.

ПРИЛОЖЕНИЕ 2

PowerShell в macOS и Linux

Начиная с 2017 г. PowerShell является проектом Open Source, исходный код размещен в репозитории на GitHub (<https://github.com/PowerShell/PowerShell>). Оболочка PowerShell версий 6 и 7, которая может работать не только в Windows, но и в macOS и в различных дистрибутивах Linux (Debian, Ubuntu, CentOS, Red Hat, openSUSE, Fedora), базируется на платформе .NET Core/.NET 5 — кросс-платформенной реализации Windows .NET Framework.

ЗАМЕЧАНИЕ

Вначале кросс-платформенная версия оболочки называлась PowerShell Core 6, чтобы явно указать ее зависимость от .NET Core. Начиная с седьмой версии, слово Core из названия убрано, теперь оболочка называется просто PowerShell. Версия оболочки, базирующаяся на полной платформе Windows .NET Framework, называется Windows PowerShell.

Установка и запуск оболочки

Процедура установки PowerShell зависит от типа операционной системы и ее версии. Здесь мы не будем подробно на этом останавливаться, соответствующие инструкции для всех поддерживаемых систем имеются на сайте Microsoft по адресу <https://docs.microsoft.com/ru-ru/powershell/scripting/install/installing-powershell>.

В дальнейшем в примерах будет рассматриваться PowerShell в операционной системе Linux Ubuntu 20.04, которая была установлена в подсистеме WSL на компьютере автора. В этой системе PowerShell ставится следующими командами:

```
sudo apt-get update
sudo apt-get install -y wget apt-transport-https software-properties-common
wget -q https://packages.microsoft.com/config/ubuntu/20.04/packages-microsoft-prod.deb
sudo dpkg -i packages-microsoft-prod.deb
sudo apt-get update
sudo add-apt-repository universe
sudo apt-get install -y powershell
```

Для запуска установленной оболочки PowerShell нужно выполнить в терминале команду `pwsh`. После вывода информации о версии PowerShell мы получим обычное приглашение командной строки с путем к текущему каталогу в качестве префикса.

```
andrey@DESKTOP-BU86I5T:~$ pwsh
PowerShell 7.1.4
Copyright (c) Microsoft Corporation.
```

```
https://aka.ms/powershell
Type 'help' to get help.
```

```
PS /home/andrey>
```

Информация об используемой платформе .NET и версии PowerShell хранится в переменной `$PSVersionTable`. Посмотрим на значение этой переменной в оболочке, запущенной под Linux:

```
PS /home/andrey> $PSVersionTable
```

Name	Value
PSVersion	7.1.4
PSEdition	Core
GitCommitId	7.1.4
OS	Linux 4.19.128-microsoft-standard...
Platform	Unix
PSCompatibleVersions	{1.0, 2.0, 3.0, 4.0...}
PSRemotingProtocolVersion	2.3
SerializationVersion	1.1.0.1
WSManStackVersion	3.0

В поле `PSVersion` указана версия оболочки, в поле `PSEdition` — ее тип, в поле `OS` — версия используемой операционной системы. В нашем случае мы используем PowerShell версии 7.1.4 Core в Linux.

В Windows PowerShell состав и значения полей переменной `$PSVersionTable` будут другими, здесь мы запускаем десктопную версию оболочки 5.1, использующую .NET Framework 4 (поле `CLRVersion`):

```
PS C:\Users\andrv> $PSVersionTable
```

Name	Value
PSVersion	5.1.19041.1151
PSEdition	Desktop
PSCompatibleVersions	{1.0, 2.0, 3.0, 4.0...}
BuildVersion	10.0.19041.1151
CLRVersion	4.0.30319.42000
WSManStackVersion	3.0
PSRemotingProtocolVersion	2.3
SerializationVersion	1.1.0.1

Отличия от Windows PowerShell

В .NET Core/.NET 5 реализована не вся функциональность платформы .NET Framework, поэтому в PowerShell 6/7 по умолчанию загружается меньше модулей и, следовательно, будет доступно меньше командлетов, чем в Windows PowerShell. Список доступных модулей можно посмотреть с помощью команды `Get-Module`:

```
PS /home/andrey> Get-Module -ListAvailable | Select Name
```

```
Name
----
Microsoft.PowerShell.Archive
Microsoft.PowerShell.Host
Microsoft.PowerShell.Management
Microsoft.PowerShell.Security
Microsoft.PowerShell.Utility
PackageManagement
PowerShellGet
PSDesiredStateConfiguration
PSReadLine
ThreadJob
```

Как видим, в PowerShell для Linux загружаются десять модулей, в то время как в Windows PowerShell по умолчанию доступны несколько десятков модулей. Часть из отсутствующих модулей напрямую связана с особенностями операционной системы Windows и не может быть перенесена на другие платформы, некоторые модули могут быть добавлены в следующих версиях PowerShell.

Таким образом, в PowerShell 6/7 отсутствуют определенные командлеты, имевшиеся в Windows PowerShell, в том числе `*-Service`, `*-Acl`, `*-Event`, `Set-ExecutionPolicy`. Некоторые из отсутствующих командлетов должны появиться в следующих версиях PowerShell.

Вспомним, какие провайдеры доступны в Windows PowerShell и какие виртуальные диски они поддерживают. Для этого выполним командлет `Get-PSDrive`:

```
PS C:\Script> Get-PSDrive | Select Name, Provider, Root
```

Name	Provider	Root
----	-----	----
Alias	Microsoft.PowerShell.Core\Alias	
C	Microsoft.PowerShell.Core\FileSystem	C:\
Cert	Microsoft.PowerShell.Security\Certificate \	
Env	Microsoft.PowerShell.Core\Environment	
Function	Microsoft.PowerShell.Core\Function	
HKCU	Microsoft.PowerShell.Core\Registry	HKEY_CURRENT_USER
HKLM	Microsoft.PowerShell.Core\Registry	HKEY_LOCAL_MACHINE
Variable	Microsoft.PowerShell.Core\Variable	
WSMan	Microsoft.WSMan.Management\WSMan	

В PowerShell для Linux провайдеры и виртуальные диски для системного реестра и хранилища сертификатов отсутствуют:

```
PS /mnt/c/Users/andrv> Get-PSDrive | Select Name, Provider, Root
```

Name	Provider	Root
----	-----	----
/	Microsoft.PowerShell.Core\FileSystem	/
Alias	Microsoft.PowerShell.Core\Alias	
Env	Microsoft.PowerShell.Core\Environment	
Function	Microsoft.PowerShell.Core\Function	
Temp	Microsoft.PowerShell.Core\FileSystem	/tmp/
Variable	Microsoft.PowerShell.Core\Variable	

При работе с PowerShell в Linux и macOS необходимо учитывать особенности самих операционных систем.

- ❑ *Зависимость от регистра символов.* Сама оболочка PowerShell, как и операционная система Windows, не зависит от регистра символов. Например, имена \$a и \$A обозначают одну и ту же переменную, название командлетов можно писать в любом регистре (Get-Process, get-process или GET-PROCESS), в путях к файлам и каталогам в Windows тоже можно использовать символы в разных регистрах (C:\Windows, c:\windows или C:\WINDOWS). Напротив, в операционных системах macOS и Linux имеется зависимость от регистра символов в путях к каталогам и файлам или в именах переменных среды. Поэтому для запуска сценариев, загрузки модулей и корректной работы автодополнения файловых путей в этих системах нужно писать точные имена с учетом регистра символов. Названия самих командлетов и переменных PowerShell по-прежнему от регистра не зависят.
- ❑ *Разделители в путях файловой системы.* В Windows в качестве разделителей в путях к файлам и каталогам можно использовать символы слеша / и обратного слеша \. В macOS и Linux используется только символ слеша /.
- ❑ *Псевдонимы команд.* В Windows PowerShell за некоторыми командлетами закреплены псевдонимы, соответствующие стандартным командам UNIX-подобных систем: ls, rm, cp, mv, cat, ps, man, mount. В PowerShell 6/7 данные псевдонимы не определены, их ввод приводит к выполнению "родных" команд операционной системы.
- ❑ *Запуск команд от имени суперпользователя (sudo).* Для выполнения какой-либо одной команды с привилегиями суперпользователя root в оболочке macOS или Linux используется команда sudo. В сеансе работы PowerShell напрямую применить sudo не получится, при необходимости нужно с помощью sudo запустить новый экземпляр оболочки, и все команды PowerShell будут выполняться от имени суперпользователя.

Отметим также, что интегрированная среда разработки PowerShell ISE доступна только в Windows PowerShell, в состав PowerShell 6/7 она не входит. Для разработки скриптов в macOS или Linux компания Microsoft рекомендует использовать редактор Visual Studio Code с расширением для поддержки языка PowerShell.

ПРИЛОЖЕНИЕ 3

Дополнительная настройка командной строки

Работа в командной строке должна приносить удовольствие. Будет очень приятно, если в приглашении терминала отображается полезная для вас информация, оболочка сама подсказывает нужные команды, а постоянно повторяемые действия можно выполнять несколькими нажатиями клавиш. Такие возможности предоставляют дополнительные модули для PowerShell, некоторые из них мы рассмотрим далее.

Модуль PSReadLine

В *главе 4* мы говорили о стандартных способах редактирования командной строки в консоли PowerShell. Модуль PSReadLine добавляет к ним новые возможности, делающие работу с оболочкой PowerShell более удобной. Перечислим некоторые из них.

- ❑ *Цветовое выделение синтаксиса при наборе команд.* Названия командлетов, параметров и переменных, ключевые конструкции и другие элементы языка PowerShell при вводе в командную строку отображаются разным цветом. Это помогает лучше понимать структуру команд и снижает вероятность ошибки при наборе.
- ❑ *Отмена (Undo) и повторное применение (Redo) изменений в командной строке.* По умолчанию для отмены изменений нужно нажать комбинацию клавиш <Ctrl>+<z>, а для повтора — комбинацию <Ctrl>+<y>.
- ❑ *Автоматическая проверка команды при вводе.* Если при наборе команды получается синтаксически неверная конструкция (например, мы забыли поставить закрывающую скобку или не написали команду после знака конвейера), то последний знак > в приглашении командной строки становится красным.
- ❑ *Многострочное редактирование команд.* Многословные команды можно располагать на нескольких строках даже без ввода знака ` разрыва строки. При этом можно свободно перемещаться по строкам команды, изменять их, добавлять новые строки выше и ниже текущей строки. Новая строка после текущей вставляется при нажатии комбинации <Shift>+<Enter>, а перед текущей строкой — при нажатии <Ctrl>+<Enter>.

- ❑ *Поиск в истории вводившихся команд.* Если вы помните только часть команды (не обязательно сначала), то по ней можно найти подходящие варианты среди тех команд, которые выполнялись в оболочке ранее. По умолчанию интерактивный поиск в таком режиме активируется нажатием клавиш <Ctrl>+<r> (поиск по истории команд назад) или <Ctrl>+<s> (поиск по истории команд вперед).
- ❑ *Вывод вариантов автодополнения в виде меню.* Для завершения ввода команды или параметра часто бывает удобно посмотреть на полный список всех возможных вариантов и выбрать в нем нужный пункт. Такое меню с вариантами автодополнения открывается при нажатии комбинации клавиш <Ctrl>+<Пробел>. Например, если после имени команды через пробел набрать знак дефиса и нажать <Ctrl>+<Пробел>, то будет выведен список всех параметров, поддерживаемых этой командой. По предложенным вариантам можно переходить с помощью клавиш со стрелками, выбор варианта — клавиша <Enter>. Для текущего выбранного свойства на экране отображается его тип.
- ❑ *Поддержка технологии автодополнения IntelliSense.* При наборе команды оболочка может сразу выдавать подходящий завершённый вариант из списка вводившихся ранее команд, который можно принять одним нажатием клавиши <→>. Активируется этот режим следующей командой: `Set-PSReadLineOption -PredictionSource 'History'`.

В Windows PowerShell 5.1 для Windows 10 и в кросс-платформенных версиях PowerShell модуль PSReadLine должен быть установлен по умолчанию. Увидеть список подключенных модулей и проверить наличие в нем PSReadLine можно с помощью командлета `Get-Module`:

```
PS C:\Users\andr> Get-Module | Select Name, Version
```

Name	Version
----	-----
Microsoft.PowerShell.Management	3.1.0.0
Microsoft.PowerShell.Security	3.0.0.0
Microsoft.PowerShell.Utility	3.1.0.0
Microsoft.WSMan.Management	3.0.0.0
PSReadline	2.1.0

Подробную информацию о модуле PSReadLine можно посмотреть следующим образом:

```
PS C:\Users\andr> Get-Module PSReadline | Format-List
```

Name	: PSReadline
Path	: C:\Program Files\WindowsPowerShell\Modules\ PSReadline\2.1.0\PSReadLine.psml
Description	: Great command line editing in the PowerShell console host
ModuleType	: Script
Version	: 2.1.0
NestedModules	: {Microsoft.PowerShell.PSReadLine2}

```
ExportedFunctions : PSConsoleHostReadLine
ExportedCmdlets   : {Get-PSReadLineKeyHandler, Get-PSReadLineOp
                    tion, Remove-PSReadLineKeyHandler, Set-PSRe
                    adLineKeyHandler...}
ExportedVariables :
ExportedAliases   :
```

Если модуль PSReadLine не обнаружен, то его нужно установить с помощью команды:

```
Install-Module PSReadLine -Force -Scope CurrentUser -SkipPublisherCheck.
```

Текущие сочетания клавиш, назначенные в PSReadLine для выполнения различных действий, можно посмотреть с помощью команды Get-PSReadlineKeyHandler:

```
PS C:\Users\andr> Get-PSReadlineKeyHandler
```

```
Basic editing functions
=====
```

Key	Function	Description
-----	-----	-----
Enter	AcceptLine	Accept the input or move to the next line if input is missing a closing token.
Shift+Enter	AddLine	Move the cursor to the next line without attempting to execute the input
Backspace	BackwardDeleteChar	Delete the character before the cursor
Ctrl+h	BackwardDeleteChar	Delete the character before the cursor
Ctrl+Home	BackwardDeleteLine	Delete text from the cursor to the start of the line
Ctrl+Backspace	BackwardKillWord	Move the text from the start of the current or previous word to the cursor to the kill ring
Ctrl+w	BackwardKillWord	Move the text from the start of the current or previous word to the cursor to the kill ring
Ctrl+C	Copy	Copy selected region to the system clipboard. If no region is selected, copy the whole line
...		

Каждая операция, поддерживаемая PSReadLine, здесь отнесена к одной из категорий:

- ❑ Basic editing functions (Основные функции редактирования);
- ❑ Cursor movement functions (Функции для перемещения курсора);
- ❑ History functions (Функции для работы с историей команд);

- ☐ Completion functions (Функции автодополнения при вводе);
- ☐ Miscellaneous functions (Различные функции).

Изменить сочетания клавиш позволяет команда `Set-PSReadlineKeyHandler`. Например, назначим клавишам `<↑>` и `<↓>` функции `HistorySearchBackward` (поиск назад по истории команд) и `HistorySearchForward` (поиск вперед по истории команд) соответственно:

```
PS C:\Users\andrv> Set-PSReadLineKeyHandler -Key UpArrow -Function
HistorySearchBackward
PS C:\Users\andrv> Set-PSReadLineKeyHandler -Key DownArrow -Function
HistorySearchForward
```

Теперь мы сможем подставлять в командную строку команды из истории, набрав в командной строке их начальные символы и нажимая клавишу `<↑>` или клавишу `<↓>`.

Чтобы не набирать при каждом запуске PowerShell команды для настройки `PSReadLine`, их нужно поместить в свой PowerShell-профиль, путь к которому хранится в переменной `$PROFILE`. В этом случае команды из профиля будут запускаться при каждом входе в оболочку. Открыть профиль для редактирования в Блокноте можно так:

```
C:\Users\andrv> notepad %PROFILE
```

Отметим также, что по умолчанию `PSReadLine` сохраняет все команды, которые выполнялись в оболочке PowerShell, в текстовом файле на диске. Посмотреть на содержимое этого файла в Блокноте можно следующей командой:

```
C:\Users\andrv> notepad (Get-PSReadLineOption).HistorySavePath
```

Интеграция с Git. Модуль posh-git

Система управления версиями Git является фактически современным стандартом для хранения и обмена кодом, ею пользуются практически все разработчики.

Для интеграции Git и PowerShell служит модуль `posh-git` (<https://github.com/dahlbyk/posh-git>). После установки этого модуля в приглашении командной строки будет отображаться информация о состоянии текущего Git-репозитория, а команды и параметры Git можно будет автоматически дополнять при вводе по нажатию клавиши `<Tab>`.

Перед тем как устанавливать `posh-git`, убедитесь, что система Git установлена и доступна в оболочке PowerShell. Для этого в консоли PowerShell нужно выполнить команду `git --version`, которая должна вывести информацию о версии Git:

```
C:\Users\andrv> git --version
git version 2.28.0.windows.1
```

Устанавливается `posh-git` стандартным образом с помощью командлета `Install-Module`:

```
C:\Users\andrv> Install-Module posh-git -Scope CurrentUser -Force
```


После установки нужно импортировать все команды из posh-git:

```
C:\Users\andrv> Import-Module posh-git
```

Команду для импорта нужно поместить в свой профиль (сценарий, путь которому хранится в переменной \$PROFILE) для автоматического выполнения при каждом запуске оболочки. Для этого можно открыть сценарий профиля в Блокноте (команда notepad \$PROFILE) и добавить туда нужную команду или просто выполнить команду Add-PoshGitToProfile из модуля posh-git.

Чтобы увидеть изменения в командной строке, нужно войти в каталог, являющийся Git-репозиторием. Создадим новый каталог git-test и создадим в нем новый репозиторий Git:

```
C:\Users\andrv> mkdir git-test
```

```
Каталог: C:\Users\andrv
```

Mode	LastWriteTime	Length	Name
----	-----	-----	----
d----	03.10.2021 10:04		git-test

```
C:\Users\andrv> cd git-test
```

```
C:\Users\andrv\git-test> git init
```

```
C:\Users\andrv\git-test [master]>
```

В приглашении командной строки после пути к текущему каталогу в квадратных скобках теперь выводится информация о состоянии Git-репозитория.

Пока мы видим только имя master текущей ветки, т. к. файлов в рабочем каталоге нет:

```
C:\Users\andrv\git-test [master]> git status
```

```
On branch master
```

```
No commits yet
```

```
nothing to commit (create/copy files and use "git add" to track)
```

Создадим три новых файла:

```
C:\Users\andrv\git-test [master]> New-Item file1.txt, file2.txt,
                                     file3.txt -ItemType File
```

```
Каталог: C:\Users\andrv\git-test
```

Mode	LastWriteTime	Length	Name
----	-----	-----	----
-a----	03.10.2021 10:28	0	file1.txt
-a----	03.10.2021 10:28	0	file2.txt
-a----	03.10.2021 10:28	0	file3.txt

```
C:\Users\andrv\git-test [master +3 ~0 -0 !]>
```

В подсказке о состоянии репозитория появились новые данные: восклицательный знак указывает на то, что в рабочем каталоге есть файлы, которые не добавлены в индекс Git, а число после знака + показывает количество таких файлов.

Добавим файл `file1.txt` в индекс Git:

```
C:\Users\andrv\git-test [master +3 ~0 -0 !]> git add file1.txt
C:\Users\andrv\git-test [master +1 ~0 -0 | +2 ~0 -0 !]>
```

Теперь в подсказке указана информация о файлах в индексе Git (слева от разделителя `|`) и о файлах в рабочем каталоге (справа от разделителя `|`). В нашем случае в индексе находится один новый файл (число после знака +), а в рабочем каталоге остались два новых файла.

Изменим файл `file1.txt`, добавив в него строку:

```
C:\Users\andrv\git-test [master +1 ~0 -0 | +2 ~0 -0 !]> "Hello!" >> file1.txt
C:\Users\andrv\git-test [master +1 ~0 -0 | +2 ~1 -0 !]>
```

Как видим, в секции рабочего каталога после знака `~` теперь указана единица — это количество измененных файлов в рабочем каталоге.

Добавим в индекс Git файл `file2.txt`:

```
C:\Users\andrv\git-test [master +1 ~0 -0 | +2 ~1 -0 !]> git add file2.txt
C:\Users\andrv\git-test [master +2 ~0 -0 | +1 ~1 -0 !]>
```

Теперь в индексе два новых файла (+2), в рабочем каталоге один новый файл (+1) и один измененный файл (~1).

Удалим файл `file2.txt` в рабочем каталоге:

```
C:\Users\andrv\git-test [master +2 ~0 -0 | +1 ~1 -0 !]> Remove-Item file2.txt
C:\Users\andrv\git-test [master +2 ~0 -0 | +1 ~1 -1 !]>
```

Информация о файлах в индексе не изменилась (+2 ~0 -0), а в рабочем каталоге теперь один новый файл (+1), один измененный файл (~1) и один удаленный файл (-1).

Создадим *коммит* (зафиксируем изменения в репозитории), в который войдут файлы `file1.txt` и `file2.txt` из индекса:

```
C:\Users\andrv\git-test [master +2 ~0 -0 | +1 ~1 -1 !]> git commit -m "First
commit"
[master (root-commit) b22296c] First commit
 2 files changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 file1.txt
 create mode 100644 file2.txt
C:\Users\andrv\git-test [master +1 ~1 -1 !]>
```

После коммита индекс Git очистился и в подсказке о состоянии репозитория осталась только информация о файлах в рабочем каталоге.

Добавим в индекс все файлы из рабочего каталога:

```
C:\Users\andrv\git-test [master +1 ~1 -1 !]> git add .
C:\Users\andrv\git-test [master +1 ~1 -1 ~]>
```

Индикатор состояния репозитория изменился со знака ! на знак ~, означающий, что в репозитории есть *незакоммиченные изменения*:

```
C:\Users\andrv\git-test [master +1 ~1 -1 ~]> git status
```

```
On branch master
```

```
Changes to be committed:
```

```
(use "git restore --staged <file>..." to unstage)
```

```
    modified:   file1.txt
```

```
    deleted:    file2.txt
```

```
    new file:   file3.txt
```

```
C:\Users\andrv\git-test [master +1 ~1 -1 ~]>
```

Таким образом, подсказка в приглашении командной строки помогает быстро понять, в каком состоянии находится Git-репозиторий. Для более наглядного представления этой информации можно изменить тему оформления консоли с помощью модуля Oh My Posh.

Оформление приглашения командной строки

Декорировать консоль PowerShell можно с помощью различных настраиваемых тем оформления, предлагаемых модулем Oh My Posh. Начиная с третьей версии, модуль Oh My Posh является кросс-платформенным, его можно применять как в операционной системе Windows, так и в macOS и Linux.

В темах Oh My Posh используются различные пиктограммы, которые отображаются с помощью шрифтов Powerline с глифами. Поэтому данные шрифты нужно предварительно установить в операционную систему и подключить в терминале.

Установка шрифтов Powerline

В темах Oh My Posh рекомендуется использовать шрифты NerdFonts, которые можно скачать, например, с сайта GitHub:

```
C:\Users\andrv> Invoke-WebRequest -Uri 'https://github.com/ryanoasis/nerd-fonts/releases/download/v2.1.0/CascadiaCode.zip' -OutFile .\nf_fonts.zip
```

С помощью команды `Expand-Archive` распакуем загруженный архив в каталог NerdFonts:

```
C:\Users\andrv> Expand-Archive .\nf_fonts.zip -DestinationPath .\NerdFonts
```

Для установки шрифтов в систему откроем в Проводнике Windows папку NerdFonts с загруженными шрифтами, выделим в ней все ttf-файлы, щелкнем правой кнопкой мыши и в контекстном меню выберем пункт **Установить для всех пользователей**.

После этого загруженный архив и каталог с его содержимым можно удалить:

```
C:\Users\andrv> Remove-Item .\nf_fonts.zip
```

```
C:\Users\andrv> Remove-Item .\NerdFonts -Recurse
```

После установки загруженных шрифтов нужно назначить шрифт CaskaydiaCove NF в качестве шрифта для профиля Windows PowerShell в терминале Windows. Это можно сделать на вкладке **Appearance** в настройках профиля Windows PowerShell (рис. ПЗ.1).

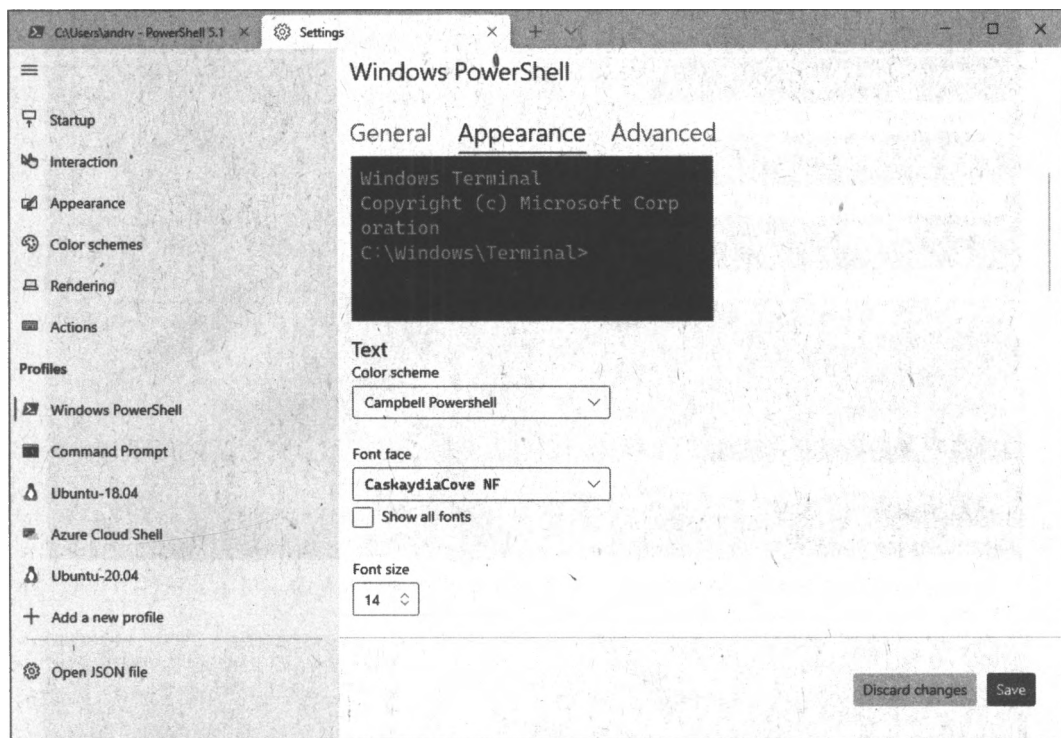


Рис. ПЗ.1. Настройка шрифта для профиля PowerShell в терминале Windows

Модуль Oh My Posh

Модуль Oh My Posh устанавливается стандартно:

```
C:\Users\andr> Install-Module oh-my-posh -Scope CurrentUser -Force
```

Модуль довольно объемный, установка его может занять продолжительное время.

После установки с помощью команды `Get-PoshThemes` можно посмотреть все стандартные встроенные темы оформления, которые поддерживает Oh My Posh (рис. ПЗ.2).

Установить понравившуюся тему можно с помощью команды `Set-PoshPrompt`, указав название темы в качестве значения параметра `-Theme`. Например, выберем тему `agnosticer` и перейдем в каталог `git-test`, являющийся репозиторием Git (рис. ПЗ.3).

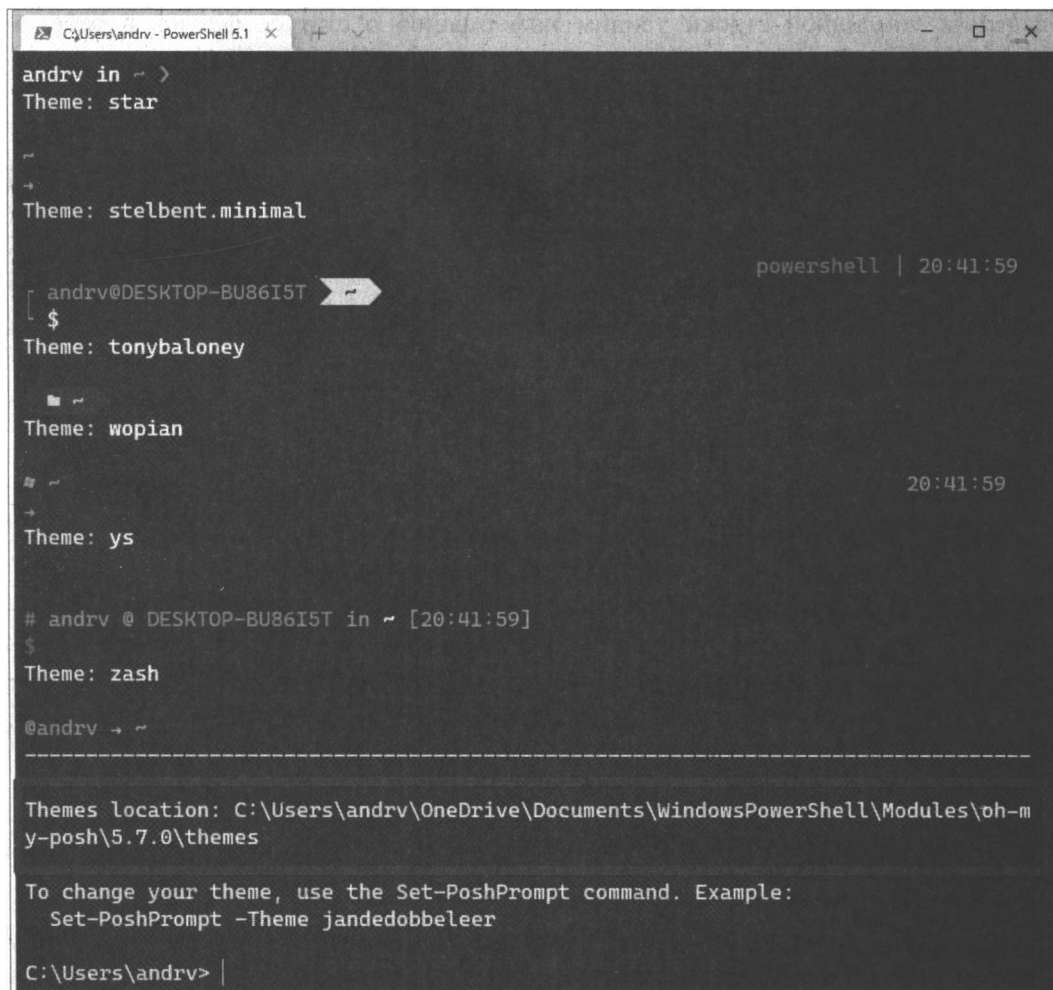
Теперь в приглашении командной строки отображаются имена пользователя и компьютера, путь к текущей директории от домашнего каталога и имя текущей ветки в Git-репозитории.

Для автоматического применения выбранной схемы при загрузке PowerShell нужно команду `Set-PoshPrompt -Theme agnoster` добавить в профиль PowerShell (напомним, что путь к профилю хранится в переменной `$PROFILE`).

По умолчанию в теме `agnoster` отключен вывод подробной информации о состоянии Git-репозитория, которую предоставляет модуль `posh-git`. Чтобы включить эту информацию, нужно в конфигурационный JSON-файл с настройками темы внести изменения, описанные в документации модуля Oh My Posh (<https://ohmyposh.dev/docs/git>).

Путь к каталогу с темами можно посмотреть в конце вывода команды `Get-PoshThemes` (рис. ПЗ.4).

Выполнив команду `dir` для этого каталога, мы по названию схемы узнаем имя соответствующего конфигурационного файла (для нашей схемы `agnoster` это файл



```
C:\Users\andr\ - PowerShell 5.1
andr in ~ >
Theme: star

Theme: stelbent.minimal

powershell | 20:41:59

[ andrv@DESKTOP-BU86I5T ~ ]
$
Theme: tonybaloney

Theme: wopian

20:41:59

Theme: ys

# andrv @ DESKTOP-BU86I5T in ~ [20:41:59]
$
Theme: zash

@andr ~
-----

Themes location: C:\Users\andr\OneDrive\Documents\WindowsPowerShell\Modules\oh-my-posh\5.7.0\themes

To change your theme, use the Set-PoshPrompt command. Example:
Set-PoshPrompt -Theme jandedobbeleer

C:\Users\andr> |
```

Рис. ПЗ.4. Путь к каталогу с темами Oh My Posh

agnoster.omp.json). Данный файл нужно открыть в текстовом редакторе и добавить в раздел, связанный с Git, параметр "properties": { "display_status": true }:

```
{
  "type": "git",
  "style": "powerline",
  "powerline_symbol": "\uE0B0",
  "foreground": "#193549",
  "background": "#95ffa4",
  "properties": {
    "display_status": true
  }
}
```

После сохранения конфигурационного файла и перезапуска оболочки мы в приглашении командной строки увидим информацию о статусе файлов в рабочем каталоге и индексе Git (рис. ПЗ.5).

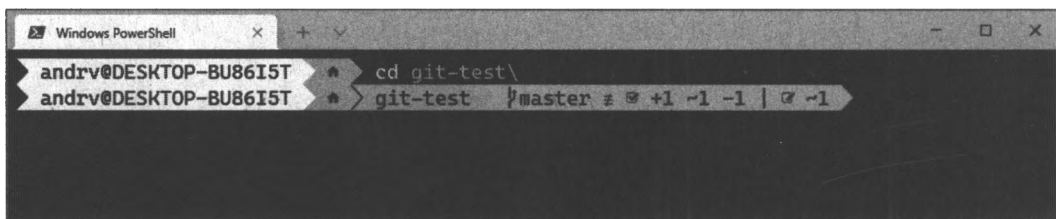


Рис. ПЗ.5. Информация о состоянии файлов в Git-репозитории для темы agnoster

Оформление списков файлов и каталогов. Модуль Terminal-Icons

Работая в командной строке, мы часто с помощью команды `Get-ChildItem` (псевдонимы `dir` и `ls`) просматриваем списки файлов и каталогов. Модуль `Terminal-Icons` делает эти списки более наглядными, выделяя различные типы файлов разным цветом и добавляя к названиям файлов соответствующие иконки.

Установим и импортируем модуль `Terminal-Icons`:

```
C:\Users\andrv> Install-Module Terminal-Icons -Scope CurrentUser -Force
C:\Users\andrv> Import-Module Terminal-Icons
```

Теперь список файлов и каталогов выглядит намного интереснее (рис. ПЗ.6).

Если вам понравился такой формат отображения файлов и каталогов, то команду импортирования модуля `Terminal-Icons` нужно добавить в сценарий-профиль `$PROFILE`.

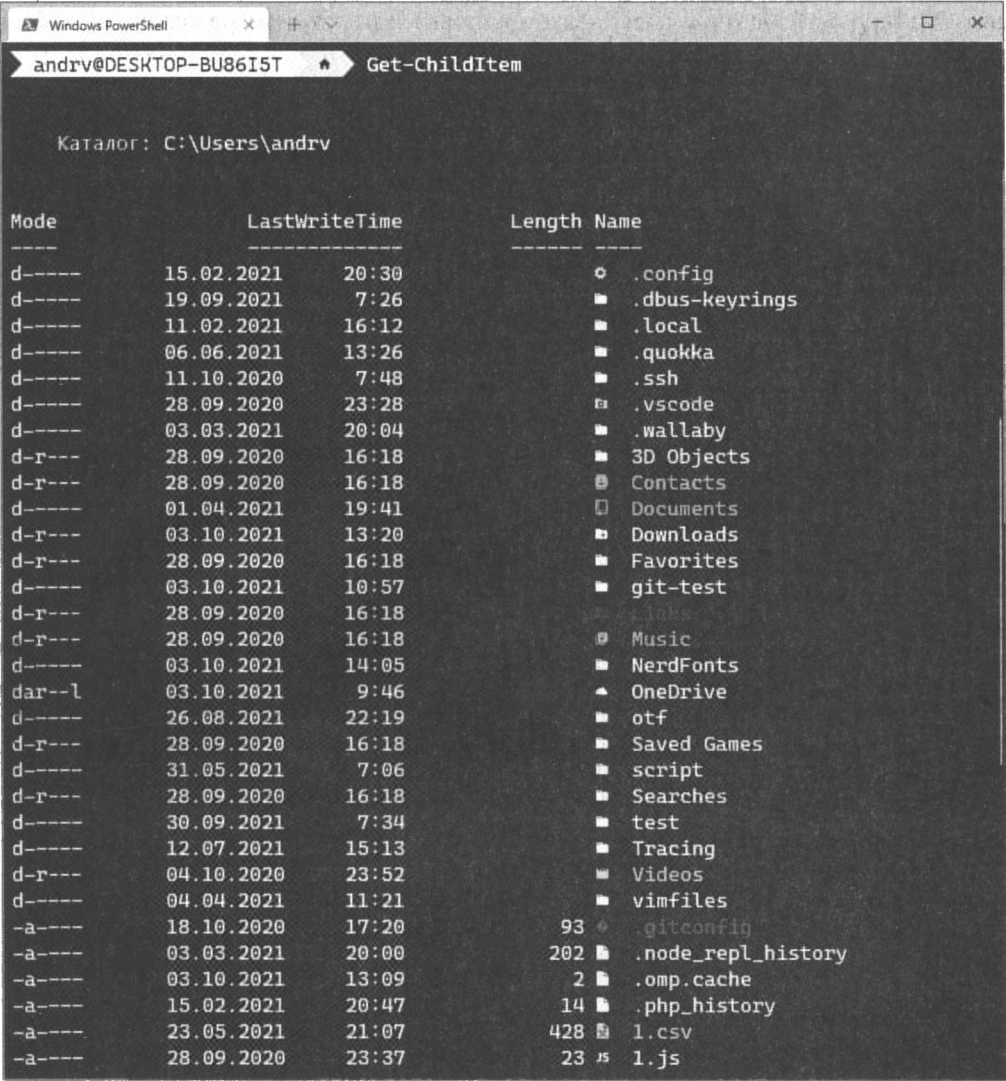


Рис. ПЗ.6. Список файлов и каталогов после подключения модуля Terminal-Icons

Предметный указатель

.

.NET-объекты и классы

- ◇ System.Convert 107
- ◇ System.DateTime 107
- ◇ System.Math 46, 106
- ◇ System.String 107
- ◇ System.TimeSpan 107
- ◇ Windows.Forms.Form 332

A

ActiveX, технология 266

API-ориентированная архитектура 21

C

camelCase 307

COM-объекты

- ◇ Excel.Application 268
 - ◇ Word.Application 268
 - ◇ WScript.Shell 243
- CSV 245

D

Dot-sourcing 196

E

Escape-последовательности 128

H

HTTP 270

J

JSON 250, 279

M

Microsoft Excel, объектная модель 267

- ◇ объект Application 267
- ◇ объект Selection 267
- ◇ объект Workbook 267
- ◇ объект Workbooks 267

Microsoft Word, объектная модель

- ◇ объект Application 267
- ◇ объект Document 267
- ◇ объект Documents 267
- ◇ объект Selection 267

O

OLE, технология

- ◇ OLE Automation 266

P

PascalCase 306

POST 279

PowerShell 6/7 43

PowerShell ISE 285

- ◇ запуск кода 285
- ◇ отладка 292
- ◇ редактирование 289

V

Visual Studio Code 294

- ◇ командная панель 297
- ◇ линтер 296

- ◇ отладка 297
- ◇ расширение для PowerShell 294
- ◇ сниппеты 296

W

- Windows Console (ConHost) 35
- Windows Forms (WinForms) 331
- Windows PowerShell
 - ◇ заголовок командного окна 84
 - ◇ приглашение командной строки 85

- Windows Presentation Foundation (WPF) 335
- Windows Script Host (WSH) 25
- Windows Terminal 36
- WMI-объекты
 - ◇ Win32_Process 260
 - ◇ Win32_Service 266

X

- XAML 335

A

- Автоматическое завершение команд 68
- Ассоциативные массивы 144
- Атрибуты
 - ◇ AllowEmptyString() 180
 - ◇ AllowNull() 180
 - ◇ CmdletBinding() 189
 - ◇ ValidateCount() 180
 - ◇ ValidateLength() 136, 180
 - ◇ ValidateNotNull() 180
 - ◇ ValidatePattern() 180
 - ◇ ValidateRange() 136, 180
 - ◇ ValidateScript() 180
 - ◇ ValidateSet() 180

Б

- Библиотека типов объекта 240

Д

- Диски PowerShell 57

Ж

- Журнал команд *См. Журнал сеанса*
- Журнал сеанса 78

К

- Командлет 47
- Командная оболочка 34

- Командная строка
 - ◇ вложенная 319
- Командно-сценарные языки 17
- Команды PowerShell
 - ◇ Add-Content 231
 - ◇ Add-History 79
 - ◇ ConvertFrom-CSV 249, 250
 - ◇ ConvertTo-CSV 249, 250
 - ◇ ConvertTo-Html 117
 - ◇ Copy-Item 231
 - ◇ Disable-PSBreakPoint 322
 - ◇ Enable-PSBreakPoint 322
 - ◇ Export-Alias 56
 - ◇ Export-CSV 248
 - ◇ Find-Module 202
 - ◇ ForEach-Object 103
 - ◇ Format-Custom 110
 - ◇ Format-List 110
 - ◇ Format-Table 110
 - ◇ Format-Wide 110
 - ◇ Get-Alias 53
 - ◇ Get-ChildItem 60, 223
 - ◇ Get-Command 51
 - ◇ Get-Content 229
 - ◇ Get-ExecutionPolicy 89, 192
 - ◇ Get-Help 71, 74
 - ◇ Get-History 78
 - ◇ Get-Host 83
 - ◇ Get-Location 60
 - ◇ Get-Member 93
 - ◇ Get-Process 92, 254
 - ◇ Get-PSBreakPoint 322, 327

Команды PowerShell (прод.)

- ◇ Get-PSDrive 57
- ◇ Get-PSProvider 58
- ◇ Get-PSRepository 201
- ◇ Get-Service 95, 262
- ◇ Get-Verb 173
- ◇ gm 93
- ◇ Group-Object 104
- ◇ Import-Alias 56
- ◇ Import-CSV 246
- ◇ Invoke-History 79
- ◇ Invoke-Item 118, 259
- ◇ Invoke-RestMethod 280
- ◇ Invoke-WebRequest 270
- ◇ kill 258
- ◇ Measure-Object 105
- ◇ Move-Item 236
- ◇ New-Item 226
- ◇ New-Object 241
- ◇ New-PSDrive 62
- ◇ Out-Default 113
- ◇ Out-File 114
- ◇ Out-GridView 115
- ◇ Out-Host 113
- ◇ Out-Null 115
- ◇ Out-Printer 114
- ◇ Read-Host 314
- ◇ Remove-Item 236
- ◇ Remove-PSBreakPoint 322, 329
- ◇ Rename-Item 235
- ◇ Restart-Service 264
- ◇ Select-Object 100
- ◇ Select-String 237
- ◇ Set-Alias 55
- ◇ Set-Content 230
- ◇ Set-ExecutionPolicy 89, 192
- ◇ Set-Location 60, 222
- ◇ Set-PSBreakPoint 322, 326
- ◇ Set-PSDebug 315
- ◇ Set-Service 265
- ◇ Sort-Object 98
- ◇ Start-Process 259
- ◇ Start-Service 264
- ◇ Start-Transcript 80
- ◇ Stop-Process 258
- ◇ Stop-Service 263
- ◇ Stop-Transcript 80
- ◇ Suspend-Service 263
- ◇ Test-Path 87, 130

- ◇ Uninstall-Module 204
- ◇ Update-Module 204
- ◇ Where-Object 95
- ◇ Write-Debug 314
- ◇ Write-Error 120
- ◇ Write-Host 313
- ◇ Write-Verbose 120
- Композиция команд 91
- Конвейеризация команд 23
- Консоль 33

Л

- Литерал 158

М

- Массивы 138
 - ◇ доступ к элементам 139
 - ◇ оператор присваивания 142
 - ◇ сохранение вывода командлетов 143
 - ◇ увеличение длины 141
 - ◇ удаление массивов 143
 - ◇ удаление элементов 142
- Метасимвол 158
- Модули
 - ◇ oh-my-posh 358
 - ◇ posh-git 355
 - ◇ PowerShell Gallery 199, 201
 - ◇ PSReadLine 352
 - ◇ Terminal-Icons 362
 - ◇ модули-сценарии 199
 - ◇ форматирования 109

О

- Объекты в PowerShell
 - ◇ анализ структуры 93
 - ◇ выделение свойств 100
 - ◇ выполнение действий в конвейере 103
 - ◇ группировка 104
 - ◇ измерение характеристик 105
 - ◇ сортировка 98
 - ◇ фильтрация 95
- Операторы
 - ◇ арифметические 149
 - ◇ логические 97, 159
 - ◇ присваивания 154
 - ◇ проверки на соответствие шаблону 157
 - ◇ сравнения 96, 155

Ошибки

- ◇ критические 206, 214, 215
- ◇ некритические 206, 214
- ◇ объект ErrorRecord 207

П

Параметры командлетов 48

- ◇ общие 50

Параметры функций 175

- ◇ валидация 179
- ◇ дополнительные атрибуты 179
- ◇ значения по умолчанию 178
- ◇ оператор Param 182
- ◇ переключатели 181
- ◇ типы 177

Переменные PowerShell

- ◇ \$\$ 131
- ◇ \$? 131, 213, 214
- ◇ \$^ 132
- ◇ \$_ 132
- ◇ \$Args 132, 173
- ◇ \$DebugPreference 121, 132
- ◇ \$Error 132, 210
- ◇ \$ErrorActionPreference 121, 132, 214
- ◇ \$Home 132
- ◇ \$Host 132
- ◇ \$InformationPreference 121
- ◇ \$Input 132, 186
- ◇ \$LASTEXITCODE 214
- ◇ \$MaximumAliasCount 132
- ◇ \$MaximumDriveCount 132
- ◇ \$MaximumErrorCount 210
- ◇ \$MaximumFunctionCount 132
- ◇ \$MaximumHistoryCount 132
- ◇ \$MaximumVariableCount 132
- ◇ \$NestedPromptLevel 320
- ◇ \$Null 130
- ◇ \$Profile 87
- ◇ \$PSHome 94, 132
- ◇ \$PSVersionTable 349
- ◇ \$StackTrace 132
- ◇ \$Transcript 81
- ◇ \$VerbosePreference 121, 132
- ◇ \$WarningPreference 132
- ◇ дополнительные атрибуты 136
- ◇ константы 136
- ◇ область видимости 197
- ◇ переменные оболочки 131

Переменные окружения См. Переменные среды Windows

Переменные среды Windows 137

Переименование устройств ввода/вывода 22

Перенаправление вывода 44, 113, 121, 122

Платформа .NET Framework 29

Политики выполнения 88

Потоки PowerShell 120

Приведение типов 134, 135

Провайдеры PowerShell 58

Протоколирование действий 80

Профили PowerShell 87

Псевдонимы команд 53

Псевдонимы типов 134

Псевдотерминал 34

С

Сборка .NET 331

Системный путь 137

Службы 261

Сниппет 289

Сплаттинг переменных 183

Справочная система PowerShell 71

Стандартные текстовые потоки 22

Статические методы 106

Строки

- ◇ в двойных кавычках 127
 - ◇ в одинарных кавычках 127
 - ◇ типа here-string 129
- Суффиксы-множители 126
- Сценарии PowerShell 53, 191
- ◇ выход 195
 - ◇ запуск 192
 - ◇ передача аргументов 194
 - ◇ создание 191

Т

Терминал 32

Точка останова 321

У

Управляющие инструкции

- ◇ Break 165
- ◇ Continue 165
- ◇ Exit 195
- ◇ If ... ElseIf ... Else 160
- ◇ Switch 166
- ◇ Trap 215
- ◇ try/catch/finally 218

Ф

Форматирование вывода 110

Функции 52, 172

◊ область видимости 195

◊ расширенные 189

Х

Хэш-таблица 102, 144

Ц

Циклы

◊ Do ... While 162

◊ For 162

◊ Foreach

▫ вне конвейера команд 163

▫ внутри конвейера команд 164

◊ While 161

Ш

Шаблоны

◊ с подстановочными символами 157

◊ с регулярными выражениями 158

Современный PowerShell

Попов Андрей Владимирович,
специалист ИТ-поддержки
и преподаватель с 25-летним стажем,
автор нескольких книг по технологиям
сценариев Windows, в том числе
«Введение в Windows PowerShell»
о первой версии оболочки и среды
выполнения Microsoft PowerShell.

Книга предназначена для пользователей и администраторов Windows, которые хотят узнать о возможностях оболочки Windows PowerShell и использовать их для автоматизации повседневных задач администрирования. Описаны базовые механизмы и основные конструкции языка PowerShell, приведены приемы создания и отладки сценариев. Даны примеры работы с файловой системой, службами и серверами автоматизации, с веб-ресурсами при помощи HTTP-запросов, подробно рассказывается о построении графического интерфейса для сценариев PowerShell. Представлены кросс-платформенные возможности PowerShell в macOS и Linux.

**Автоматизация
решения повседневных
задач администратора**

СИСТЕМНЫЙ
АДМИНИСТРАТОР



КАТЕГОРИЯ: операционные системы



191036, Санкт-Петербург,
Гончарная ул., 20
Тел.: (812) 717-10-50,
339-54-17, 339-54-28
E-mail: mail@bhv.ru
Internet: www.bhv.ru